



Ultra Accelerator Link Consortium, Inc.  
Specification

UALink 200G  
Data Link and Physical Layers  
Version 2.0

## Legal Notice

### **LEGAL NOTICE FOR THIS SPECIFICATION FROM ULTRA ACCELERATOR LINK CONSORTIUM, INC.**

**© 2025 ULTRA ACCELERATOR LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.**

This UALink\_200 DL-PL Rev 2.0 Specification (this “**UALink Specification**” or this “**document**”) is owned by and is proprietary to Ultra Accelerator Link Consortium, Inc., a Delaware nonprofit corporation (sometimes referred to as “**UALink**” or the “**UALink Consortium**” or the “**Company**”) and/or its successors and assigns.

### **NOTICE TO USERS WHO ARE MEMBERS OF ULTRA ACCELERATOR LINK CONSORTIUM, INC.:**

Members of the UALink Consortium (sometimes referred to as a “**UALink Member**”) must be and remain in compliance with all of the following UALink Consortium documents, policies and/or procedures (collectively, the “**UALink Governing Documents**”) in order for such UALink Member’s use and/or implementation of this UALink Specification to receive and enjoy any of the rights, benefits, privileges, and/or protections of UALink Consortium membership, including without limiting those set forth in: (i) the UALink Consortium’s Intellectual Property Policy; (ii) the UALink Consortium’s Bylaws; (iii) any and all other UALink Consortium policies and procedures; and (iv) the UALink Member’s Participation Agreement.

### **NOTICE TO NON-MEMBERS OF ULTRA ACCELERATOR LINK CONSORTIUM, INC.:**

If you are **not** a UALink Member and you have obtained a copy of this UALink Specification, you only have a right to review this document or make reference to or cite this document. Any references or citations to this document must acknowledge the Ultra Accelerator Link Consortium, Inc.’s sole and exclusive copyright ownership of this UALink Specification. The proper copyright citation or reference is as follows: “**© 2025 ULTRA ACCELERATOR LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.**” When making any such citation or reference to this document you are not permitted to revise, alter, modify, make any derivatives of, or otherwise amend the referenced portion of this document in any way without the prior express written permission of the Ultra Accelerator Link Consortium, Inc.

Nothing contained in this UALink Specification shall be deemed as granting (either expressly or impliedly) to any party that is **not** a UALink Member: (i) any kind of license to implement or use this UALink Specification or any portion or content described or contained therein, or any kind of license in or to any other intellectual property owned or controlled by the UALink Consortium, including without limitation any trademarks of the UALink Consortium; or (ii) any benefits and/or rights as a UALink Member under any UALink Governing Documents. For clarity, and without limiting the foregoing notice in any way, if you are **not** a UALink Member but still elect to implement this UALink Specification or any portion described herein, you are hereby given notice that your election to do so does not give you any of the rights, benefits, and/or protections of the UALink Members, including without limitation any of the rights, benefits, privileges or protections given to a UALink Member under the UALink Consortium’s Intellectual Property Policy.

### **LEGAL DISCLAIMERS AND ADDITIONAL NOTICE FOR ALL PARTIES:**

THIS DOCUMENT AND ALL SPECIFICATIONS AND/OR OTHER CONTENT PROVIDED HEREIN ARE PROVIDED ON AN “**AS IS**” BASIS. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW,

ULTRA ACCELERATOR LINK CONSORTIUM, INC (ALONG WITH THE CONTRIBUTORS TO THIS DOCUMENT) HEREBY DISCLAIM ALL REPRESENTATIONS, WARRANTIES AND/OR COVENANTS, EITHER EXPRESS OR IMPLIED, STATUTORY OR AT COMMON LAW, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, VALIDITY, AND/OR NON-INFRINGEMENT.

In the event this UALink Specification makes any references (including without limitation any incorporation by reference) to another standard's setting organization's or any other party's ("**Third Party**") content or work, including without limitation any specifications or standards of any such Third Party ("**Third Party Specification**"), you are hereby notified that your use or implementation of any Third Party Specification: (i) is not governed by any of the UALink Governing Documents; (ii) may require your use of a Third Party's patents, copyrights or other intellectual property rights, which in turn may require you to independently obtain a license or other consent from that Third Party in order to have full rights to implement or use that Third Party Specification; and/or (iii) may be governed by the intellectual property policy or other policies or procedures of the Third Party which owns the Third Party Specification. Any trademarks or service marks of any Third Party which may be referenced in this UALink Specification is owned by the respective owner of such marks.

The **UALINK™** trademark, the **ULTRA ACCELERATOR LINK CONSORTIUM™** trademark, and related logos (the "**UALink Trademarks**") are trademarks owned by Ultra Accelerator Link Consortium, Inc., a Delaware corporation, and it hereby reserves all rights, title and interest in and to all of its UALink Trademarks.

## Table of Contents

|                                          |    |
|------------------------------------------|----|
| Legal Notice.....                        | 2  |
| Table of Contents.....                   | 4  |
| List of Figures.....                     | 8  |
| List of Tables.....                      | 10 |
| Revision History.....                    | 11 |
| 1 Introduction.....                      | 12 |
| 2 Data Link .....                        | 13 |
| 2.1 Overview.....                        | 13 |
| 2.2 Data Link Features .....             | 14 |
| 2.2.1 Packing Flits .....                | 14 |
| 2.2.2 DL message service .....           | 14 |
| 2.2.3 UART.....                          | 14 |
| 2.2.4 Tx Pacing, Rx Rate Adaptation..... | 15 |
| 2.3 Flit Format.....                     | 16 |
| 2.3.1 DL Flit Overview .....             | 16 |
| 2.3.2 Flit Header.....                   | 17 |
| 2.3.3 Segment Header.....                | 17 |
| 2.3.4 Flit packing rules.....            | 18 |
| 2.3.5 TL Flit to DL Flit Mapping.....    | 21 |
| 2.3.6 DL Flit to 64B/66B encoding.....   | 23 |
| 2.3.7 CRC.....                           | 23 |
| 2.4 DL messages .....                    | 24 |
| 2.4.1 Message Overview .....             | 24 |
| 2.4.2 Basic Messages .....               | 25 |
| 2.4.3 Control Messages.....              | 29 |
| 2.4.4 UART Messages.....                 | 36 |
| 2.5 Transmitter Pacing .....             | 41 |
| 2.5.1 Overview.....                      | 41 |
| 2.5.2 Switch.....                        | 41 |
| 2.5.3 Accelerator.....                   | 42 |
| 2.5.4 Sequence.....                      | 42 |
| 2.6 Link Level Replay .....              | 42 |
| 2.6.1 Overview.....                      | 42 |
| 2.6.2 Flit Header.....                   | 43 |

|        |                                        |    |
|--------|----------------------------------------|----|
| 2.6.3  | Term Definitions.....                  | 44 |
| 2.6.4  | Rx Flags and Counters.....             | 45 |
| 2.6.5  | Tx Flags and Counters.....             | 46 |
| 2.6.6  | General Rules.....                     | 47 |
| 2.6.7  | Round Trip Time.....                   | 55 |
| 2.7    | Link State and Errors.....             | 56 |
| 2.7.1  | DL Link States.....                    | 56 |
| 2.7.2  | Correctable Errors.....                | 59 |
| 2.7.3  | Uncorrectable Errors.....              | 59 |
| 2.7.4  | Error Containment.....                 | 59 |
| 2.8    | Link Resiliency.....                   | 59 |
| 2.8.1  | Introduction.....                      | 59 |
| 2.8.2  | Transmit.....                          | 60 |
| 2.8.3  | Receive.....                           | 61 |
| 2.8.4  | Reordering.....                        | 61 |
| 2.8.5  | DL Link State.....                     | 62 |
| 2.8.6  | Flit Sequences.....                    | 63 |
| 2.9    | Link Folding Operation.....            | 65 |
| 2.9.1  | Introduction.....                      | 65 |
| 2.9.2  | Link Folding.....                      | 66 |
| 2.9.3  | Link Unfolding.....                    | 67 |
| 2.9.4  | Link Folding Recovery.....             | 68 |
| 3      | Physical Layer.....                    | 70 |
| 3.1    | Introduction.....                      | 70 |
| 3.2    | Reconciliation Sublayer (RS).....      | 73 |
| 3.2.1  | Introduction.....                      | 73 |
| 3.2.2  | Data Flow.....                         | 73 |
| 3.2.3  | DL Flits.....                          | 74 |
| 3.2.4  | Control Flits.....                     | 74 |
| 3.2.5  | Data and Control Blocks and Codes..... | 76 |
| 3.2.6  | Link fault signaling.....              | 77 |
| 3.2.7  | Flit and Lane Alignment.....           | 78 |
| 3.2.8  | Receive State Machine.....             | 79 |
| 3.2.9  | Link Power down.....                   | 79 |
| 3.2.10 | Link Power up.....                     | 79 |
| 3.3    | PCS/PMA modifications.....             | 79 |

|            |                                           |     |
|------------|-------------------------------------------|-----|
| 3.3.1      | Introduction .....                        | 79  |
| 3.3.2      | DL Flit to PCS codeword alignment .....   | 80  |
| 3.3.3      | Reduced FEC interleave.....               | 80  |
| 3.3.4      | Decode Encode .....                       | 81  |
| 3.3.5      | Rate Matching .....                       | 81  |
| 3.3.6      | Back-to-Back DL Flits .....               | 82  |
| 3.3.7      | Rapid Alignment Markers.....              | 82  |
| 3.3.8      | PMA Symbol Pair Demultiplexing .....      | 84  |
| 3.4        | PCS and FEC for 100GBASE-R .....          | 84  |
| 3.4.1      | Removed functional Blocks.....            | 85  |
| 3.4.2      | Transmit Function .....                   | 86  |
| 3.4.3      | Receive Function .....                    | 86  |
| 3.5        | PCS for 200GBASE-R and 400GBASE-R.....    | 88  |
| 3.5.1      | Transmit Function .....                   | 88  |
| 3.5.2      | Receive Function .....                    | 89  |
| 3.6        | PCS for 800GBASE-R .....                  | 90  |
| 3.6.1      | Transmit Function .....                   | 91  |
| 3.6.2      | Receive Function .....                    | 92  |
| 3.7        | Low Latency FEC Interleave.....           | 93  |
| 3.7.1      | 400GBASE-KR2/CR2 (2-way interleave) ..... | 93  |
| 3.7.2      | 200GBASE-KR1/CR1 (2-way interleave) ..... | 93  |
| 3.7.3      | 200GBASE-KR1/CR1 (1-way interleave) ..... | 93  |
| 3.8        | Link Resiliency.....                      | 96  |
| 3.8.1      | Transmit Alignment .....                  | 96  |
| 3.8.2      | Physical Layer ID.....                    | 96  |
| 4          | Appendices.....                           | 97  |
| Appendix A | DL CRC Example (informative) .....        | 97  |
| Appendix B | DL to PL Interface (informative) .....    | 98  |
| B.1        | Tx Signals .....                          | 98  |
| B.2        | Tx Waveforms.....                         | 100 |
| B.3        | Tx TDM Initialization .....               | 102 |
| B.4        | Rx Signals .....                          | 103 |
| B.5        | Rx Waveforms.....                         | 104 |
| B.6        | Start Up Waveform .....                   | 105 |
| B.7        | Link Event and Error Timing.....          | 106 |

Evaluation Copy

## List of Figures

|                                                              |    |
|--------------------------------------------------------------|----|
| Figure 2-1 DL block Diagram.....                             | 13 |
| Figure 2-2 UART.....                                         | 15 |
| Figure 2-3 DL 640-Byte Flit Overview.....                    | 17 |
| Figure 2-4 DL Flit with segment details.....                 | 19 |
| Figure 2-5 Flit packing flow chart.....                      | 21 |
| Figure 2-6 TL Flit[0] example 1.....                         | 22 |
| Figure 2-7 TL Flit[1] example 1.....                         | 22 |
| Figure 2-8 TL Flit[0] example 2.....                         | 23 |
| Figure 2-9 DL Flit to 64B/66B Encoding.....                  | 23 |
| Figure 2-10 Single Request Flow.....                         | 25 |
| Figure 2-11 Two Requests Flow.....                           | 26 |
| Figure 2-12 Single Successful Request Flow.....              | 30 |
| Figure 2-13 Single Unsuccessful Request Flow.....            | 30 |
| Figure 2-14 Single decision pending Request Flow.....        | 31 |
| Figure 2-15 Conflicting request flow.....                    | 32 |
| Figure 2-16 Identical Request Flow.....                      | 32 |
| Figure 2-17 Vendor Defined Packet.....                       | 38 |
| Figure 2-18 Pacing.....                                      | 41 |
| Figure 2-19 Ack Replay Request valid range.....              | 49 |
| Figure 2-20 Rx Flow Chart.....                               | 52 |
| Figure 2-21 Tx Flow Chart.....                               | 53 |
| Figure 2-22 Round Trip Time.....                             | 55 |
| Figure 2-23 Link State with and without Link Resiliency..... | 56 |
| Figure 2-24 DL Link State.....                               | 57 |
| Figure 2-25 Link Resiliency.....                             | 60 |
| Figure 2-26 Transmit Resiliency.....                         | 60 |
| Figure 2-27 Receive Resiliency.....                          | 61 |
| Figure 2-28 Transmit Offset.....                             | 62 |
| Figure 2-29 Initial Received Order.....                      | 62 |
| Figure 2-30 After reorder.....                               | 62 |
| Figure 2-31 Reorder with DL Flits.....                       | 62 |
| Figure 2-32 DL “sub” State Machines.....                     | 63 |
| Figure 2-33 Initial Link Up to DL NOP.....                   | 63 |
| Figure 2-34 Initial Link Up to DL Up.....                    | 64 |
| Figure 2-35 B Receives Local Fault.....                      | 64 |
| Figure 2-36 B Receives Remote Fault.....                     | 64 |
| Figure 2-37 Fast Recovery.....                               | 64 |
| Figure 2-38 Link Down Recovery to DL NOP.....                | 64 |
| Figure 2-39 Link Down Recovery to DL Up.....                 | 65 |
| Figure 2-40 Lik Folding.....                                 | 66 |
| Figure 2-41 Link Folding Ladder Diagram.....                 | 67 |
| Figure 2-42 Link Folding Sequence to PwrDn.....              | 67 |
| Figure 2-43 Link Folding Sequence to Off.....                | 67 |
| Figure 2-44 Link Unfolding in DL Fault.....                  | 68 |
| Figure 2-45 Link Unfolding to DL Up.....                     | 68 |
| Figure 2-46 Link Folding Error.....                          | 68 |
| Figure 2-47 Link Folding Error Recovery to DL NOP.....       | 69 |
| Figure 2-48 Link Folding Error Recovery to DL Up.....        | 69 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| Figure 3-1 Physical Layer Block Diagram.....                     | 71  |
| Figure 3-2 64B/66B Block Codes.....                              | 77  |
| Figure 3-3 RAM to AM transition.....                             | 83  |
| Figure 3-4 100GBASE-R.....                                       | 85  |
| Figure 3-5 200GBASE-R & 400GBASE-R.....                          | 88  |
| Figure 3-6 800GBASE-R.....                                       | 91  |
| Figure 3-7 200GBASE-KR1/CR1, 1-Way Interleave.....               | 95  |
| Figure 3-8 Code Word Alignment.....                              | 96  |
| Figure Appendix B-1 Tx TDM timing qualified by valid .....       | 100 |
| Figure Appendix B-2 Tx TDM timing (not qualified by valid).....  | 100 |
| Figure Appendix B-3 Active Assertion Timing .....                | 101 |
| Figure Appendix B-4 Active de-assertion.....                     | 101 |
| Figure Appendix B-5 DL Enters PwrDn State .....                  | 102 |
| Figure Appendix B-6 PL Local Fault Detected .....                | 102 |
| Figure Appendix B-7 Rx TDM timing qualified by valid .....       | 105 |
| Figure Appendix B-8 Rx TDM timing (not qualified by valid) ..... | 105 |
| Figure Appendix B-9 Start Up Timing.....                         | 106 |
| Figure Appendix B-10 Link Event and Error .....                  | 106 |

## List of Tables

|                                                              |     |
|--------------------------------------------------------------|-----|
| Table 2-1 Sector Allocation per Segment .....                | 16  |
| Table 2-2 Segment Header .....                               | 18  |
| Table 2-3 DL Message Types.....                              | 24  |
| Table 2-4 TL Rate Notification .....                         | 26  |
| Table 2-5 Device ID Request.....                             | 27  |
| Table 2-6 Port ID Request.....                               | 28  |
| Table 2-7 No-Op Message .....                                | 29  |
| Table 2-8 Tx Ready Notification.....                         | 29  |
| Table 2-9 Channel Negotiation .....                          | 34  |
| Table 2-10 Link Width Negotiation .....                      | 36  |
| Table 2-11 Vendor Defined Packet Type Length (TL) DWord..... | 38  |
| Table 2-12 UART Stream Reset Request.....                    | 39  |
| Table 2-13 UART Stream Reset Response.....                   | 39  |
| Table 2-14 UART Stream transport message.....                | 40  |
| Table 2-15 UART Stream Credit Update .....                   | 40  |
| Table 2-16 Explicit Sequence Number Flit Header .....        | 44  |
| Table 2-17 Command Flit Header.....                          | 44  |
| Table 2-18 Tx_ack_time_out defaults .....                    | 47  |
| Table 2-19 Skew budget .....                                 | 61  |
| Table 3-1 100Gbps Serial Clauses.....                        | 72  |
| Table 3-2 200Gbps Serial Clauses.....                        | 72  |
| Table 3-3 Sequential Ordered Sets.....                       | 77  |
| Table 3-4 Reduced Interleave FEC .....                       | 80  |
| Table 3-5 RAM period.....                                    | 82  |
| Table Appendix B-1 Transmit DL to PL Data Path Signals.....  | 99  |
| Table Appendix B-2 DL to PL Data Path Signals .....          | 104 |

## Revision History

| Rev                    | Date       | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UALink 200G 2.0, DL/PL |            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 0.7                    | 2025.11.12 | <p>Initial Version based UALink 200G 1.0 specification, with the following additions:</p> <p>ECN DL Down Delay, September 17, 2025<br/>           ECR Link Resiliency, November 4, 2025<br/>           ECR Link Folding, November 4, 2025<br/>           ECR Back Pressure from the TL, September 30, 2025<br/>           ECR DL PL interface October 20, 2025<br/>           ECR CRC Example calculation September 22, 2025</p> <p>Resolved Errata Batch 1: A103001, B103002, B103003, B103004, A103006, B103007, B103008, B103009.</p> <p>Resolved Errata Batch 2: A103010, B103011, A103012, B103013, A103014, B103015, B103016, C103017, C103019, C103020, A103021, A103023, B103024, C103025, C103026, B103027, B103028, B103029, A103030, <del>B103031</del>, <del>B103032</del>, <del>B103033</del>, A103034.</p> |
| 1.0 RC                 | 2026.1.6   | <p>Fixed broken cross reference</p> <p>Change release to 1.5, from 1.x.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 1.0 RC                 | 2026.1.12  | Fixed header to be 200G (not 128G)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 1.0 RC1                | 2026.3.3   | Corrected miss applied B103003.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1.0 RC                 | 2026.3.10  | Renamed from 1.5 to 2.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## 1 Introduction

UALink supports different physical Layer technologies. This document describes the UALink variant based on IEEE 802.3dj Ethernet. IEEE 802.3dj supports a per lane top signaling rate for 200Gbs, PAM4. The per lane signaling rate at 100Gbs is also supported.

The data link is unique for UALink 200G. The Flit size used is 640-Bytes to match the 680-byte code word size defined in 802.3 Clause 91, 119, and 172. Link level replay is not part of the 802.3 physical layer; therefore, the data link supports replay.

The station supports 4 lanes at 200Gbs per lane. The station's bandwidth is therefore 800Gbs.

## 2 Data Link

### 2.1 Overview

The Block diagram is shown below. The Data Link sits between the Transaction layer and the Physical Layer. The Data Link packs 64-byte Flits from the transaction layer into 640 Bytes Flits for the Physical Layer. The Data Link also provides a message service between link partners that originates and terminates at the Data Link layer. The message service is used for advertising the Transaction Layer rate, querying device and port ID on connected Link Partner, and other functions. The message service also provides for a UART style communication between link partners, intended for F/W communications. Link level replay is provided on a 640 Byte Flit basis. A 32-bit CRC is computed and checked and is part of the 640 Byte Flit.

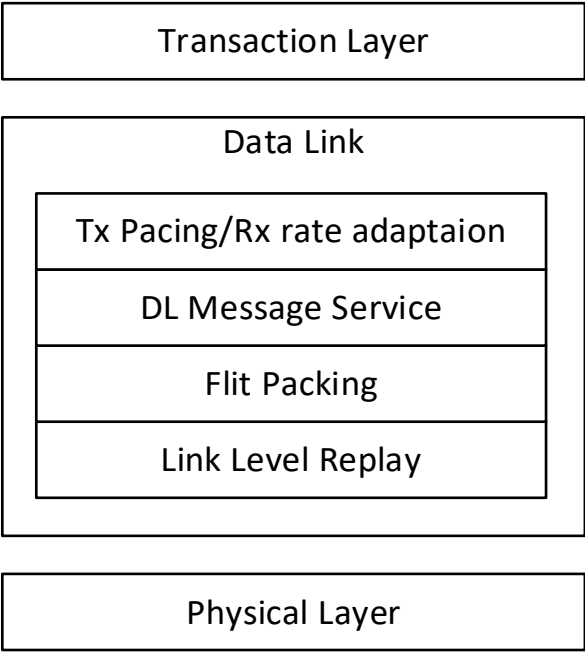


Figure 2-1 DL block Diagram

## 2.2 Data Link Features

### 2.2.1 Packing Flits

The main function of the Data Link in the egress direction is to pack 64-byte Flits from the Transaction Layer into 640-byte DL Flits and unpack 640-byte DL Flits into 64-byte Flits for the transaction layer in the ingress direction. DL messages are also packed into DL Flits.

### 2.2.2 DL message service

In band DL to DL messages are supported via alternate segments that are packed into the 640-byte DL Flit payload, along with the TL Flits data. DL to DL messages provide support for the following:

- F/W message sequencing via UART
- Negotiating Channel online/offline.
  - Channel 0 For TL Flits
  - Channel 4 For DL UART messages
- Advertising transaction layer rates
- Requesting Link Partners Device ID and port number

### 2.2.3 UART

The UART provides a communication path between firmware agents on both sides of a given link. Hardware driven credit-based flow control is provided across the link via DL messages to prevent Rx buffer overflow.

The initial use case for the UART is to enable vendor defined communication between link partners. Security may also make use of the UART. Future version may define additional standard-based communication.

See Figure 2-2, Firmware writes to the UART Tx buffer, and the data is inserted into the DL Flit and transmitted to the link partner. The receiving DL removes the data from the DL Flit and places it in the UART Rx buffer. Firmware on the receiving side reads the data.

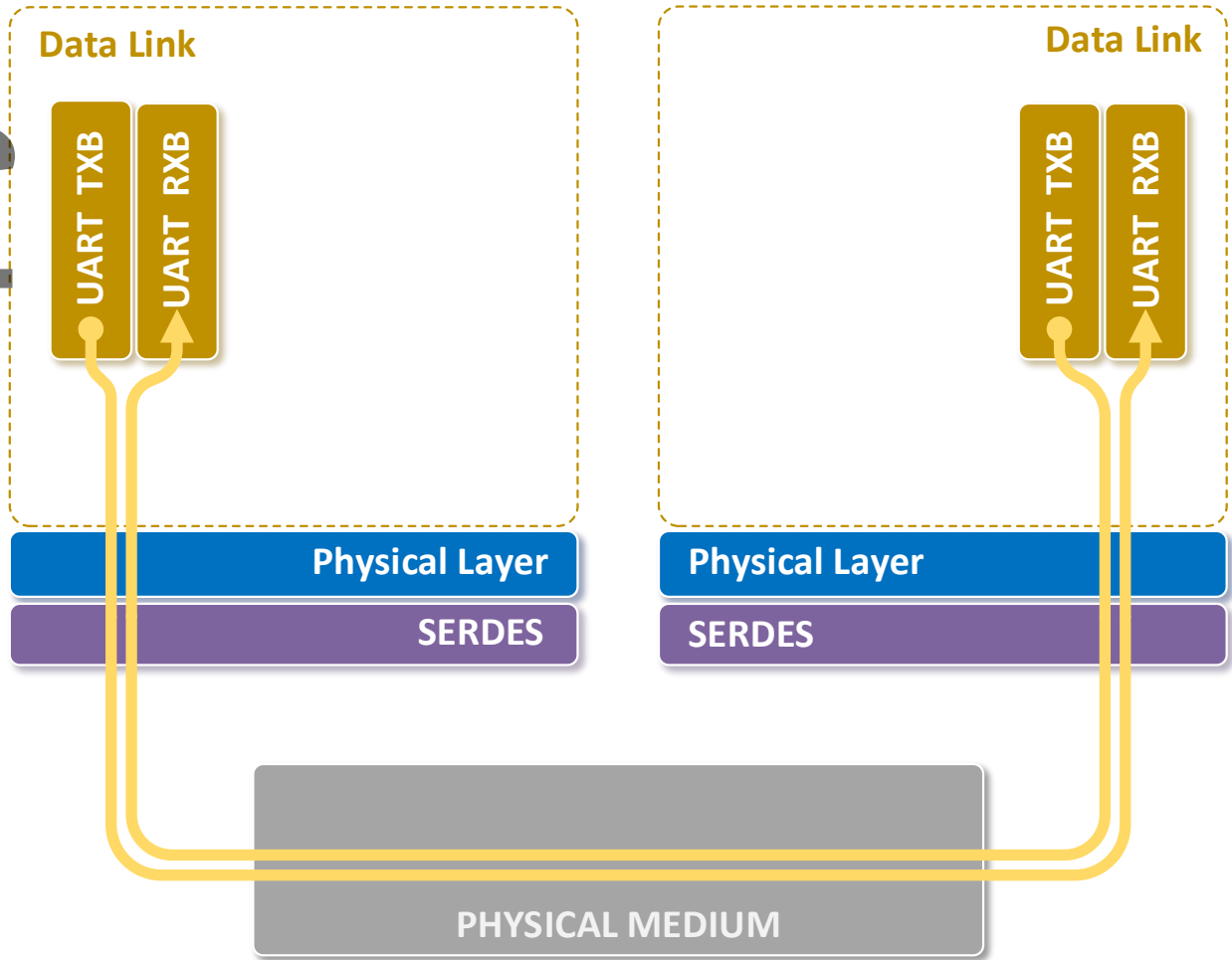


Figure 2-2 UART

#### 2.2.4 Tx Pacing, Rx Rate Adaptation

Tx Pacing provides a mechanism for a link partner to limit the transmission rate of TL Flits from its link partner. This enables Accelerators to operate their UPLI Clock at lower than line rate and not overflow its Rx FIFO in the Rx rate adaption logic.

## 2.3 Flit Format

### 2.3.1 DL Flit Overview

The 640B Flit is comprised of five segments. Each segment is comprised of an integer number of sectors. A sector is 4 bytes. The number of sectors per segment varies based on header and CRC placement in the segment and is described below. The half segment allocation is also described and defines how far to zero fill when there is no TL Flit to pack, see Figure 2-5.

| Segment Header | Number of payload Sectors | Number of payload bytes | Half Segment Sector ranges |
|----------------|---------------------------|-------------------------|----------------------------|
| SH0            | 32 Sectors                | 128-bytes               | [0:15], [16:31]            |
| SH1            | 32 Sectors                | 128-bytes               | [32:47], [48:63]           |
| SH2            | 32 Sectors                | 128-bytes               | [64:79],[80:95]            |
| SH3            | 31 Sectors                | 124-bytes               | [96:111],[112:126]         |
| SH4            | 30 Sectors                | 120-bytes               | [127:142],[143:156]        |

**Table 2-1 Sector Allocation per Segment**

The FH[2:0] fields contain the Flit Header information. This indicates the type of Flit, and sequence number to aid in link level replay, and other information. This is described in detail in section 2.3.2.

The segment header (SH) defines the starting content for that segment.

The 4-byte CRC is calculated over the entire contents of the DL Flit.

The diagram below describes the placement different non data fields of the DL Flit:

- FH[2:0] Flit header
- SH[4:0] segment headers
- CRC
- Segment payload

A DL NOP Flit has all the segment headers set to 0 and all segment payloads set to 0.

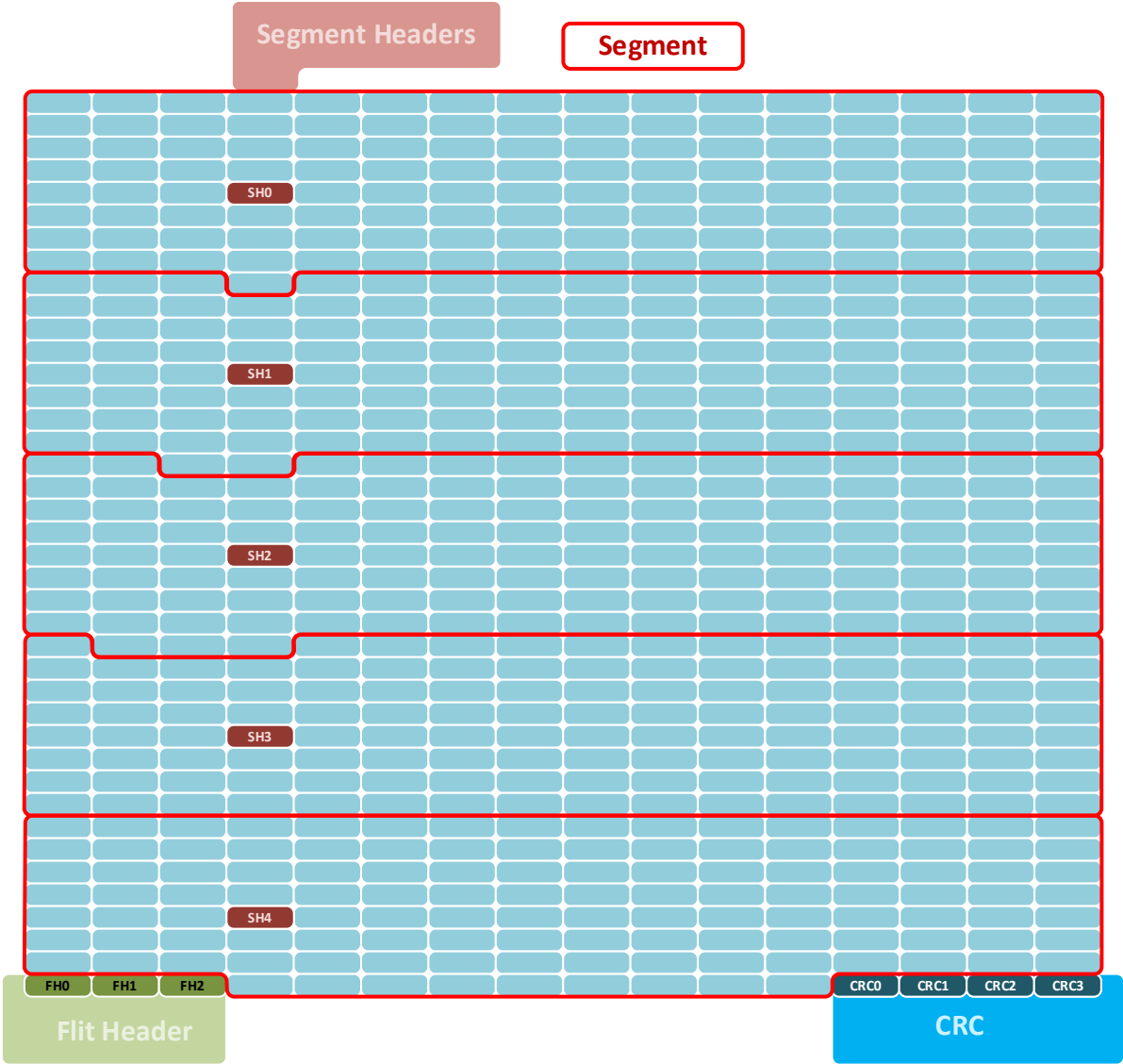


Figure 2-3 DL 640-Byte Flit Overview

### 2.3.2 Flit Header

See section 2.6.2.

### 2.3.3 Segment Header

The segment header describes the TL Flit sequence that starts in the segment. Up to two 64-byte TL Flits are contained in a TL Flit sequence. The Segment payload may be less than the 128-bytes, and some segments may contain alternative sectors, therefore some of the TL Flit sequence will carry over into the next segment.

The segment header is 8 bits. The field encodings are shown below. TL Flit[0] and Message[0], indicate the presence of a TL Flit and message data associated with the first TL Flit that is packed into the segment, if present, see Figure 2-5. TL Flit[1] and Message[1] are associated with the second TL Flit if present.

| Field Name | Position | Description                                                                                                    |
|------------|----------|----------------------------------------------------------------------------------------------------------------|
| DAltSector | [0]      | DL Alternative sector<br>0b-No alternative sector in segment<br>1b-DL alternative sector in segment            |
| Reserved   | [1]      | Reserved                                                                                                       |
| Message[0] | [3:2]    | TL Flit[0] Message bit indicators<br>if TL Flit[0] is not present then reserved<br>else Message bit indicators |
| TL Flit[0] | [4]      | TL Flit[0] present<br>0b - TL Flit[0] not present<br>1b - TL Flit[0] present                                   |
| Message[1] | [6:5]    | TL Flit[1] Message bit indicators<br>if TL Flit[1] is not present then reserved<br>else Message bit indicators |
| TL Flit[1] | [7]      | TL Flit present<br>0b - TL[1] Flit not present<br>1b - TL[1] Flit present                                      |

Table 2-2 Segment Header

**2.3.3.1 DL Alternative Sector**

When the DAltSector bit is set, this indicates that the segment contains an alternative sector. A sector is 4 bytes. The Alternative sectors are used for carrying DL-DL messages, see section 2.3.4.

**2.3.3.2 TL Flit Present**

When this bit is set it indicates that there is TL Flit that starts in this segment.

**2.3.3.3 Message**

When there is TL Flit that starts in this segment, this field contains the 2-bit message bits with it, 1-bit for each  $\frac{1}{2}$  TL Flit. The message data is simply meta data that is carried transparently over the DL, with the associated TL Flit.

**2.3.4 Flit packing rules**

The diagram below describes the DL Flit field locations including segment payload details. The Sx.By describes the sector number in the segment (Sx), and the byte number in the segment(By).

# Ultra Accelerator Link Consortium Inc. (UALink) – UALink 200G DL/PL 2.0

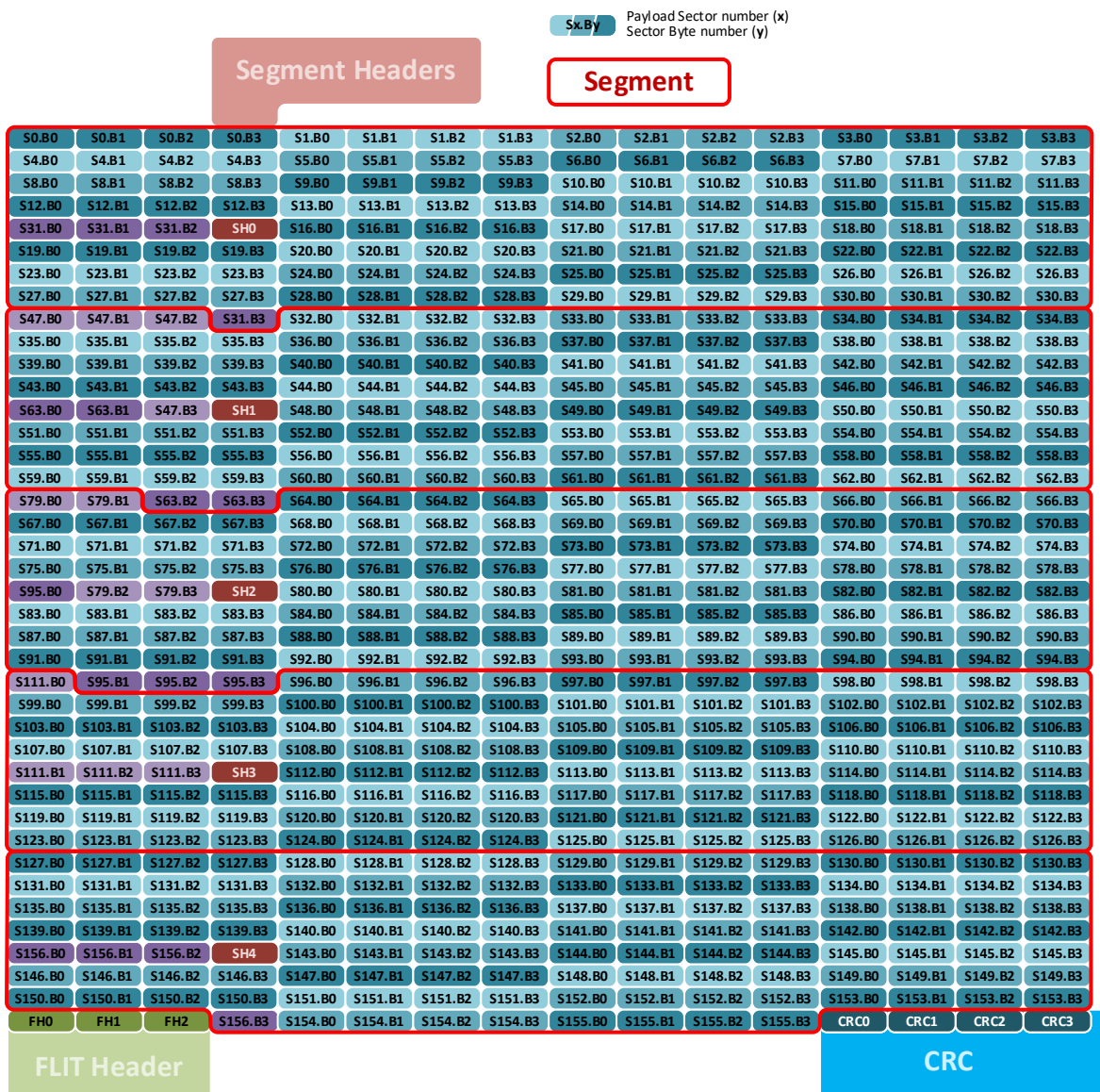


Figure 2-4 DL Flit with segment details

The following describe the Flit packing rules.

- The DL Flit packer operates on a sector basis, a sector is 4 bytes.
- DL Alternative sectors have priority, if an DL alternative sector is indicated then it is placed in the first sector of the segment, before the current TL Flit, or carry over from a previous TL Flit. The first sector is the lowest sector number in the segment see Table 2-1.
- A TL Flit takes up sixteen sectors (64-bytes).
- A DL message (Alternative sectors) takes up one sector (4-bytes).
- There shall be at least one unallocated sector in the segment to start adding a TL Flit.
- If a TL Flit does not pack into the current segment the remainder is carried over to the next segment.
- The SH is encoded as 0x00 when no TL Flits and no DL message start in the segment. If all SHs are 0x00 the DL Flit is a payload Flit. It shall not be converted to a NOP Flit.
- TL Flits shall be packed in the order received.

- Up to 2 TL Flits start packing into a segment.
- TL Flits shall be packed on the fly to reduce transmitter latency, the first tick of the packer may not have a TL Flit[0] to pack, the remaining ½ segment is zero filled, the 2<sup>nd</sup> tick of the packer may have a TL Flit[1] available, and that starts to pack, if sectors are available. See Table 2-2 for ½ segment definition.
- In the special case where the first ½ segment is filled with a previous TL Flit carry over and an alternative sector, and there is a TL Flit to add, it is designated TL Flit[0].
- Note: with a 512-bit (64-byte) data path, a DL Flit is packed every 10 clock ticks. DL overhead of 12 bytes (and occasional DL messages) is such that TL Flits cannot be continuously packed every 2 ticks. Other data widths are possible, the same packing behavior shall be met.

Packing flow described below (see Figure 2-5):

1. Start packing a segment.
2. If there is a DL ALT sector, it is placed in the first sector of the segment and continue.
3. If there is carry over from the previous Segment pack it in sector order and continue.
4. If TL Flit[0] is available, on this clock tick, then start packing it in sector order into the current segment and continue, else zero unallocated sectors to the end of the half segment and goto (6).
5. If TL Flit[0] completes packing continue, else save remaining TL Flit[0] in carry over and done.
6. If there are unallocated sectors continue, else done.
7. If TL Flit[1] is available, on this clock tick, then start packing it in sector order into the current segment else zero remaining unallocated sectors and done.
8. If TL Flit[1] packing is completed then done, else save remaining TL Flit[1] in carry over and done.
9. goto (1)

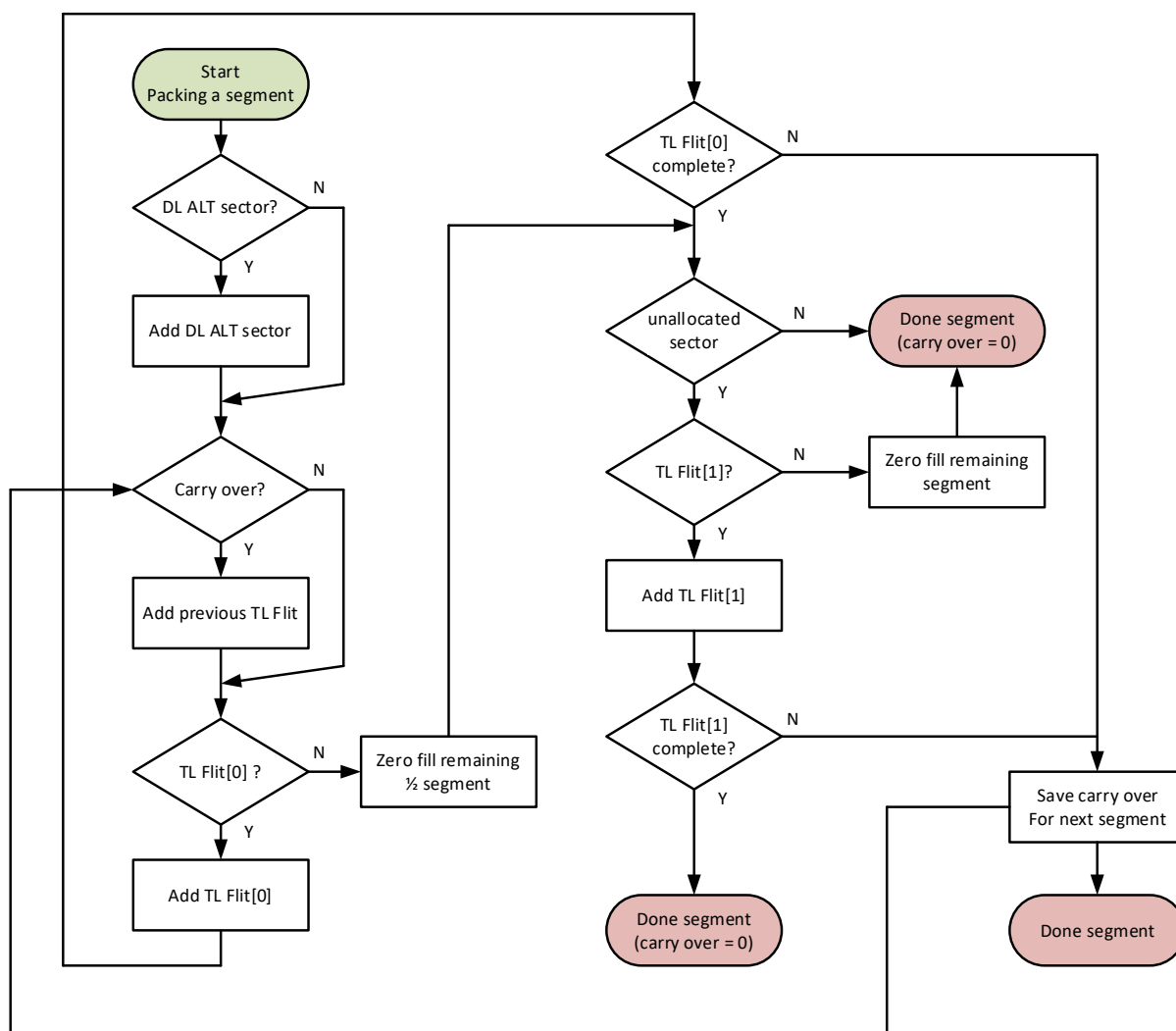


Figure 2-5 Flit packing flow chart

### 2.3.5 TL Flit to DL Flit Mapping

In the following examples the TL Flit is shown in mirrored view so that it aligns with the logical view of the DL Flit. The DL Flit sequential ordering is left to right, top to bottom, i.e., in the order of sector and byte numbering.

Figure 2-6 describes that TL Flit[0] starts packing into segment[0]. The message bits[1:0] along with TL Flit[0] present is encoded into SH0. TL Flit sector[0] maps to DL Flit sector[0], and so on.

In this example there is no carry over from a previous TL Flit.

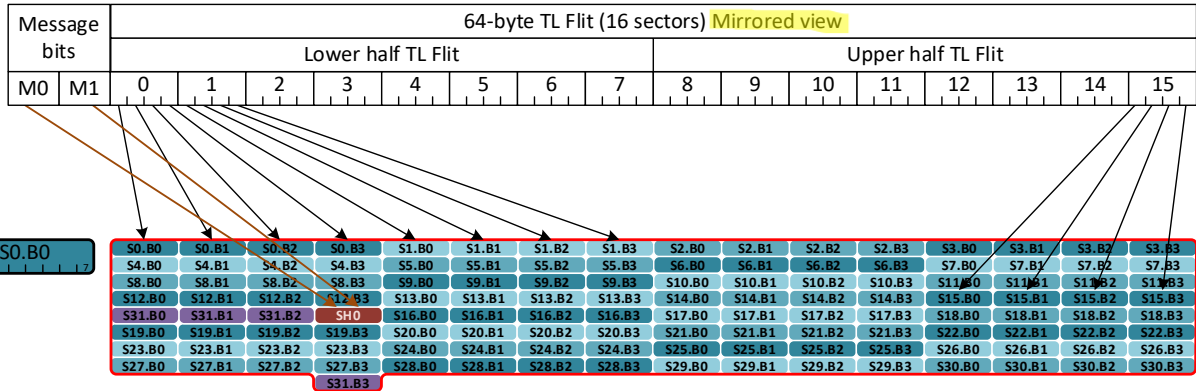


Figure 2-6 TL Flit[0] example 1

Figure 2-7 describes that TL Flit[1] starts packing into segment[0]. The message bits[1:0] along with TL Flit[1] present is encoded into SH0. TL Flit sector[0] maps to DL Flit sector[16], and so on.

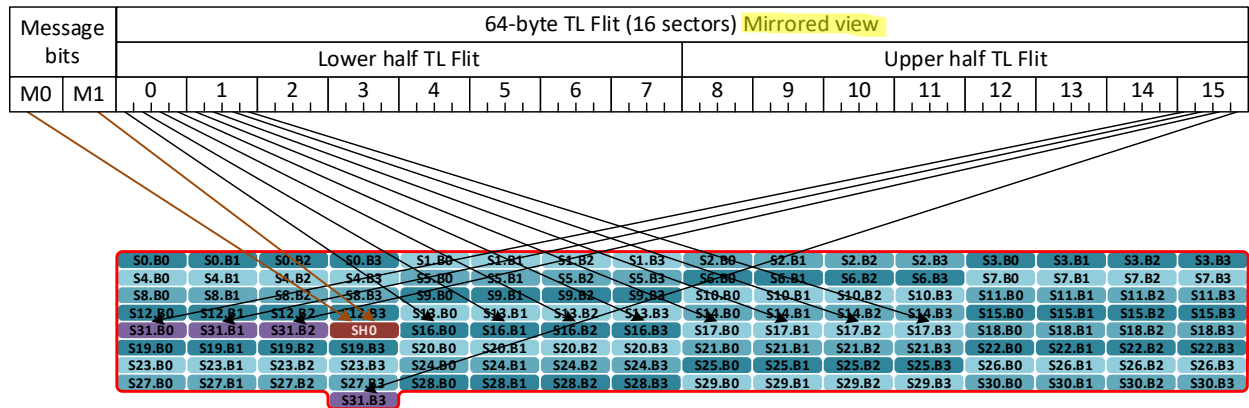


Figure 2-7 TL Flit[1] example 1

Figure 2-8 describes that TL Flit[0] starts packing into segment[4]. The message bits[1:0] along with TL Flit[0] present is encoded into SH4. TL Flit sector[0] maps to DL Flit sector[143], and so on, into the next segment and DL Flit.

There is no space for TL Flit[1].

In this example there is an alternative sector and a previous carry over up to sector[142].

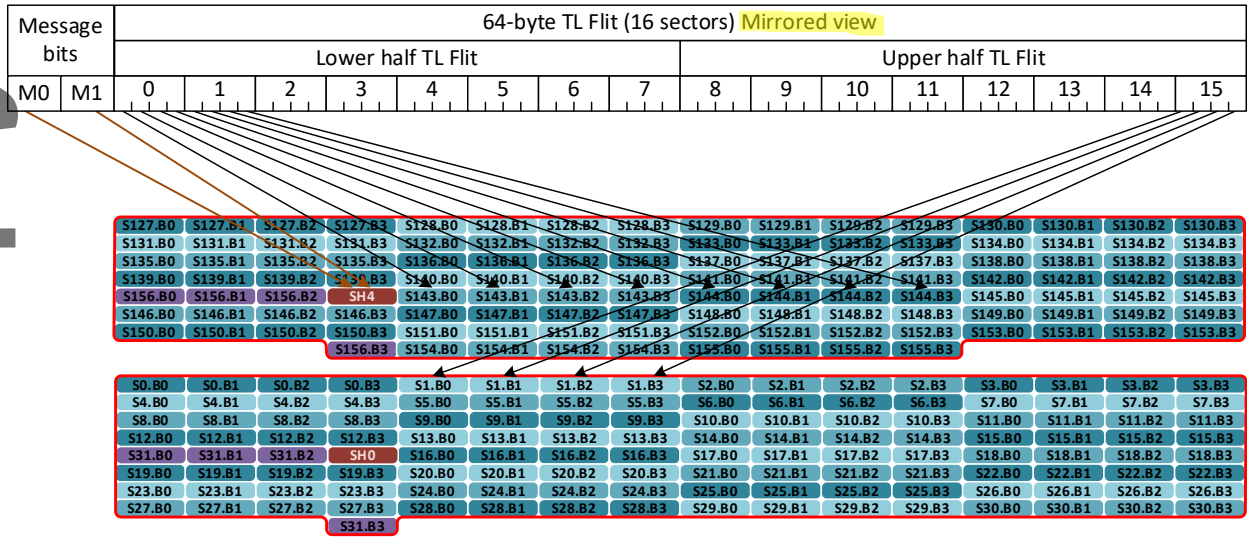


Figure 2-8 TL Flit[0] example 2

### 2.3.6 DL Flit to 64B/66B encoding

The DL Flit to 64B/66B encoding is shown below. The DL Flit is redrawn with the same width as the 64-bit PCS interface, and show reflected in the x-axis. The sequence is left to right, bottom to top, i.e., in sector order. The Sync Header (SH) is set to 0b01 for data code.

- Note: The Sync Header is added in the RS layer. The DL only transmits and receives data Flits.

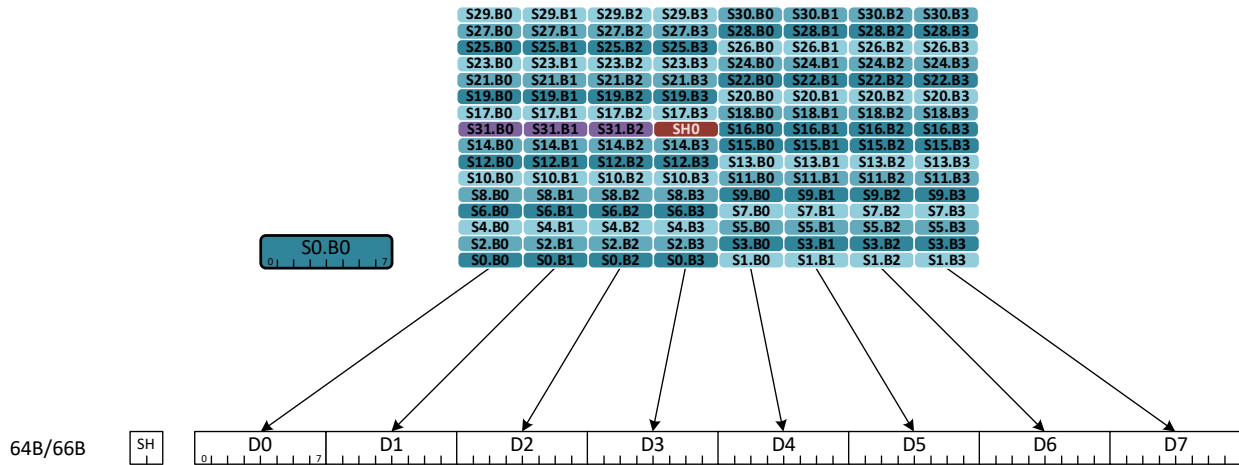


Figure 2-9 DL Flit to 64B/66B Encoding

### 2.3.7 CRC

A 4-byte CRC is calculated and placed in the last bytes in the DL Flit.

The CRC calculation follows 802.3, see clause 3.2.9 Frame Check Sequence (FCS) field. The CRC is calculated over the entire 640Bytes, with the CRC field padded to 0x0.

The CRC is transmitted in the following order:  $x^0, x^1, \dots, x^{31}$ . This is the reverse order compared to IEEE 802.3 FCS. CRC[0] contains bits  $x^0, x^1, \dots, x^7$ , CRC[1] contains bits  $x^8, x^9, \dots, x^{15}$ , etc.

## 2.4 DL messages

Reserved fields shall be set to 0x0 and shall be ignored by receiving link partner.

### 2.4.1 Message Overview

#### 2.4.1.1 Message Types

Any Segment of the DL Flit may contain an DL alternative sector (DLAltSector). The DLAltSector is used to send DL messages. DL messages originate at the DL and terminate at the DL.

All messages have bit 0 as a reserved.

A summary of the message classes and types are shown below.

| Message class<br>(mclass) | code   | Message Type<br>(mtype)           | code  |
|---------------------------|--------|-----------------------------------|-------|
| Basic Messages            | 0b0000 | No-Op message                     | 0b000 |
|                           |        | Tx Ready Notification             | 0b001 |
|                           |        | TL Rate Notification              | 0b100 |
|                           |        | Device ID Request                 | 0b101 |
|                           |        | Port Number Request and Response  | 0b110 |
| Control Messages          | 0b1000 | DL Channel On/Offline negotiation | 0b100 |
|                           |        | Link Width Negotiation            | 0b000 |
| UART Messages             | 0b0001 | UART Stream Transport Message     | 0b000 |
|                           |        | UART Stream Credit Update         | 0b001 |
|                           |        | UART Stream Reset Request         | 0b110 |
|                           |        | UART Stream Reset Response        | 0b111 |

Table 2-3 DL Message Types

#### 2.4.1.2 Message arbitration

There are several sources of messages. All messages are a single DWord sequence, except for UART Stream Transport Message. This can be up to 33 Dwords. UART Stream Transport Message shall be packed sequentially into each Segment, which may span multiple Flits. Other DL message shall be blocked while the UART Stream Transport Message is transmitted.

There are two levels of arbitration. The first Level is within each Message type. Round Robin is used between each of the Basic Messages, to select a potential winner for the group. Round Robin is used between each of the Control Messages, to select a potential winner for the group. Round Robin is used between each of the UART Messages, to select a potential winner for the group. The final level of arbitration is round robin between the Basic Message group, the Control Message group, and the UART Message group.

## 2.4.2 Basic Messages

Basic messages are used to send information from one link partner to another, or to request information from one link partner to another. There is no negotiation. A link partner shall not have more than one outstanding request for this mclass, i.e. the current request must be completed before a new request is made for this mclass.

### 2.4.2.1 Generic Flow

#### 2.4.2.1.1 Single Request

The single request flow is shown below. In this case the local link partner makes a request, and the remote link partner shall accept the request with an Ack response.

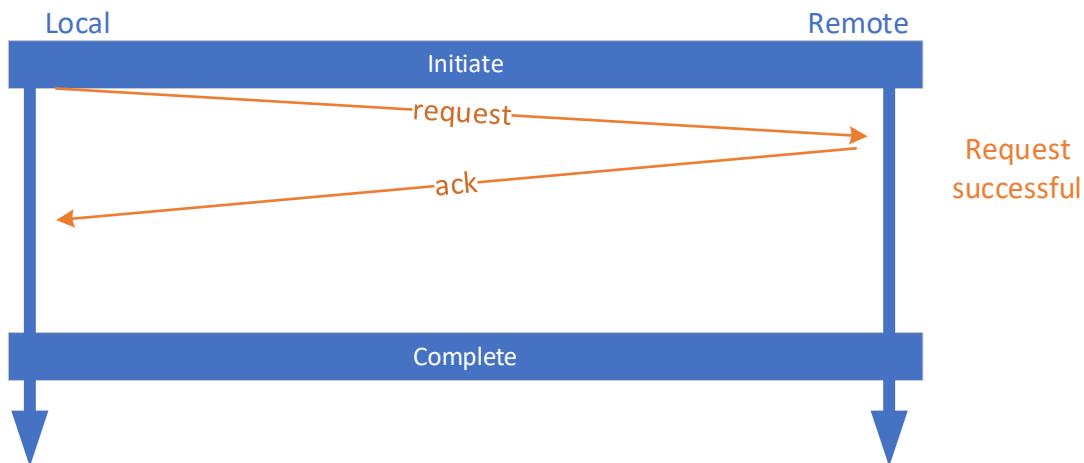


Figure 2-10 Single Request Flow

#### 2.4.2.1.2 Two Requests

It is possible that two requests are made at the same time with the same mclass and mtype. These requests are independent and thus operate as two independent sequences. This is shown below.

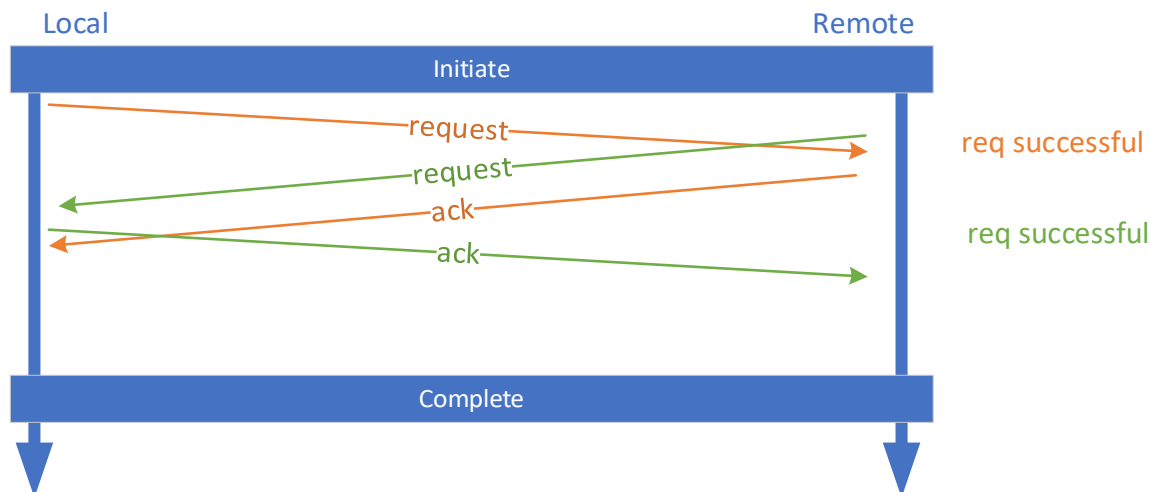


Figure 2-11 Two Requests Flow

#### 2.4.2.2 TL Rate Notification

The TL Rate Notification Message is a message used to convey change in the TL rate (clock frequency) to a connected link partner. The use case of this message is described in section 2.5. It is a unidirectional message only for notification. The link partner shall respond with an Ack with in 1us of the request.

| Field      | Bit   | Description                                                  |
|------------|-------|--------------------------------------------------------------|
| Compressed | 0     | Set to 0, not supported                                      |
| Reserved   | 1     | Reserved                                                     |
| mclass     | 5:2   | Message Class                                                |
| mtype      | 8:6   | Message Type                                                 |
| Reserved   | 11:9  | Reserved                                                     |
| Ack        | 12    | 0: Request message<br>1: Response message                    |
| Reserved   | 15:13 | Reserved                                                     |
| Rate       | 31:16 | TL rate, only valid for Request message, reserved otherwise. |

Table 2-4 TL Rate Notification

Rate is expressed as 50.0 KHz per lsb. Full rate at 1562.5 MHz is coded at 31, 250, decimal. The reference clock rate 156.25 MHz would be coded as 3,125, decimal. The reference clock rate is the minimum rate that shall be supported.

A 1x4, 800G DL is assumed to be 512 bits wide at 1562.5 MHz, and thus  $512 * 1562.5 \text{ MHz} = 800\text{Gbps}$ . Other data widths are possible and shall normalize to 512-bit width. A 2x2, 400G DL per link is assumed to be 256 bits wide at 1562.5 MHz, and thus  $256 * 1562.5 \text{ MHz} = 400\text{Gbps}$ . Other data widths are possible and shall normalize to 256-bit width. A 4x1, 200G DL per link is assumed to be 128 bits wide at 1562.5 MHz, and thus  $128 * 1562.5 \text{ MHz} =$

200Gbps. Other data widths are possible and shall normalize to 128-bit width. **Note:** There is overhead in the DL Flit, 4 bytes CRC, 3 bytes Flit header, and 5 bytes segment header. There are 628-bytes of TL payload per 640-byte DL Flit. Backpressure from the Tx pipeline will naturally limit the throughput to  $(628/640)*200 = 196.25$  Gbps. This is equivalent to a register setting of 30,664, decimal, or 1533.20 MHz.

#### 2.4.2.3 Device ID Request

To aid in the determination of the scale out network topology a Link partner can request the ID of its link partner. The link partner shall respond within 1.0 us of the request. If the Link partner has not been configured with an ID, then it returns 0x0 in the Valid bit, as well as the ID field. The requesting link partner advertises its ID if known, in the request.

When the Ack field is Request:

- Valid indicates if the ID is valid.
- Type indicates switch or Accelerator.
- ID indicates the requestor ID, set to 0x0 if valid is set to 0x0.

When the Ack field is Response:

- Valid indicates if the ID is valid.
- Type indicates switch or Accelerator.
- ID indicates the responder ID, set to 0x0 if valid is set to 0x0.

| Field      | Bit   | Description                                                |
|------------|-------|------------------------------------------------------------|
| Compressed | 0     | Set to 0, not supported                                    |
| Reserved   | 1     | Reserved                                                   |
| mclass     | 5:2   | Message Class                                              |
| mtype      | 8:6   | Message Type                                               |
| Reserved   | 11:9  | Reserved                                                   |
| Ack        | 12    | 0: Request message<br>1: Response message                  |
| Reserved   | 15:13 | Reserved                                                   |
| ID         | 25:16 | 10-bit Switch or accelerator ID                            |
| Reserved   | 28:26 | Reserved                                                   |
| Type       | 30:29 | 0: for a switch<br>1: for an accelerator<br>other reserved |
| Valid      | 31    | 0: if ID is not valid<br>1: if ID is valid                 |

Table 2-5 Device ID Request

#### 2.4.2.4 Port Number Request and Response

To aid in the determination of the scale up network topology a Link partner can request the port number of its partner, attached on the link. The link partner shall respond with in 1us of the

request. If the Link partner has not been configured with its port number, then it returns 0x0 in the Valid bit, and the port number field is undefined and set to 0x0. The requesting link partner advertises its port number if known, in the request.

When the Ack field is Request:

- Valid indicates if the number is valid.
- Port number indicates the port number of the device, if valid is set.

When the Ack field is Response:

- Valid indicates if the number is valid.
- Port number indicates the port number of the device, if valid is set .

| Field       | Bit   | Description                                                  |
|-------------|-------|--------------------------------------------------------------|
| Compressed  | 0     | Set to 0, not supported                                      |
| Reserved    | 1     | Reserved                                                     |
| mclass      | 5:2   | Message Class                                                |
| mtype       | 8:6   | Message Type                                                 |
| Reserved    | 11:9  | Reserved                                                     |
| Ack         | 12    | 0: Request message<br>1: Response message                    |
| Reserved    | 15:13 | Reserved                                                     |
| Port number | 27:16 | 12-bit port number.                                          |
| Reserved    | 30:28 | Reserved                                                     |
| Valid       | 31    | 0: if port number is not valid<br>1: if port number is valid |

Table 2-6 Port ID Request

#### 2.4.2.5 No-OP Message

No-Op messages are used only in UART reset sequences. 40 No-Op messages shall be transmitted during UART reset sequence to flush any data between the transmitter and receiver. The mtype and mclass fields are set according to Table 2-3. No-Op messages are not ACK'd.

| Field      | Bit  | Description             |
|------------|------|-------------------------|
| Compressed | 0    | Set to 0, not supported |
| Reserved   | 1    | Reserved                |
| mclass     | 5:2  | Message Class           |
| mtype      | 8:6  | Message Type            |
| Reserved   | 31:9 | Reserved                |

Table 2-7 No-Op Message

**2.4.2.6 Tx Ready Notification**

The Tx Ready Notification message request is optionally sent when the Link width is unfolding. All devices that support Link Width folding must respond to this message with an ACK within 1us of the request. Devices that advertise support during link width negotiation via Tx Ready support bit, see section 2.4.3.3, must transmit this message when the PL transmitter that is being activated starts transmitting valid symbols. Receiving this message request may be used as a hint to enable the CDR lock on the newly powered receiver.

| Field      | Bit   | Description                               |
|------------|-------|-------------------------------------------|
| Compressed | 0     | Set to 0, not supported                   |
| Reserved   | 1     | Reserved                                  |
| mclass     | 5:2   | Message Class                             |
| mtype      | 8:6   | Message Type                              |
| Reserved   | 11:9  | Reserved                                  |
| Ack        | 12    | 0: Request message<br>1: Response message |
| Reserved   | 31:13 | Reserved                                  |

Table 2-8 Tx Ready Notification

**2.4.3 Control Messages**

Control messages are used by the DL to negotiate a change in operation on the Link. Once the Negotiation completes successfully the change takes place. If the negotiation unsuccessfully completes, then no change occurs. The Link is peer to peer and either link partner may request a change. Some link partner types (i.e., Switches) are not permitted to initiate some requests. A link partner shall not have more than one outstanding request for this mclass, i.e. the current request must be completed before a new request is made for this mclass.

Once a link partner receives a request it shall not schedule a request of the same mclass, until the current request completes. It is possible for two requests to occur at the same time, or near the same time, such that two conflicting or identical request exists, of the same mclass and mtype. When a request is made any subsequent conflicting or identical request of the same mclass mtype that is received the link partner shall not respond with a decision pending, it shall respond with an Ack or Nack.

There is a resolution function, for each mtype, such that conflicting requests resolve to one request being Ack'd and the other request being Nack'd.

**2.4.3.1 Generic Flow****2.4.3.1.1 Single Successful Request Flow**

The single successful request flow is shown below. In this case the local link partner makes a request, and the remote link partner accepts the request with an Ack. The local link partner transmits a confirming ACK to the remote link partner.

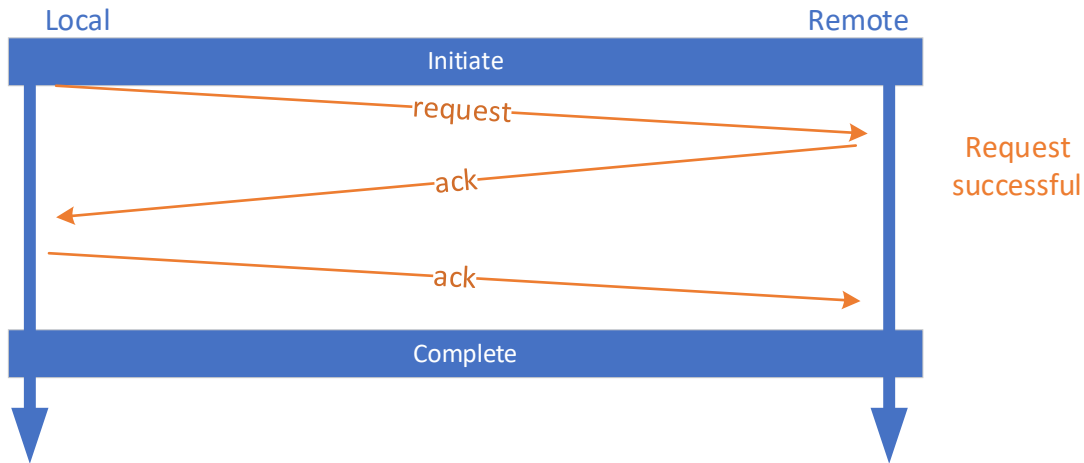


Figure 2-12 Single Successful Request Flow

#### 2.4.3.1.2 Single Unsuccessful Request Flow

The single unsuccessful request flow is shown below. In this case the local link partner makes a request, and the remote link partner rejects the request with an Nack.

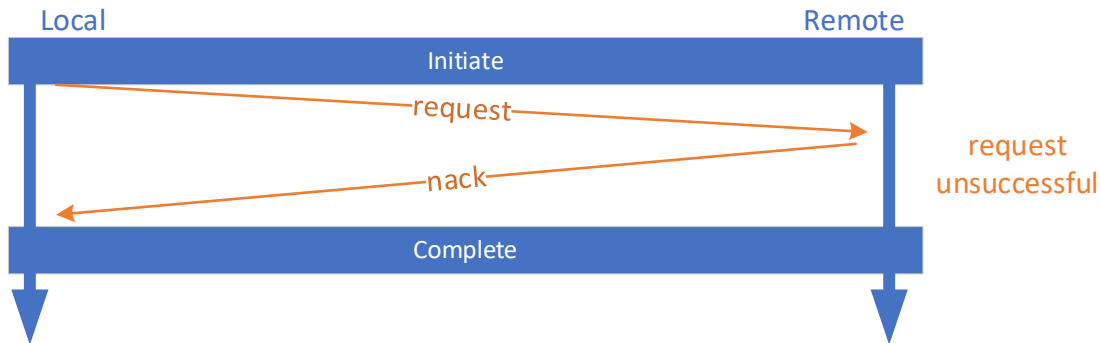


Figure 2-13 Single Unsuccessful Request Flow

#### 2.4.3.1.3 Single decision pending Request Flow

The single decision pending request flow is shown below. In this case the local link partner makes a request, and the remote link partner responds with decision pending Decision. The Remote link partner is required to issue a request later. The Local link partner shall not issue the same mclass mtype request until after the Remote link partner issues a new request of the decision pending mclass and mtype and it is completed.

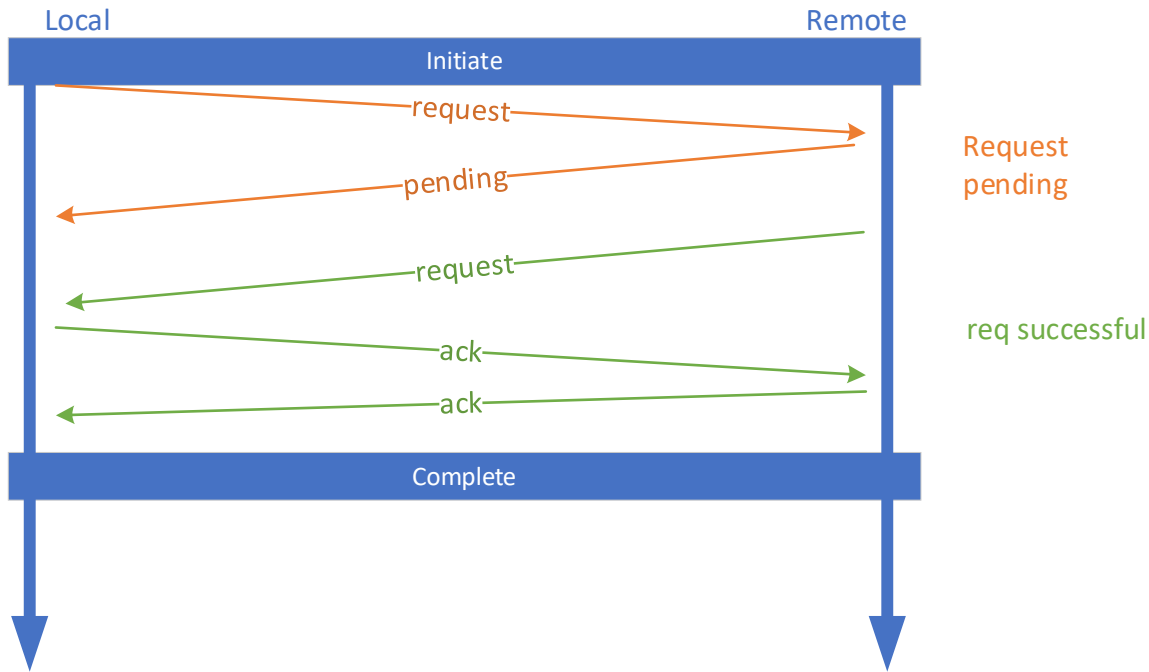


Figure 2-14 Single decision pending Request Flow

**2.4.3.1.4 Conflicting Request Flow**

The conflicting request flow is shown below. In this scenario both local and remote link partners make a request before they have received the conflicting request. The remote link partner compares its transmitted request to the received request, and determines that the received request should be acknowledged, and sends the Ack. The Local link partner compares its transmitted request to the received request and determines that the received request should not be acknowledged and sends the Nack. The local link partner receives the Ack and sends the confirming Ack to the remote link partner.

The local link partner is required to transmit the Nack and Ack in the order shown. Responses relating to received requests or responses shall be in the order received. The resolution function is the same in both link partners, so that they both decide consistently how to resolve the conflict.

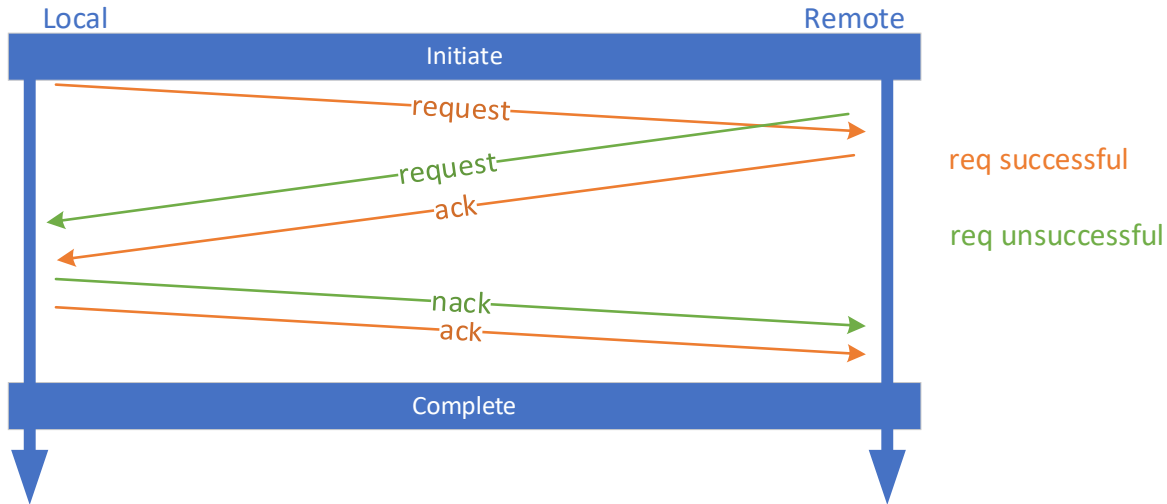


Figure 2-15 Conflicting request flow

2.4.3.1.5 Identical Request Flow

The identical request flow is shown below. In this scenario both local and remote link partners make a request before they have received the same request. The remote link partner compares its request to the received request, and determines that the received request should be acknowledged, and sends the Ack. The Local link partner compares its request to the received request, and determines that the received request should be acknowledged and sends the Ack. The local link partner receives the Ack and sends the confirming Ack to the remote link partner. The remote link partner receives the Ack and sends the confirming Ack to the local link partner.

Both local/remote link partners are required to send the Acks in the order shown. Responses relating to received requests or responses shall be in the order received. The resolution function is the same in both link partners, so that they both decide consistently how to resolve the conflict.

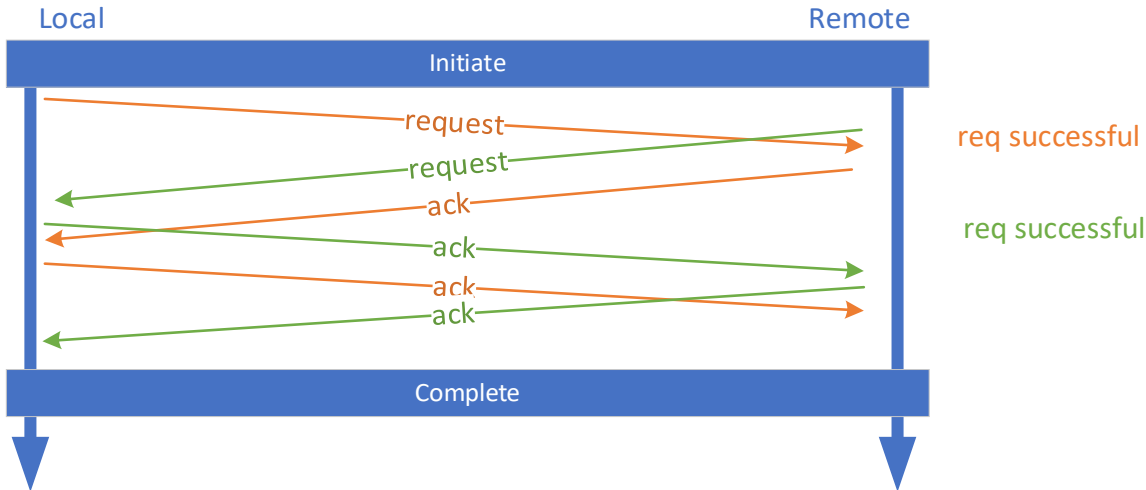


Figure 2-16 Identical Request Flow

### 2.4.3.2 DL Channel Online/Offline Negotiation Message

The format is shown below. These messages are used to negotiate the Channel offline and online. By default, all Channels are offline after reset.

The Following Channel are defined:

- Channel 0: TL Flits
- Channel 4: DL UART Stream Transport Messages

When a Channel is offline it shall not transmit data associated with that Channel. Received Channel data that are offline is silently discarded. When Channel 0 is offline, transmitted DL Flits shall have the TL Flit[1:0] set to 0 in the segment header. When Channel 4 is offline, transmitted DL Flits shall not have mclass set to 0b0001 and mtype set to 0b000 in alternative sectors, i.e., UART Stream Transport Messages.

There are only two states online and offline, thus it is not possible to have conflicting requests from each link partner. Requests for the current state are not permitted. If a request is received for the current state, it is Ack'd, and an error is logged by the link partner that received the request.

When a request is received a response (Ack, Nack, or decision pending) shall be transmitted within 1.0 us. When a request is received for a Channel online (and the current state is offline for that Channel) the response shall be Ack or decision pending. The link partner that responded with decision pending to an online request shall transmit a request for Channel online with in 10ms. When a request is received for a Channel offline (and the current state is online for that Channel) the response shall be Ack.

When the Channel.Command field is Request:

- Channel.TargetState indicates the desired online/offline state of the transmitting link-partner.
- Channel.Response is reserved and shall be set to 0x0.

When the Channel.Command field is decision pending:

- Channel.TargetState is reserved and shall be set to 0x0.
- Channel.Response is the requested Channel.TargetState that it received from the remote link-partner.

When the Channel.Command field is Ack or Nack:

- Channel.TargetState indicates the desired online/offline state of the transmitting link-partner.
- Channel.Response is the requested Channel.TargetState that it received from the remote link-partner.

| Field      | Bit | Description             |
|------------|-----|-------------------------|
| Compressed | 0   | Set to 0, not supported |
| Reserved   | 1   | Reserved                |
| mclass     | 5:2 | Message Class           |

|                     |       |                                                                                         |
|---------------------|-------|-----------------------------------------------------------------------------------------|
| mtype               | 8:6   | Message Type                                                                            |
| Reserved            | 15:9  | Reserved                                                                                |
| Channel.TargetState | 19:16 | 0xxx: Channel offline<br>1xxx: Channel online<br>xNNN: Channel ID                       |
| Channel.Command     | 23:20 | 0100: Request<br>0110: Ack<br>0111: NACK<br>1000: decision pending<br>others: reserved. |
| Channel.Response    | 27:24 | 0xxx: Channel offline<br>1xxx: Channel online<br>xNNN: Channel ID                       |
| Reserved            | 31:28 | Reserved                                                                                |

Table 2-9 Channel Negotiation

### 2.4.3.3 Link Width Negotiation

Link-width negotiation is a message service the DL provides for two link partners to negotiate link width change of the UALink link. See section 2.9.

Only accelerators may request a link-width change. Link Width requests shall be made for the full width of the link, i.e., both physical layers active, or the folded width of the link, i.e., one physical layer active. Requests received that do not meet this requirement shall be NACK'd.

The following describes the format and handshake sequence for link width negotiation message.

When a Link-width negotiation message is received a response is generated with the following rules:

- If priority is set, any request for link width wider than current link width will be NACK'd.
- If priority is unset, any request for link width less than maximum link width allowed will be ACK'd.
- A request of the current link width is NACK'd.

When a link-width request is made with Priority set to 1, the request is urgent, likely due to a thermal excursion or other power-delivery related excursion. The request shall be acknowledged, even if performance requirements might indicate otherwise.

When a link-width negotiation is sent with the assertion of Priority, from either link-partner, hardware shall set a soft lockout such that hardware-requested link-width increases shall not be accepted if the soft lockout remains in force. The occurrence of this soft-lockout shall be communicated to firmware and firmware is responsible for clearing the soft-lockout to allow the system to resume normal link-width change operation.

In the event of conflicting link-width change requests, the winner is determined by the following rules:

- If different PLs are requested, i.e. PL0 and PL1, PL0 shall win, independent of priority.
- Smallest link-width request with Priority set to 1.
- The widest link-width request if neither link-partner advertises Priority set to 1.

From the perspective of each link-partner, link-width negotiation is complete when the link partner has sent a Link Width Negotiation DL-Message with Command set to Ack and has received a Link-Width Negotiation DL-Message with Command set to Ack. When a link-partner issues a Link-Width Negotiation DL-Message with Command set to decision pending, that link partner is effectively terminating the current Link-Width Negotiation DL-Message exchange and accepting responsibility to re-initiate a Link-Width Negotiation DL-Message exchange to bring the negotiation to a conclusion.

When a link-partner loses a negotiation round, that agent shall not re-request a link-width change until one of the following events occurs:

- The link partner that lost the most recent negotiation has a link-width request that is both different from its most recent request and different from the current link-width.
- The link partner that won the most recent negotiation initiates a link-width change request.

When a request is received a response (Ack, Nack, or decision pending) shall be transmitted within 1.0 us. The link partner that responded with the decision pending shall transmit a request within 10ms.

When the Channel.Command field is Request:

- Channel.TargetState indicates the desired link width of the transmitting link-partner.
- Channel.Response is reserved and shall be set to 0x0.

When the Channel.Command field is decision pending:

- Channel.TargetState is reserved and shall be set to 0x0.
- Channel.Response is the requested Channel.TargetState that it received from the remote link-partner.

When the Channel.Command field is Ack or Nack:

- Channel.TargetState indicates the desired link width state of the transmitting link-partner.
- Channel.Response is the requested Channel.TargetState that it received from the remote link-partner.

The Tx Ready support indicates if the device supports sending the Tx Ready Notification Message, see section 2.4.2.6.

| Field               | Bit   | Description                                                                                       |
|---------------------|-------|---------------------------------------------------------------------------------------------------|
| Compressed          | 0     | Set to 0, not supported                                                                           |
| Reserved            | 1     | Reserved                                                                                          |
| mclass              | 5:2   | Message Class                                                                                     |
| mtype               | 8:6   | Message Type                                                                                      |
| Reserved            | 14:9  | Reserved                                                                                          |
| Priority            | 15    | folding priority                                                                                  |
| Channel.TargetState | 19:16 | 0000: full link width<br>0001: folded link width PL0 active<br>1001: folded link width PL1 active |

*Data Link*

|                  |       |                                                                                                                      |
|------------------|-------|----------------------------------------------------------------------------------------------------------------------|
|                  |       | Others reserved                                                                                                      |
| Channel.Command  | 23:20 | 0100: Request<br>0110: Ack<br>0111: NACK<br>1000: decision pending<br>others: reserved.                              |
| Channel.Response | 27:24 | 0000: full link width<br>0001: folded link width PL0 active<br>1001: folded link width PL1 active<br>Others reserved |
| Tx Ready Support | 28    | 0: does not support Tx Ready Notification message request<br>1: supports Tx Ready Notification message request       |
| Reserved         | 31:29 | Reserved                                                                                                             |

Table 2-10 Link Width Negotiation

## 2.4.4 UART Messages

### 2.4.4.1 Protocol Overview

The UART provides a mechanism for sending data across a fraction of the link bandwidth between a UART Transmit Buffer on one end and a UART Receive Buffer on the other end. It is a bidirectional protocol. 32-bits may be sent as an alternative sector every segment. A segment is 1024 bit, thus approximately 3% of the link bandwidth could be utilized for UART Stream Transport Messages.

The UART Stream Transport Message has variable length, and the length is indicated in the first DWord of the UART Stream Transport Message. The First DWord is not stored in the UART Transmit Buffer or UART Receiver Buffer. The length of the UART Stream Transport Message is determined dynamically as a function of available credits and UART Transmit Buffer fill. Subsequent Dwords are the message data and is stored in the UART Transmit Buffer and UART Receiver Buffer.

The recommended UART Transmit Buffer and UART Receive Buffer is 128 Dwords each, per stream. Currently a single stream is defined, however there is provision for up to 8 streams in the stream ID fields.

#### 2.4.4.1.1 Initialization

The initial state of the UART Transmit Buffer and UART Receive Buffer shall be empty. Channel 4 shall be enabled prior to operation. The initial state of the transmit and receive credit counters shall be 0, i.e., the transmitter has no credits to send data.

#### 2.4.4.1.2 Stream Reset

The stream reset sequence is described below:

1. Local F/W determines a UART stream reset is required.
2. The UART stream is disabled, steps 3 to 6 below.
3. The UART Transmit Buffer and UART Receive Buffers for the affected stream is flushed.
4. The credit counts for the disabled stream are reset to 0x0.
5. Any subsequent writes to the disabled UART Transmit Buffer are discarded.
6. Any subsequent receive data from the link partner for the disabled stream is discarded. Error reporting on these discards are suppressed pending the completion of the reset handshake

7. all DL messages (all classes, all types) are blocked to prevent further pollution.
8. 40 DL No-Op Messages are transmitted to ensure run-out of any existing UART Stream Transport Message
9. A UART Stream Reset Request Message is transmitted.
10. DL messages (all classes, all types) are unblocked to allow forward progress.
11. Wait for UART Stream Reset Response with status = SUCCESS
  - a. After a 10ms timeout, if a Reset Response with status = SUCCESS is not returned, loop back to step (7)
12. Normal operation resumes

The Remote link partner that receives the UART Stream Reset Request Message performs the following sequence:

1. The UART stream is disabled, steps 2 to 4 below.
2. The UART Transmit Buffer and UART Receive Buffers for the affected stream is flushed.
3. The credit counts for the disabled stream are reset to 0x0.
4. Any subsequent writes to the disabled UART Transmit Buffer are discarded.
5. A UART Stream Reset Response Message is transmitted with status = SUCCESS.
6. Normal operation resumes.

#### 2.4.4.1.3 Flow Control

Flow control is managed by two modulo  $2^{12}$  Credit Counters, per direction one in the receiver (Receiver Credit Counter) and one in the transmitter (Transmit Credit Counter). During initialization both counters are set to 0x0. Each count value represents a DWord.

The Receiver Credit Counter rules are described below.

- Receiver Credit Counter is initialized to 0 during reset.
- When reset is released Receiver Credit Counter is updated to the size of the UART Receiver Buffer.
- When a DWord is read out of UART Receive Buffer, the Receiver Credit Counter is incremented by 1.
- If Channel 4 is enabled, then a UART Stream Credit Update is scheduled for transmission with the Receiver Credit Counter value in the DataFCSeq field under the following conditions:
  - No UART Stream Credit Update have been scheduled since reset.
  - When Receiver Credit Counter is incremented, and 4 Flits have been Transmitted.

The Transmit Credit Counter rules are described below.

- Transmit Credit Counter is initialized to 0 during reset.
- The Transmit Credit Counter is updated every time a UART Stream Transport Message is sent.

A UART Stream Transport Message shall be scheduled when all the following are true:

- Channel 4 is enabled.
- The UART Transmit Buffer has a DWord or more in it.
- The most recently received DataFCSeq field from the UART Stream Credit Update minus the local Transmit Credit Counter, using modulo  $2^{12}$  subtraction, is greater than 0. This indicates that there is room in the UART Receive Buffer.

The length field of the UART Stream Transport Message is set to the minimum of:

- The result of the modulo subtraction above minus 1
- The UART Transmitter Buffer fill minus 1
- 32 minus 1

#### 2.4.4.1.4 Vendor Defined Packet TLV

There is no relationship between the UART Stream transport Message length and the Length of the Vendor Defined Packet. Shown below illustrates a Vendor Defined Packet [i] that spans 3 UART Stream Transport Messages. The first DWord of the Vendor Defined Packet shown below, is a TL describing the Type and Length of Vendor Defined Packet, the subsequent 3 Dwords V[2:0] describe the Value of the message.

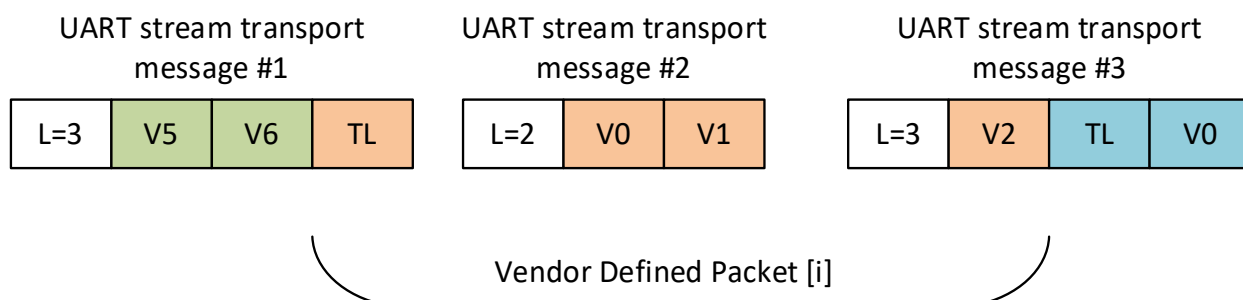


Figure 2-17 Vendor Defined Packet

The first DWord of the Vendor Defined Packet is described below.

| Field     | Bit   | Description                                                                                                        |
|-----------|-------|--------------------------------------------------------------------------------------------------------------------|
| Length    | 7:0   | Payload Length - 1<br>0x00: payload length = 1 DWord<br>0xFF: payload length = 256 DWords                          |
| Type      | 15:8  | Message type                                                                                                       |
| Vendor ID | 31:16 | The vendor ID assigned by the PCI-SIG to the vendor that defined this packet, or 0xFFFF for UALink defined packet. |

Table 2-11 Vendor Defined Packet Type Length (TL) DWord

#### 2.4.4.2 UART Stream Reset Request

The UART Stream Reset Request message is shown below. This is sent to initiate a reset sequence, either for a single stream, or all streams.

| Field      | Bit  | Description                      |
|------------|------|----------------------------------|
| Compressed | 0    | Set to 0, not supported          |
| Reserved   | 1    | Reserved                         |
| mclass     | 5:2  | Message Class                    |
| mtype      | 8:6  | Message Type                     |
| Stream ID  | 11:9 | 000: stream 0<br>others reserved |

|            |       |                                            |
|------------|-------|--------------------------------------------|
| allStreams | 12    | 0: only stream indicated<br>1: all streams |
| Reserved   | 31:13 | Reserved                                   |

Table 2-12 UART Stream Reset Request

#### 2.4.4.3 UART Stream Reset Response

The UART Stream Reset Response message is shown below. This is sent to report the status of a reset sequence, either for a single stream, or all streams.

| Field      | Bit   | Description                                |
|------------|-------|--------------------------------------------|
| Compressed | 0     | Set to 0, not supported                    |
| Reserved   | 1     | Reserved                                   |
| mclass     | 5:2   | Message Class                              |
| mtype      | 8:6   | Message Type                               |
| Stream ID  | 11:9  | 000: stream 0<br>others reserved           |
| allStreams | 12    | 0: only stream indicated<br>1: all streams |
| Status     | 15:13 | 000: success<br>others reserved            |
| Reserved   | 31:16 | Reserved                                   |

Table 2-13 UART Stream Reset Response

#### 2.4.4.4 UART Stream Transport Message

The UART Stream Transport Message is shown below. The UART Stream Transport Message is transmitted a DWord at a time via 32-bits per segment. The message length is specified in Dwords as indicated. The maximum length of the payload is 32 DWords. The maximum length of the transport message is 33 Dwords. The UART Stream Transport Message shall be transmitted continuously, i.e., without another DL messages inserted between any of the UART Stream Transport Message Dwords.

| Field      | Bit   | Description                                                            |
|------------|-------|------------------------------------------------------------------------|
| Compressed | 0     | Set to 0, not supported                                                |
| Reserved   | 1     | Reserved                                                               |
| mclass     | 5:2   | Message Class                                                          |
| mtype      | 8:6   | Message Type                                                           |
| Stream ID  | 11:9  | 000: stream 0<br>others reserved                                       |
| Reserved   | 26:12 | Reserved                                                               |
| Length     | 31:27 | Length of payload +1 DWords, i.e.<br>length = 0 means 1 DWord payload. |

|                 |                       |                                  |
|-----------------|-----------------------|----------------------------------|
| DWord payload 0 | 63:32                 | First payload DWord              |
| DWord payload 1 | 95:64                 | Second payload DWord, if needed. |
| DWord payload n | $(n+2)*32-1:(n+1)*32$ | N'th payload DWord, if needed.   |

Table 2-14 UART Stream transport message

#### 2.4.4.5 UART Stream Credit Update

The UART Stream Credit Update message is shown below. This is used to advertise credit availability from receiver to transmitter.

| Field      | Bit   | Description                        |
|------------|-------|------------------------------------|
| Compressed | 0     | Set to 0, not supported            |
| Reserved   | 1     | Reserved                           |
| mclass     | 5:2   | Message Class                      |
| mtype      | 8:6   | Message Type                       |
| Stream ID  | 11:9  | 000: stream 0<br>others reserved   |
| Reserved   | 19:12 | Reserved                           |
| DataFCSeq  | 31:20 | Data flow control sequence update. |

Table 2-15 UART Stream Credit Update

## 2.5 Transmitter Pacing

### 2.5.1 Overview

The transaction buffers are in the TL, and credit-based flow control guarantees that there is always room in the receive TL buffers. The TL is clocked by the UPLI Clock. Accelerators are permitted to change their UPLI Clock to a lower frequency such that its throughput is lower than the physical layer throughput.

**Note:** Changing to a lower clock frequency is a power reduction mechanism often used in accelerators and CPUs.

Transmitter Pacing is required on the transmitting device to prevent the DL Rx FIFO overflow on the receiving device, when the receiving UPLI Clock is operating at a lower frequency. The Rx FIFO is written at a rate derived from the recovered clock. This rate is 800Gb/s by default and is implemented as 512-bits (64-bytes) at 1562.5MHz = 800Gb/s. The read rate could be slower if the UPLI Clock has been reduced, thus causing the Rx FIFO to fill up.

Pacing is defined as the rate that a Tx Flit from the TL is admitted to the DL Tx FIFO. The DL modulates at “Ready” signal to the TL indicating on which of the clock ticks’ data may transfer.

- Note: The “ready” signal is one possible implementation, others are possible. This is for illustration purposes only.

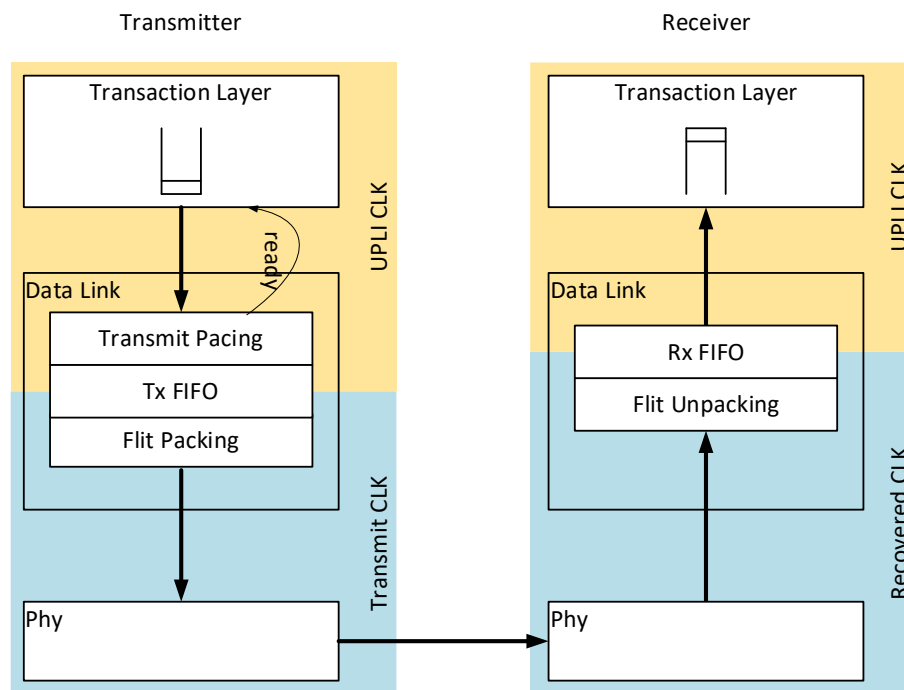


Figure 2-18 Pacing

### 2.5.2 Switch

Switches are required to run at a fixed UPLICLK, such that the throughput is 800Gb/s, 512-bits at 1562.5MHz. If the Accelerator changes its UPLICLK it sends a TL Rate Notification Message to the Switch. The Switch adjusts the transmit pacing rate to avoid Rx FIFO overflows in the accelerator based on receiving this message.

### 2.5.3 Accelerator

Accelerators are permitted to change their UPLI Clock. If the Accelerator intends to change its UPLI Clock it shall inform its link partner of this via TL rate notification messages. It is permitted to have both accelerators on a link operating at different rates. It is permitted that accelerators independently changing their rates at the same time.

### 2.5.4 Sequence

The following rules apply to UPLI Clock rate change:

1. Default rate is 800G bits/s or 1562.5MHz.
2. Only Accelerators may change their UPLI Clock frequency.
3. Changing the UPLI Clock frequency requires reprogramming a PLL, therefore an intermediate UPLI Clock frequency between rates changes shall be to the reference clock of 156.25MHz. Switching between PLL derived clock and 156.25 MHz reference clock is performed with a glitchless clock mux.
4. When an accelerator intends to change its UPLI Clock frequency:
  - a. Adjust its Tx pacing to the reference clock rate.
  - b. Transmit a TL Rate Notification Message for the reference clock rate.
  - c. Wait for TL Rate Notification Message ACK.
  - d. Mux its UPLI Clock to the reference clock.
  - e. Adjust its PLL for the to the new UPLI Clock frequency.
  - f. Mux its UPLI Clock to the new PLL frequency, with Tx pacing set to the link partner's rate if it has a lower UPLI Clock rate.
  - g. Transmit a TL Rate Notification Message for the new clock rate.
  - h. Wait for TL Rate Notification Message ACK.
  - i. Done.
5. When a link partner receives a TL rate notification message:
  - a. It registers this value.
  - b. It adjusts its Tx pacing if needed (i.e., link partner has advertised a lower rate).
  - c. Transmits with a TL Rate Notification Message ACK within 1.0 us.
  - d. Done.

## 2.6 Link Level Replay

### 2.6.1 Overview

Link level replay ensures guaranteed in order delivery of DL Flits in the presence of bit errors that cannot be corrected by the physical layer FEC. The Transmitter keeps a copy of payload Flits (i.e., not NOP Flits), until the receiver positively acknowledges them. The unacknowledged Flits are stored in the TxReplay buffer. The TxReplay buffer shall be large enough to cover the round-trip time (RTT) of the link otherwise the link will not be able to run at full bandwidth. If the TxReplay buffer is full, waiting for positive acknowledgments (ACKs), new DL payload Flits shall not be transmitted. In their place NOP Flits are transmitted. If the TxReplay buffer is full or in a replay state, the DL shall back pressure the TL and not accept any additional TL Flits and ensure that no accepted TL Flits are lost.

The PCS Receiver performs FEC correction prior to forwarding the Flit to the DL, only Flits that pass FEC correction are forwarded to the DL. The CRC is checked in the DL. If the CRC fails, then the DL Flit is deemed bad, and the Flit is discarded.

If the CRC is good, then one of the following occur: a standard replay is scheduled by the Receiver via a Replay Request when the receiver determines the received sequence numbers are out of order or an Ack is scheduled when the receiver determines the received sequence numbers are in order.

When the Transmitter receives an Ack the TxReplay buffer removes entries up to and including the sequence number indicated by ackReqSeq field in the Ack. When the Transmitter receives a Standard Replay Request, it starts replaying all DL Payload Flits, currently held in the transmit replay buffer, starting with the sequence number indicated by the ackReqSeq field in the Replay Request. A Replay Request is not an implicit Ack; no entries are removed from the TxReplay buffer when a Replay Request is received.

When Replay Requests are sent, three Replay Requests shall be sent, to improve reliability, all requesting the same sequence number. Upon receiving a new Replay Request, the receiver shall ignore subsequent Replay Requests during the Replay Request Ignore Window. The three copies of the Replay Request shall be issued as quickly as possible such that no more than one copy of the Replay Request will be sent in each FEC-interleave group.

- Note: this ensures that the loss of any one FEC-interleave group will result in the loss of no more than one copy of the Replay Request.

There are two formats for Flit headers:

- Explicit Sequence Number Flit:** this Flit carries the full 9-bit sequence number, but no information regarding Ack or Replay Request
- Command Flit:** this Flit carries only the lower 3-bits of the sequence number, as well as Ack or Replay Request indication, and the full 9-bit sequence number that is being Aced or replay requested.

When a Command Flit is received, the full 9-bit sequence number can generally be calculated based on previously received full 9-bit sequence number (Explicit Flit), and subsequent 3-bit sequence numbers (command Flit). The receiver performs checks to ensure that this can be calculated unambiguously. If the sequence number cannot be unambiguously determined a replay is triggered. The transmitter schedules Explicit Flits every 7 Flits to aid the calculation being unambiguous.

## 2.6.2 Flit Header

### 2.6.2.1 Explicit Sequence Number Flit Header

The Explicit Sequence Number Flit (or “Explicit Flit” for short) header is shown below. This contains the full 9-bit sequence number. There is no Ack or Replay Request indication.

| Field    | Bit   | Description                                                                                                                                                                                          |
|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| op       | 23:21 | When payload ==0 (NOP):<br>0b000: NOP Flit<br>others: Reserved<br>When payload ==1:<br>0b000: Original transmission of payload Flit “org”<br>0b001: Replay of payload Flit “rpy”<br>others: Reserved |
| payload  | 20    | 1: payload Flit<br>0: NOP Flit                                                                                                                                                                       |
| reserved | 19:17 | Reserved                                                                                                                                                                                             |

|           |      |                                                        |
|-----------|------|--------------------------------------------------------|
| flitSeqNo | 16:8 | Sequence number of Flit. This is the full 9-bit value. |
| reserved  | 7:0  | Reserved                                               |

Table 2-16 Explicit Sequence Number Flit Header

### 2.6.2.2 Command Flit Header

The Command header is shown below. This contains the full 9-bit sequence number for the Flits that is being acknowledged or not acknowledged, along with the lower 3-bits of the flitSeqNo, identified as flitSeqLo.

| Field     | Bit   | Description                                                                                                                         |
|-----------|-------|-------------------------------------------------------------------------------------------------------------------------------------|
| op        | 23:21 | 0b010: Ack<br>0b011: Standard Replay Request “req”<br>others: Reserved                                                              |
| payload   | 20    | 1: payload Flit<br>0: NOP Flit                                                                                                      |
| ackReqSeq | 19:11 | Full sequence number of Ack Flit that is being acknowledged or Sequence number of the Replay Request. This is the full 9-bit value. |
| flitSeqLo | 10:8  | Lower 3 bits of Flit Sequence Number.                                                                                               |
| reserved  | 7:0   | Reserved                                                                                                                            |

Table 2-17 Command Flit Header

## 2.6.3 Term Definitions

### 2.6.3.1 Explicit Sequence Number Flit

A Payload Flit with op equal to 0b000 or 0b001. A NOP Flit with op 0b000. This uses the format described in Table 2-16. Explicit Flits contain the full 9-bit sequence number.

### 2.6.3.2 Command Flit

A payload or NOP Flit with op equal 0b010 or 0b011. In other words, an Ack or Replay Request Flit.

### 2.6.3.3 Ack Flit

A Flit with Replay op 0b010. This uses the format described in Table 2-17.

### 2.6.3.4 Standard Replay Request Flit

A Flit with Replay op 0b011. This uses the format described in Table 2-17.

### 2.6.3.5 Standard Replay Request

A Replay Request that requests a replay of all DL Payload Flits starting from a specified sequence number.

### 2.6.3.6 Tx Replay Buffer

The buffer which stores information for transmitted DL Payload Flits until the DL Payload Flit has been acknowledged by the Link partner.

### 2.6.3.7 Rx Replay Buffer

The buffer which stores information for received DL Payload Flits, until the DL Payload Flit has been released to and consumed by the Receiver.

### 2.6.3.8 Replay Request Ignore Window

A time window in which received Replay Request Flits are ignored so that only a single replay action will be triggered from the multiple copies of the Replay Request that were issued.

## 2.6.4 Rx Flags and Counters

### 2.6.4.1 Rx\_seq\_calc

This 9-bit value is calculated based on the received Flit. Command or Explicit header type as follows.

If Flit is an Explicit Flit:

- $Rx\_seq\_calc = flitSeqNo$

Else: # Command Flit

- $\delta_{lo} = (flitSeqLo - Rx\_last\_seq\_calc \& 0x7) \% 8$

If  $\delta_{lo} == 0$  and flit is payload:

- $\delta_{lo} = 8$
- $Rx\_seq\_calc = (Rx\_last\_seq\_calc + \delta_{lo}) \% 512$

Default value is 0x1FF.

### 2.6.4.2 Rx\_last\_seq\_calc

This 9-bit value is updated based on the Rx\_seq\_calc calculation of the last Flit received. Default value is 0x1FF. See Rx Enqueuing Rules.

### 2.6.4.3 Rx\_last\_ack

This 9-bit value is updated based on the ackReqSeq field of the last Ack Flit received. Default value is 0x1FF. See Rx Ack and Replay Request Processing Rules.

### 2.6.4.4 Rx\_bad\_crc\_count

This 3-bit value incremented based receiving Flits with bad CRC. The counter does not roll over and saturates at 0x7. Bad CRC can lead to sequence number ambiguity resulting in a loss of sync between Rx\_last\_seq\_calc at the receiver and the actual sequence number that the Flit was created with. Default value is 0x0. See Rx Ingress Rules.

### 2.6.4.5 Rx\_unexpected\_count

This 8-bit value is incremented based on receiving Flits with unexpected sequence number while the Rx is in the replay state. The counter does not roll over and saturates at 0xFF. Default value is 0x0. See Rx Enqueuing Rules.

### 2.6.4.6 Rx\_replay\_limit

This 8-bit value set the limit in Flit times that will trigger resending Replay Requests. Default value is 50. This should be set to twice the round-trip latency of the link, in Flit times. See Rx Enqueuing Rules.

### 2.6.4.7 Rx\_ambiguous

This flag indicates if the received flitSeqLo field is ambiguous. This occurs when too many CRC errors occur declaring that future reception of a flitSeqLo field is untrustworthy. Default value is 0x0. See Rx Ingress Rules.

#### 2.6.4.8 Rx\_replay

This flag indicates if the Rx is in a replay state or not. Default value is 0x0. See Rx Enqueuing Rules.

#### 2.6.4.9 Rx\_replay\_ignore\_count

This 4-bit value defines the remaining number of flits for which a received Replay Request will be ignored. This counter counts down and saturates at 0x0. Default value is 0x0. See Rx Ingress Rules.

### 2.6.5 Tx Flags and Counters

#### 2.6.5.1 Tx\_replay\_req\_seq\_no

This 9-bit value holds the sequence number that will be sent 3 times as replica Replay Requests. Default value is 0x0. See Tx Scheduling.

#### 2.6.5.2 Tx\_replay\_req\_count

This 2-bit value indicates how many Replay Requests are left to send. This counter counts down and saturates at 0x0. Default value is 0x0. See Rx Enqueuing Rules and Tx Scheduling.

#### 2.6.5.3 Tx\_last\_seq

This 9-bit value indicates the last sequence number added to TxReplay. This number is incremented for each payload Flit added to the TxReplay buffer. A 9-bit value is stored in the TxReplay buffer along with the Flit, the transmitted Flit may ultimately use the 3-bit flitSeqLo field for a command Flit. Default value is 0x1FF. See Flit sequence number rules, Rx Ack and Replay Request Processing Rules, and Tx Source Flit Rules.

#### 2.6.5.4 Tx\_replay

This 1-bit value indicates if the Tx is in a replay state or not. Default value is 0x0. See Tx Enqueue Rules and Tx Source Flit Rules.

#### 2.6.5.5 Tx\_first\_replay

This 1-bit value indicates the first Flit of a replay, and it shall be transmitted as an Explicit Sequence Number Flit . Default value is 0x0. See Rx Ack and Replay Request Processing Rules and Tx Scheduling.

#### 2.6.5.6 Tx\_explicit\_count

This 3-bit value determines when an Explicit Sequence Number Flit shall be sent. This down counter saturates at 0x0 forcing an Explicit Sequence Number Flit to be sent. Default value is 0x7. See Tx Scheduling.

#### 2.6.5.7 Tx\_ack\_counter

While unacknowledged DL Payload Flits are present in the Tx Replay Buffer, this 24-bit counter keeps track of the time, in Flit times, waiting since the last received ack . Default value is 0x0. See Tx Forward progress.

#### 2.6.5.8 Tx\_ack\_time\_out

This 24-bit register is programed with the threshold for the Tx\_ack\_counter. Default value is calculated for 10.24ms. See Table below for programing as function of serial rate and width, Flit time is a function of serial rate and link width. See Tx Forward progress.

| Serial rate | 100G |   |   | 200G |   |   |
|-------------|------|---|---|------|---|---|
| lanes       | 1    | 2 | 4 | 1    | 2 | 4 |

|                            |         |         |         |         |         |           |
|----------------------------|---------|---------|---------|---------|---------|-----------|
| Register default (decimal) | 200,000 | 400,000 | 800,000 | 400,000 | 800,000 | 1,600,000 |
| Delay                      | 10.24ms | 10.24ms | 10.24ms | 10.24ms | 10.24ms | 10.24ms   |

Table 2-18 Tx\_ack\_time\_out defaults

## 2.6.6 General Rules

### 2.6.6.1 Flit sequence number rules

- Valid Flit Sequence Numbers are 1 to 511, 0 is reserved for future use. 511 wraps to 1.
  - Any (sequence number expression)%511 implicitly wraps 511 to 1.
- NOP Flits do not consume a Flit Sequence Number.
- A NOP Flit uses Tx\_last\_seq for its sequence number.
- A payload Flit uses Tx\_last\_seq + 1 for its sequence number when it is added to the TxReplay buffer.

### 2.6.6.2 Rx Ingress Rules

When an ingress Flit is received:

- Rx\_replay\_ignore\_count is decremented by 1, saturates at 0x0.
- If the Flit is valid i.e., CRC check passes and no reserved codes in the “op” field, then proceed with both:
  - Rx Ack and Replay Request Processing Rules and
  - Rx Enqueuing Rules
- Else # the Flit is invalid
  - Rx\_bad\_crc\_count += 1
  - If Rx\_bad\_crc\_count >= 7 then set Rx\_ambiguous to 1
  - If Rx\_replay = 1 then
    - Rx\_unexpected\_count += 1
    - If Rx\_unexpected\_count >= Rx\_replay\_limit
      - Set Tx\_replay\_req\_count to 3
      - Clear Rx\_unexpected\_count to 0
  - Discard Flit
  - increment CRC error counter

The following additional rules are required for implementations that support backpressure from the TL. Chiplet implementations where the TL is on a different silicon than the DL, and the chiplet interconnect implements replay, i.e. UCle, shall support this capability. The TL may back pressure the DL in the receive direction during a UCle replay or retrain. Non chiplet implementation are encouraged to support backpressure from the TL. In Non chiplet implementation miss programming of transmitter pacing, see section 2.5, could also cause back pressure from the TL.

- If the TL asserts back pressure to the DL, and there is no room to store the DL Flit then:
  - Rx\_bad\_crc\_count += 1
  - If Rx\_bad\_crc\_count >= 7 then set Rx\_ambiguous to 1
  - If Rx\_replay = 1 then
    - Rx\_unexpected\_count += 1
    - If Rx\_unexpected\_count >= Rx\_replay\_limit
      - Set Tx\_replay\_req\_count to 3
      - Clear Rx\_unexpected\_count to 0
  - Discard Flit
  - increment back pressure discard counter

Any buffering in the receive path that was filled to accommodate the back pressure shall be reduced via the reception of NOP DL Flits or empty DL Flits. Empty DL Flits have 0x00 for every segment header and have no TL Flit carry over from the previous DL Flit.

### 2.6.6.3 Rx Ack and Replay Request Processing Rules

When an ingress Flit is received that passes CRC Check:

- A Command Flit with ackReqSeq equal to 0 is dropped and error is logged.
- If both of the following are true:
  - The DL Flit is a Replay Request Flit
  - Rx\_replay\_ignore\_count equals 0x0

Then

- If both are true:
  - $(\text{ackReqSeq} - \text{Rx\_last\_ack} - 1) \% 511 \leq 254$
  - $(\text{Tx\_last\_seq} - \text{ackReqSeq}) \% 511 \leq 254$

Then

- Set Tx\_replay to 1
- Set Tx\_first\_replay to 1
- Set Rx\_replay\_ignore\_count to 12
- Schedule replay with the ackReqSeq from the received Flit as the next Flit to transmit

Else

- Ignore the DL Replay Request command in the ingress DL Flit
- Optionally Log unexpected Replay Request

Else if the DL Flit contains an Ack command, then:

- If both are true:
  - $(\text{ackReqSeq} - \text{Rx\_last\_ack}) \% 511 \leq 255$
  - $(\text{Tx\_last\_seq} - \text{ackReqSeq}) \% 511 \leq 255$

Then

- Rx\_last\_ack = ackReqSeq
- Remove all DL payload Flits with sequence number lower than or equal to Rx\_last\_ack from the TxReplay buffer.

Else

- Ignore the DL Ack command in the ingress DL Flit
- Optionally Log unexpected Ack

The term ackReqSeq above is from the Flit header field for the received Flit.

An example is shown below of a TxReplay buffer with sequence numbers 5 through 9. The Rx\_last\_ack variable is set to 4 as that is the last Ack that was received. Tx\_last\_seq variable is set to 9 the most recent entry in the TxReplay buffer. Modulo math is ignored for simplicity.

The Ack should have an ackReqSeq number 4 or greater. An Ack could be sending the same ackReqSeq 4, for example that was the last DL Flit received. An ackReqSeq number lower than 4

would be unexpected. The Ack should have an ackReqSeq number 9 or less. It would be unexpected to receive an Ack for a higher sequence number than what the transmitter has sent.

The Replay Request should have an ackReqSeq number 5 or greater. A Replay Request could be sending ackReqSeq 5, which indicates to Replay all unacknowledged DL payload Flits with sequence number 5 and higher. An ackReqSeq number lower than 5 in a replay request would be unexpected, those sequence numbers are not in the TxReplay buffer. The Replay Request should have an ackReqSeq number 9 or less. Sequence numbers 10 and higher are not in the TxReplay buffer.

The ackReqSeq that falls outside of the expected range are ignored.

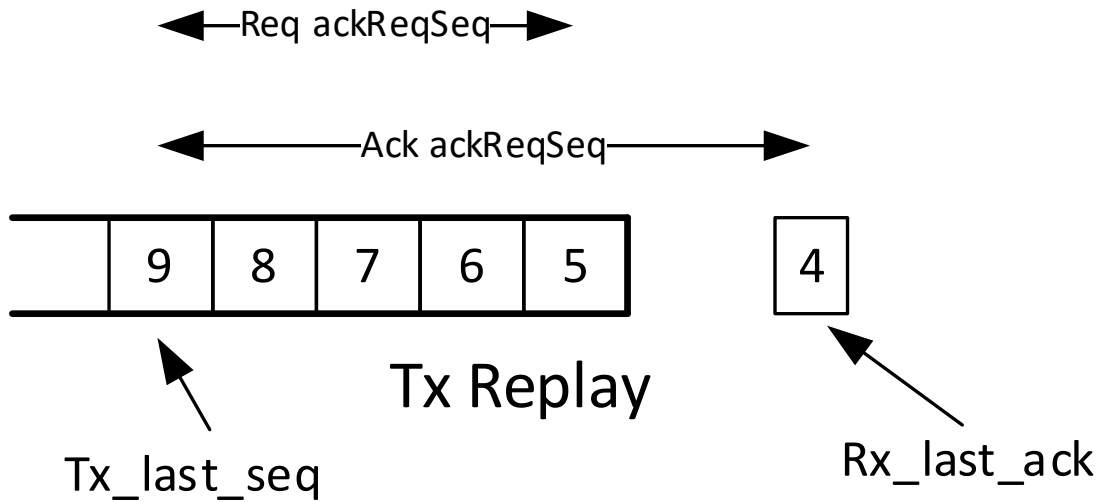


Figure 2-19 Ack Replay Request valid range

#### 2.6.6.4 Rx Enqueuing Rules

When an ingress Flit is received that passes CRC Check:

- An Explicit Flit with flitSeqNo equal to 0 is dropped and error is logged.
- If at least one of the following are true regarding the received Flit:
  - It is an Explicit Sequence Number Flit
  - $Rx\_ambiguous == 0$  and  $Rx\_replay == 0$

Then

- If either of the following are true: # expected sequence number
  - Flit is NOP and  $Rx\_seq\_calc == Rx\_last\_seq\_calc$
  - Flit is payload and  $Rx\_seq\_calc == Rx\_last\_seq\_calc + 1$

Then:

- if payload Flit: add Flit to receive queue
- Clear  $Rx\_unexpected\_count$  to 0
- $Rx\_last\_seq\_calc = Rx\_seq\_calc$
- Clear  $Rx\_replay$  to 0
- Clear  $Rx\_ambiguous$  to 0
- Clear  $Rx\_bad\_crc\_count$  to 0

Else

If Rx\_replay == 0 then: # unexpected sequence number and not in Replay

- Set Tx\_replay\_req\_count to 3
- Set Rx\_replay to 1
- Clear Rx\_unexpected\_count to 0

Else:

- Rx\_unexpected\_count += 1
- If Rx\_unexpected\_count >= Rx\_replay\_limit
  - Set Tx\_replay\_req\_count to 3
  - Clear Rx\_unexpected\_count to 0

Else If Rx\_replay == 1 then:

- Rx\_unexpected\_count += 1
- If Rx\_unexpected\_count >= Rx\_replay\_limit
  - Set Tx\_replay\_req\_count to 3
  - Clear Rx\_unexpected\_count to 0

#### 2.6.6.5 Tx Enqueue Rules

All the following shall be true, for TxReplay to accept a Flit:

- Tx\_replay == 0
- TxReplay buffer is not full
- There are less than 255 unacknowledged Flits in TxReplay

When the TxReplay is accepting Flits, the DL shall provide Flits back-to-back. Flits can be either payload or NOP.

#### 2.6.6.6 Tx Source Flit Rules

At every Flit interval a Flit shall be transmitted, assuming it is in the appropriate DL Link States. Flits are sourced from the TxReplay buffer, during a Standard Replay. Flits are sourced from the normal data flow when not in a Standard Replay. The following describe the rules:

- If Tx\_replay == 1 and there are Flits that are scheduled for replay, then:
  - Send the next Replay Flit
  - If all Flits are sent:
    - Set Tx\_replay to 0
- Else: # Tx\_replay == 0
  - Send the Flit from the DL stream. If the Flit is a payload, then the Flit is added to TxReplay

#### 2.6.6.7 Tx Scheduling

The scheduler decides what type of Flit to send. The rules for this are described below.

- Update Tx\_explicit\_count -= 1
- If Tx\_first\_replay == 1 then:
  - Clear Tx\_first\_replay to 0
  - Set Tx\_explicit\_count to 0x7
  - Set op to Replay (0b001)
- Else if Tx\_explicit\_count <= 0 then:
  - Set Tx\_explicit\_count to 0x7
  - If Tx\_replay == 1 then:
    - Set op to Replay (0b001)

- Else:
    - Set op to Original (0b000)
- Else if Tx\_replay\_req\_count > 0 and this is a new codeword group since last Replay Request was sent then:
  - If Tx\_replay\_req\_count == 3 then:
    - Set Tx\_replay\_req\_seq\_no to Rx\_last\_seq\_calc + 1
  - Tx\_replay\_req\_count -= 1
  - Set op to Replay Request (0b011)
  - Set ackReqSeq to Tx\_replay\_req\_seq\_no
- Else:
  - Set op to Ack (0b010)
  - Set ackReqSeq to Rx\_last\_seq\_calc

#### 2.6.6.8 Tx Forward progress

The Tx\_ack\_counter is decremented when there are unacknowledged Flits in the Tx Replay buffer, saturating at 0x0. The Tx\_ack\_counter is rearmed to Tx\_ack\_time\_out when an Ack is received that removes Flits from the Tx Replay buffer. If the Tx\_ack\_counter reaches 0 then the DL enters the DL Idle state.

### 2.6.6.9 Rx Flow Chart

The Rx Flow chart is shown below. Implicit in section 2.6.6.4 is that Flits are dropped unless they are explicitly enqueued. This diagram shows the explicit Flit drop and optional error counters being incremented.

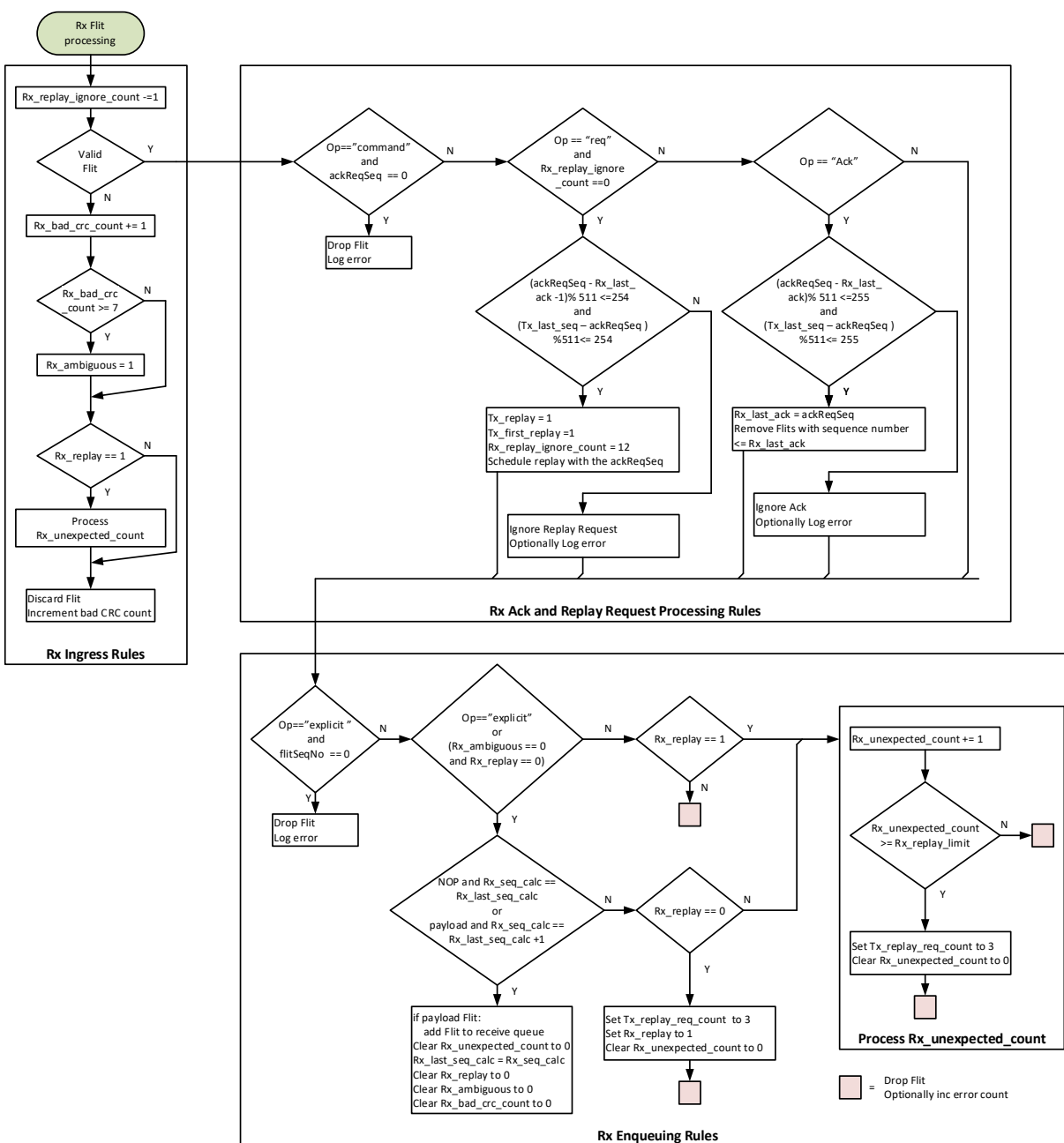


Figure 2-20 Rx Flow Chart

### 2.6.6.10 Tx Flow Chart

The Tx Flow chart is shown Below.

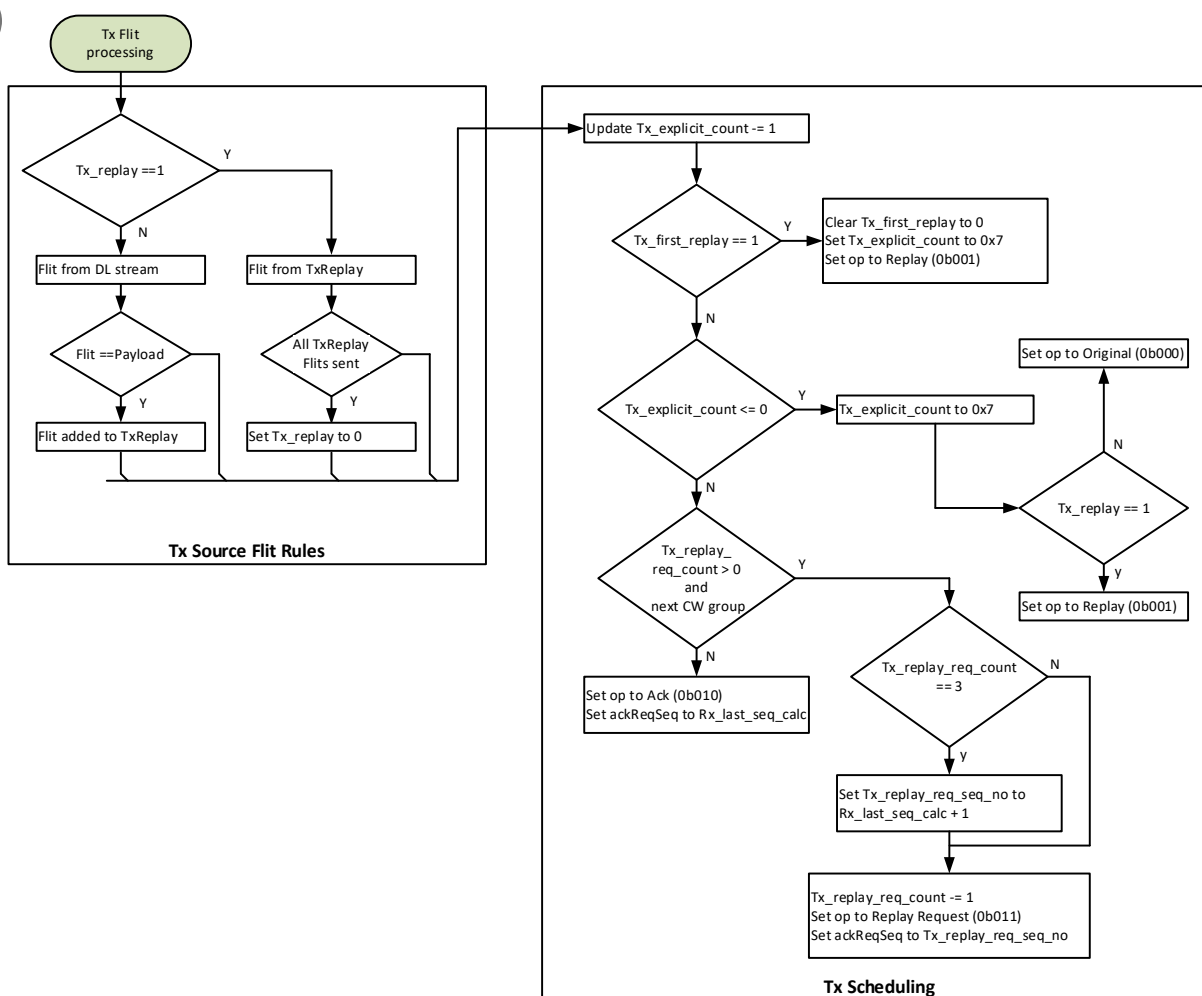


Figure 2-21 Tx Flow Chart

### 2.6.6.11 Link Width Unfolding

The following rules apply during recovery from a failed Physical layer and Link Unfolding . See sections 2.8.6.3, 2.8.6.4, and 2.9.3.

1. DL Flits in the Tx Replay Buffer that were created at a folded width are replayed at a proportionately lower rate when operating at an unfolded width. For example, the folded width is 50% of the unfolded width, and thus DL Flits created at a folded width are replayed at 50% of the rate at the unfolded width.
2. The lower rate is created via the insertion of NOPs. For example, in the case above every other DL Flit would be a NOP, for a 50% rate.

DL Flits created at a folded width may be packed at a higher density of TL Flits then if they were packed with an unfolded width. If they are replayed at the normal rate, then this could cause an overflow at the receiver.

Evaluation Copy

### 2.6.7 Round Trip Time

The TxReplay buffer should be sized to cover the round-trip time of the Link, to prevent stalling the transmit pipeline. Once the TxReplay buffer is full the DL will stop accepting data from the TL, and NOP DL Flits will be sent.

The diagram in Figure 2-22 depicts the elements of round-trip time.

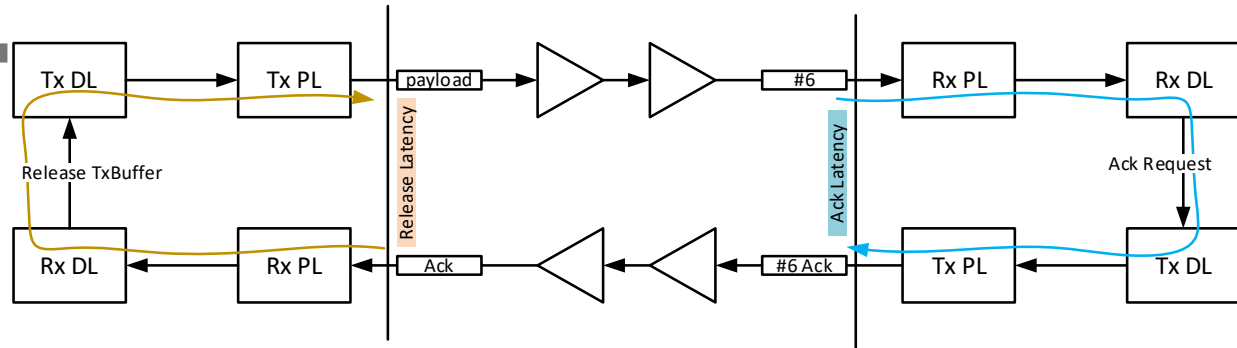


Figure 2-22 Round Trip Time

**Ack Latency:** This is the time from the first bit of an Explicit Sequence Number Flit received to the first bit of the Ack Flit, for the indicated sequence number, transmitted. This shall include any worst-case scheduling delay see section 2.6.6.7. Measured at the package pins.

**Release Latency:** This is the time from the first bit of an Ack Flit received to the first bit of the Payload Flit transmitted, when the TxReplay buffer is in a stalled state due to lack of Ack Flits. Measured at the package pins.

**Channel Latency:** This is the propagation time through the channel. This could include up to two Retimers and includes cable and other interconnect delays.

The RRT is equal to the sum of the Ack Latency + Release Latency + 2x channel latency.

Device vendors should publish their Ack Latency and Release Latency on the data sheets, as well Retimer Vendors should publish their latency on the data sheets.

For example, a RTT = 1,000ns for 200GBASE-KR1/CR1 would equal 25,000 bytes, or 40 Flits, rounded up.

## 2.7 Link State and Errors

The following sections describe the DL Link states. When operating without Link Resiliency there is a mapping of one physical layer to a single DL. When operating with Link Resiliency two physical layers are bonded to a single DL.

The following diagram shows the Link Resiliency logical view on the left. Each Physical Layer is tracked for DL Link State, in its respective “sub” link state machine. Each RS indicates fault status (local or remote) to the DL along with Idle Control Flit reception. This is an input to each “sub” Link State machine. The combination of Link State A and Link State B determine if the Link is up, and if the replay logic shall be reset. The combination of Link States is implicitly described in each “sub” Link State Machine.

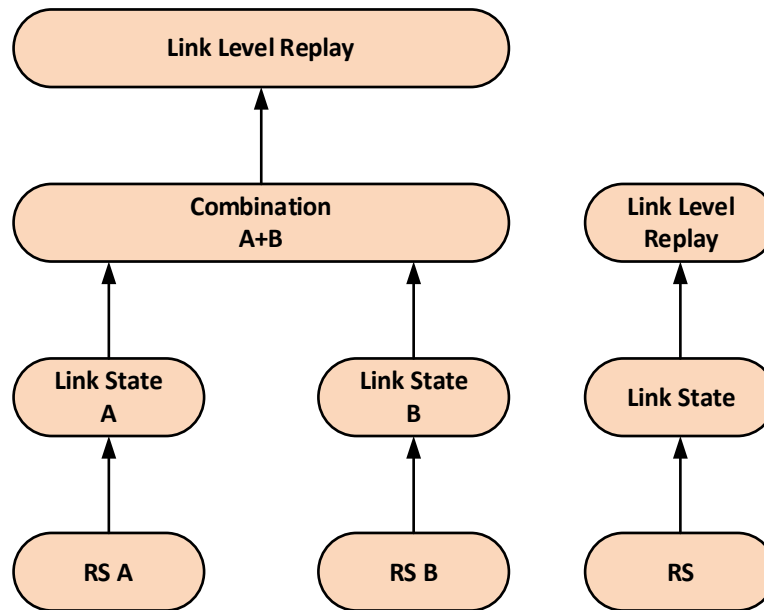


Figure 2-23 Link State with and without Link Resiliency

### 2.7.1 DL Link States

The DL link state diagram is shown below.

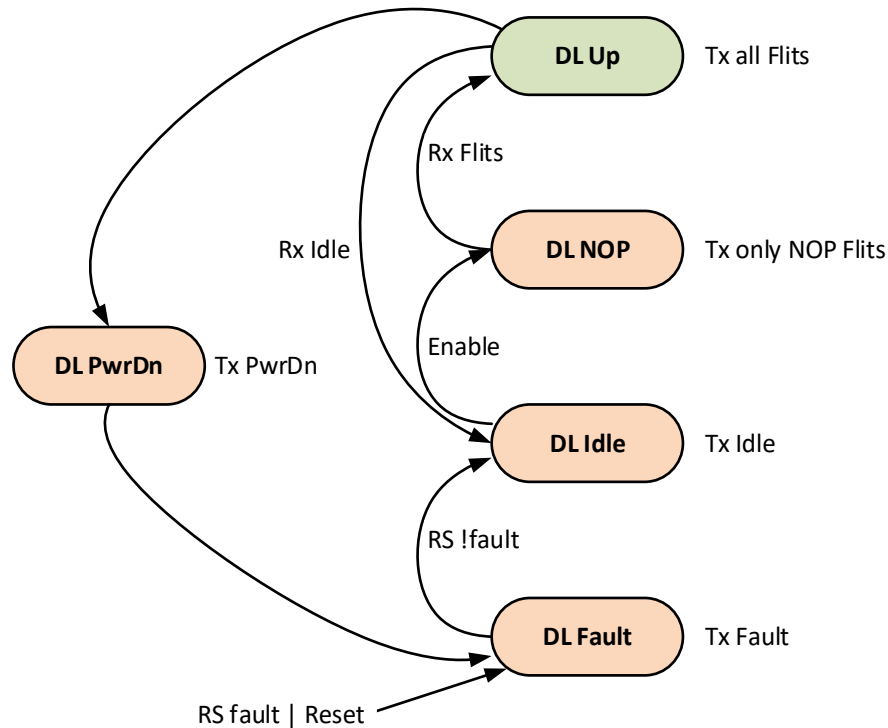


Figure 2-24 DL Link State

The DL has the flowing states:

#### 2.7.1.1 DL Fault

The transmitter shall send Idle if receiving remote fault or send remote fault if a local fault is detected, in accordance with the Link Fault Signaling state machine in the RS. DL Fault is the reset state and can be entered from any state if a local or remote fault is indicated via the Link Fault Signaling state machine in the RS. The DL is link down in this state, when operating without Link Resiliency.

The replay logic shall be reset to its initial state, including the replay buffer, under any of the following conditions.

- When operating without Link Resiliency, and the DL enters this state.
- When operating with Link Resiliency and if one DL “sub” state machine enters this state while no other DL “sub” state machines are in the DL Up state.

The state transitions are described below:

- The next state is DL Idle if the RS is not indicating any fault condition. The RS indicates fault condition when receiving local or remote faults from the PCS.

#### 2.7.1.2 DL Idle

The transmitter shall send Idle. The DL is link down in this state, when operating without Link Resiliency. When the DL, or one DL in the case of link Resiliency, enters this state from DL Up the replay logic shall be reset to its initial state, including the replay buffer.

- The next state is DL Fault if RS indicates a fault.

- The next state is DL NOP if this is the first time this state has been entered since DL PwrDn was exited, after successful completion of Link Width Negotiation to restore the Link to full width, and another DL is in the DL Up state.
- The next state is DL NOP if this is the first time this state has been entered since DL PwrDn was exited, and the physical layer exited DL PwrDn as a result of Link Folding Recovery, see section 2.9.4, and another DL is in the DL Up state, and a control register bit is set to enable this transition. Default register setting is enabled.
- The next state is DL NOP if enabled via a higher layer. The enablement shall occur explicitly before each transition, i.e. F/W writes a register bit each time.

#### 2.7.1.3 DL NOP

The transmitter will send only NOP DL Flits. The RS will inject codewords containing alignment markers or all Idle into the data stream as required. The replay state machine shall not expect ACKs in this state, the link partner may only be transmitting Idle. The DL is link down in this state, when operating without Link Resiliency. The DL shall process any received DL Flit, for example the two DL Flits that trigger the transition to DL Up may be payload Flits and shall be processed normally.

- The next state is DL Fault if RS indicates a fault.
- The next state is DL Up if ten NOP DL Flits have been sent and two consecutive DL flits are received, the received DL Flits may be NOP Flits or payload Flits.
- The next state is DL Idle if the DL is operating with Link Resiliency and another “sub” state machine transitions from DL Up to another state.

#### 2.7.1.4 DL Up

The transmitter will send DL Flits. The RS will inject codewords containing alignment markers or all Idle into the data stream as required. The DL is link up in this state. Note: the RS will pause the DL data stream and inject Remote Fault Flits if the RS signals a local fault condition or will inject Idle Control Flits if a Remote Fault Flit is received, see section 3.2.6.

- The next state is DL Fault if RS indicates a fault after a programmable delay period of continuous fault indication. The delay is between 0 and 10ms in 1 ms steps, with a default of 5ms.
- The Next state is DL PwrDn at the successful completion of Link Width Negotiation to reduce the Link to ½ width, see 2.4.3.3, and the request to change link width did not initiate at his device, and this is the physical layer that is scheduled to be powered down.
- The Next state is DL PwrDn if a PwrDn Control Flit is received.
- The next state is DL Idle if four consecutive Idle control Flits are received by the RS and 10us have passed since the RS indicated local fault.
- The next state is DL Idle if directed from an error containment event, see 2.7.4.
- The next state is DL Idle if directed from a time out event, see 2.6.6.8.

Note: four consecutive Idle control Flits received by the RS, indicates that the link partner has moved to DL Idle state , provided the link partner did not receive Remote Fault Flits.

#### 2.7.1.5 DL PwrDn

The DL signals the RS to send PwrDn Control Flits in this state. If the Transmitter has sent at least 10 PwrDn Control Flits and received one PwrDn Control Flits, then the PL places the PMD Tx and Rx in a low power state, see Section 2.9.

- The next state is DL Fault at the successful completion of Link Width Negotiation to restore the Link to full width, see 6.4.3.3. This negotiation occurs on the active physical layer associated with the DL.

- The next state is DL Fault if all three are true:
  - a local or remote fault is received
  - and no PwrDn Control Flits have been received since entering this state
  - and this state was not entered by receiving at PwrDn Control Flit in DL Up
- The next state is DL Fault if the active “sub” State Machine is not in DL Up.
- The next state is DL Fault if the active “sub” State Machine is in DL Up and either a remote or local fault is indicated by the active RS.

### 2.7.2 Correctable Errors

Correctable errors are bit errors that may occur in the layers below the DL Replay function. These bit errors are either corrected by the PCS FEC or fail CRC and are replayed. FEC correction statistics are provided in the PCS FEC logic. CRC error counts, and replay counts are provided in the replay logic.

### 2.7.3 Uncorrectable Errors

Uncorrectable errors may occur at layers above the DL replay function. These data and control paths shall have appropriate parity protection to detect soft or hard errors.

### 2.7.4 Error Containment

The goal of error containment is to prevent propagation of data that is known to have errors.

#### 2.7.4.1 RS and PCS

Error containment below the DL replay, i.e., RS and PCS is covered by FEC and CRC replay. Only data that passes FEC correction and DL CRC check is forwarded to the TL.

#### 2.7.4.2 Data Link

Ingress Direction: After unpacking, the DL transfers TL Flits to the TL. If any TL Flits are determined to be in error, via parity error or other means, they are flagged as errored to the TL or dropped before transfer to the TL. Subsequent TL Flits are dropped. When operating with Link Resiliency if the PL ID on one PL matches another PL, or if the PL ID changes on any PL from one number to another number (without the physical layer indicating local fault between the change) then all DL Flits are flagged as errored to the TL or dropped before transferring to the TL. Subsequent TL Flits are dropped

Egress Direction: When the TL indicates an errored TL, or a parity error is detected during packing, the CRC for the DL Flit is inverted. This guarantees the DL Flit will fail CRC check at the link partner. Subsequent DL Flits are not sent to the RS, including NOP Flits.

In both cases above the DL goes link down, via a state transition to DL Idle.

## 2.8 Link Resiliency

### 2.8.1 Introduction

Link Resiliency adds a logical function below the replay logic as shown below. Link Resiliency is an optional capability that is defined for links that have 2 or more lanes. Link Resiliency bonds two independent physical layers to the DL. In the transmit direction DL Flits are time division multiplexed to both physical layers. In the receive direction DL Flits are time division demultiplexed from both physical layers. During a physical layer fault condition, the DL will reconfigure to only transmit and receive on the non-faulted physical layer. Link Level Replay is at

the combined level, i.e., there is one Link Level Replay entity for two bonded physical layers. DL link state is maintained per physical layer.

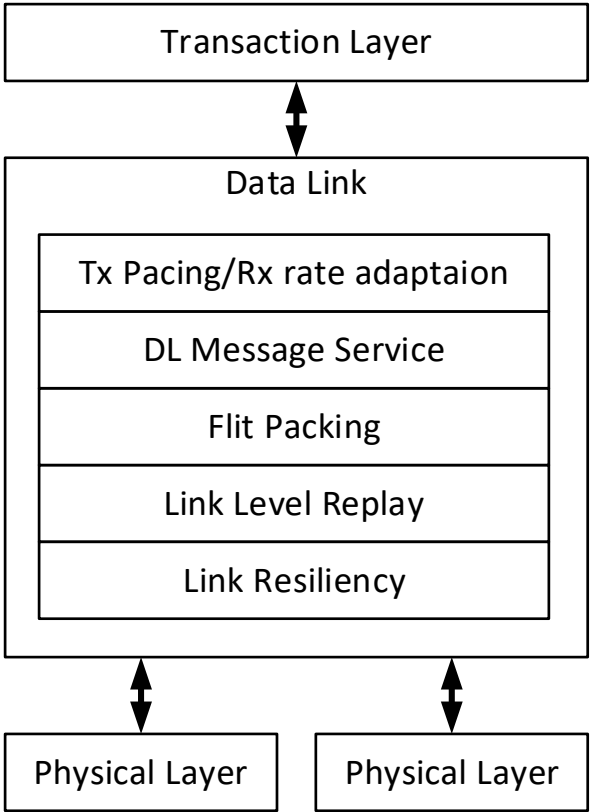


Figure 2-25 Link Resiliency

2.8.2 Transmit

When the DL is operating with a single physical layer in a non-faulted state DL Flits shall be sent only to the non-faulted physical layer.

When the DL is operating with both physical layers in a non-faulted state DL Flits shall be sent to each physical layer alternatively in a round robin fashion. The first DL Flit shall be sent out on physical layer A. This is shown conceptually below. By convention the Tx shall map TL[A] to the lower order lane(s) of the station. Due to Lane reordering, PL[A] may or may not be received on the lower order lanes(s) at the Link partners receiver.

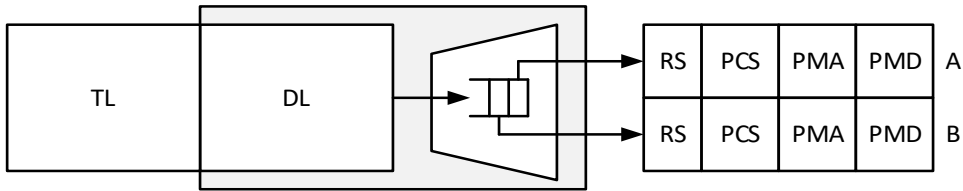


Figure 2-26 Transmit Resiliency

In order to reduce latency in the transmit mux buffering the attached physical layers shall be synchronized as described in section 3.8. The maximum additional latency incurred through the transmit mux buffering is ½ Flit time.

### 2.8.3 Receive

When the DL is operating with a single physical layer in a non-faulted state DL Flits shall be received only from the non-faulted physical layer.

When the DL is operating with both physical layers in a non-faulted state DL Flits shall be received from each physical layer alternatively in a round robin fashion. This is shown conceptually below. Flits from each physical layer are merged into a single stream.

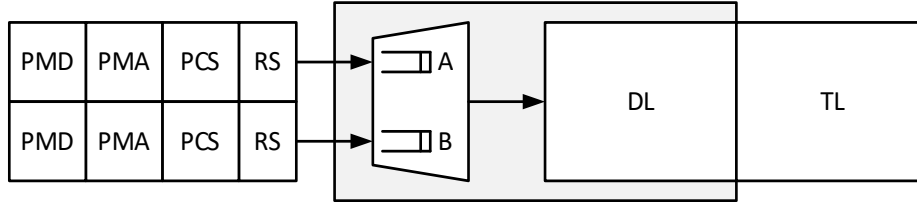


Figure 2-27 Receive Resiliency

Different physical layers may have different skews. This skew may cause the Flits to be received such that all Flits received from one physical layer are delayed and are received out of order with respect to the strict incremental, monotonic transmit order.

IEEE 802.3 skew point requirements are shown below for reference, along with UALink Requirements.

| Skew points         | 400G<br>Table 116-8<br>802.3 – 2022 | UALink<br>100G<br>Serial | 800G<br>Table 169-5<br>802.3dj | UALink<br>200G<br>Serial |
|---------------------|-------------------------------------|--------------------------|--------------------------------|--------------------------|
| SP1 (ns)            | 29                                  | 29                       | 16                             | 16                       |
| SP2 (ns)            | 43                                  | 43                       | 25                             | 25                       |
| SP3 (ns)            | 54                                  | 54                       | 36                             | 36                       |
| SP4 (ns)            | 134                                 | 134                      | 116                            | 116                      |
| SP5 (ns)            | 145                                 | 145                      | 127                            | 127                      |
| SP6 (ns)            | 160                                 | 160                      | 145                            | 145                      |
| At PCS receive (ns) | 180                                 | 180                      | 152                            | 152                      |

Table 2-19 Skew budget

In addition, 802.3 requires 4ns of skew variation, Skew Variation is defined as the change in Skew between any PCS lane and any other PCS lane over the entire time that the link is in operation. With Link Resilience Skew Variation is defined as the change in Skew between any PL and any other PL over the entire time that both PLs are in operation.

### 2.8.4 Reordering

The DL receive path shall reorder the DL payload Flits to conform to the strict incremental, monotonic transmit order. Reordering is required to adjust for any skew between the physical layers. The RS indicates when alignment markers are received by the PCS, and what the PL ID is, see section 3.8. In the Transmit direction the RS synchronizes the transmission of alignment markers, see section 3.8. When operating with Link Folding, unfolding the Link uses rapid alignment markers

(RAMs) to achieve alignment faster than regular alignment markers. The DL shall infer the “imaginary” RAM placement for the active PL, it is every 32 , 64 or 128 code words from the AM, see section 3.3.7. The “imaginary” RAMs on the active PL are used to align to the RAMs of the PL to be added. The following diagrams depict AMs being used to obtain alignment and reorder. This enables reordering to occur before an AM is received on the PL to be added. The reorder steps are as follows:

The transmitted Control Flit sequence is shown in DL Idle below. AM is for a Control Flit that contains the alignment marker and the PL ID, and Idle is for Idle Control Flit.

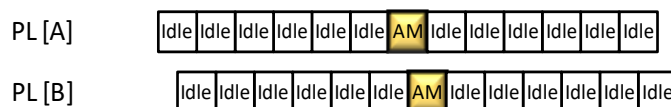


Figure 2-28 Transmit Offset

Due to the skew, the order received of the Control Flits could be as shown below. The DL receives the indication that a Control Flit was received by the RS, the type of control Flit and in the case of AM Flit the PL ID.

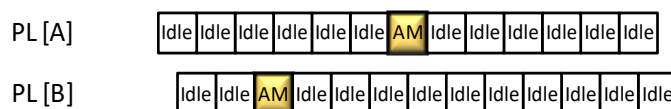


Figure 2-29 Initial Received Order

The DL shall use the relative indication of the Alignment Markers along with the PL ID to perform the reordering, as shown below.

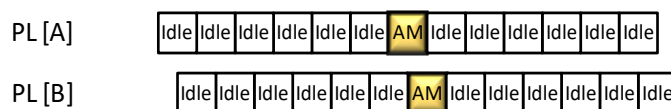


Figure 2-30 After reorder

When the payload DL Flits are received the reordering shall already be completed, as shown below.

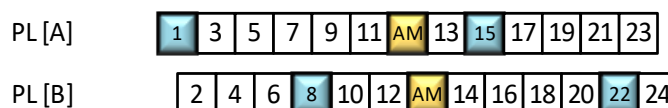


Figure 2-31 Reorder with DL Flits

Due to skew variation, the DL shall continuously monitor the alignment marker, and Flit reception and adjust the skew delta as required. There shall be 0 ppm between all lanes of the station when transmitted.

Alignment markers shall always be received prior to a Link recovery, i.e. a link that was receiving DL Flits on only one physical layer and now receiving DL Flits on both physical layers, and thus the DL shall determine the reorder prior to DL Flits being received.

## 2.8.5 DL Link State

With Link Resilience there is an associated DL “sub” Link State machine, as described in section 2.7.1, per physical layer. Fault information from each RS is signaled to each DL “sub” Link State, as shown below.

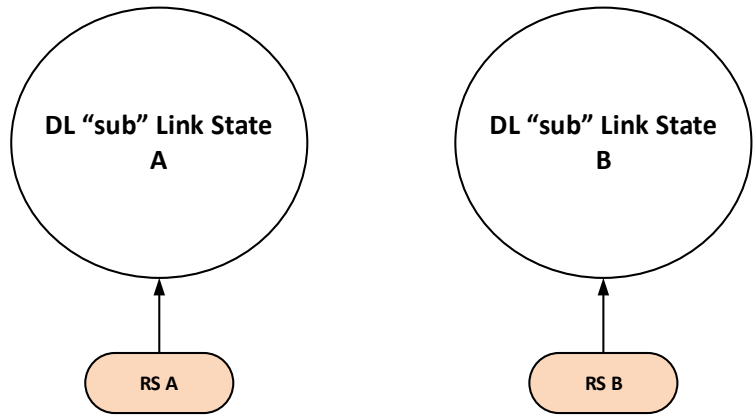


Figure 2-32 DL “sub” State Machines

The coordination of these “sub” state machines has the following rules:

- When all of the “sub” Link State machines are in DL Idle:
  - F/W shall enable all of the “sub” Link State machines to advance to the DL NOP state, when F/W intends to enable the Link.
  - The register bits, which enable advancing to the DL NOP state, one for each physical layer, shall be written at the same time. The control bits shall be in the same register.
- If one or more of the “sub” Link State machines is in DL Up then the DL is up, else the DL is down.

2.8.6 Flit Sequences

In the following diagrams the yellow rectangles represent a code word filled with control blocks, i.e., Control Flit. AM is for the alignment marker, IDLE is for Idle code blocks, Fault is for remote fault. The blue rectangles represent a code word filled with data blocks. NOP is explicitly a NOP Flit, DL Flit is typically a payload Flit, however, it could be a NOP Flit due to the Tx Replay Buffer being full. The AMs are shown for reference to describe the required transmit skew at the physical layers, see section 3.8. There is no implied timing between the Flit transitions and the AM placement.

2.8.6.1 Initial Link Up

The figure below shows the transition of both physical layers from DL Idle to DL NOP at the transmitter. Physical layer A shall be the first to transition and request NOP Flits from the DL. Physical layer B shall be the next to transition ½ Flit later.

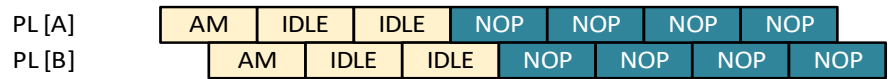


Figure 2-33 Initial Link Up to DL NOP

The figure below shows the transition of both physical layers from DL NOP to DL Up at the transmitter. DL Flit is typically a DL payload. Physical layer B shall be the next to transition ½ Flit later.

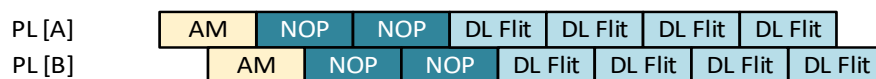


Figure 2-34 Initial Link Up to DL Up

The initial Link Up shall occur with both physical layers. F/W shall not enable the transition from DL Idle to DL NOP unless both physical layers are in DL Idle.

### 2.8.6.2 Link Fault

The figure below shows the transition of one physical layer transitioning to a fault at the transmitter. In this scenario physical layer B detects a local fault and transmits Remote Fault. All the DL Flits are transmitted by physical layer A.

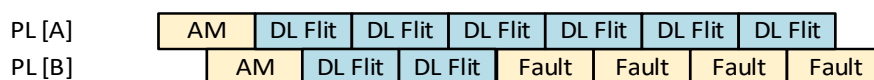


Figure 2-35 B Receives Local Fault

The figure below shows the link partner behavior to the above fault condition. Physical layer B receives remote fault Flits and responds with Idle. All the DL Flits are transmitted by physical layer A.



Figure 2-36 B Receives Remote Fault

Fault scenarios generally require link level replay at the DL to replay lost Flits.

### 2.8.6.3 Fast Link Recovery

Fast Link recovery occurs when the faulted physical layer does not trigger a DL transition to DL Fault, the programmable timeout in DL Up did not expire.

The figure below shows the transition of physical layer B from sending Idle to DL Flits at the transmitter.

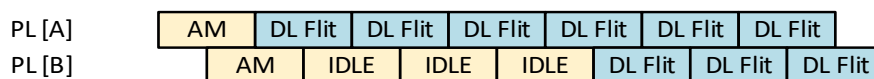


Figure 2-37 Fast Recovery

Recovery scenarios shall determine the reordering prior to receiving DL Flits, and thus not incur out of order and replays during the transition.

### 2.8.6.4 Link Down Recovery

Link down recovery occurs when the faulted physical layer does trigger a DL transition to DL Fault, the programmable timeout in DL Up did expire.

The figure below shows the transition of physical layer B from DL Idle to DL NOP at the transmitter, while physical layer A is in DL Up.

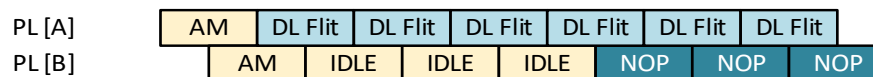


Figure 2-38 Link Down Recovery to DL NOP

The figure below shows the transition of physical layer B from DL NOP to DL Up at the transmitter, while physical layer A is in DL Up.

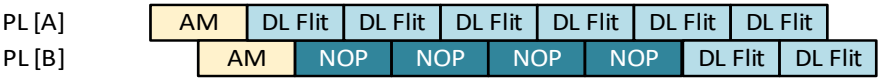


Figure 2-39 Link Down Recovery to DL Up

Recovery scenarios shall determine the reordering prior to receiving DL Flits, and thus not incur out of order and replays during the transition.

2.9 Link Folding Operation

2.9.1 Introduction

Link Folding is a power saving feature. When the Link is folded the SerDes on the inactive Lanes are placed in a low power state. Link Folding adds additional functionality to Link Resiliency. Link Folding is an optional capability. Link Resiliency must be supported to support Link Folding. Both Link Resiliency and Folding are defined for links that have 2 or more lanes. Link Resiliency and Folding bonds two independent physical layers to the DL. The DL can operate in two modes:

- Full bandwidth mode with all Physical Layers active. In the transmit direction DL Flits are time division multiplexed to all physical layers. In the receive direction DL Flits are time division de-multiplexed all both physical layers.
- Half bandwidth mode with half of Physical Layer active. In the transmit direction DL Flits are sent to half of the physical layers. In the receive direction DL Flits are received from half of the physical layers.

Operation at half bandwidth could be a result of a physical layer failure or deliberate action to save power, i.e., folding the Link.

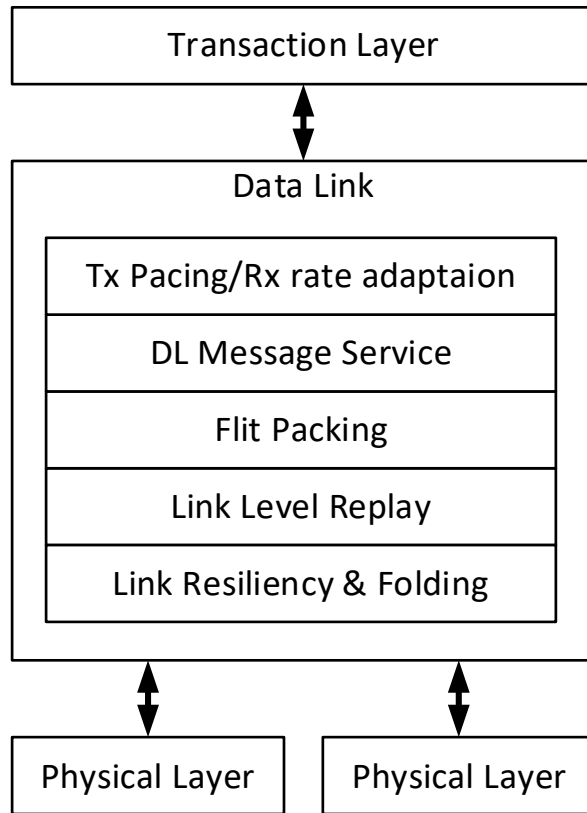


Figure 2-40 Lik Folding

### 2.9.2 Link Folding

An implementation specific mechanism in the Accelerator decides to fold the link to save power. The TL signals the DL to fold the Link.

The power down ladder diagram is shown below.

1. The Initiator sends requests to reduce width, see section 2.4.3.3
2. The Target ACKs
3. The Initiator ACKs

The last steps occur only on the attached physical Layer to be powered down.

1. The Target enters DL PwrDn Link State, and signals the PL to send PwrDn Control Flits
2. The Initiator enters DL PwrDn Link State, and signals the PL to send PwrDn Control Flits
3. The Initiator places Tx and Rx in a low power state
4. The Target places Tx and Rx in a low power state

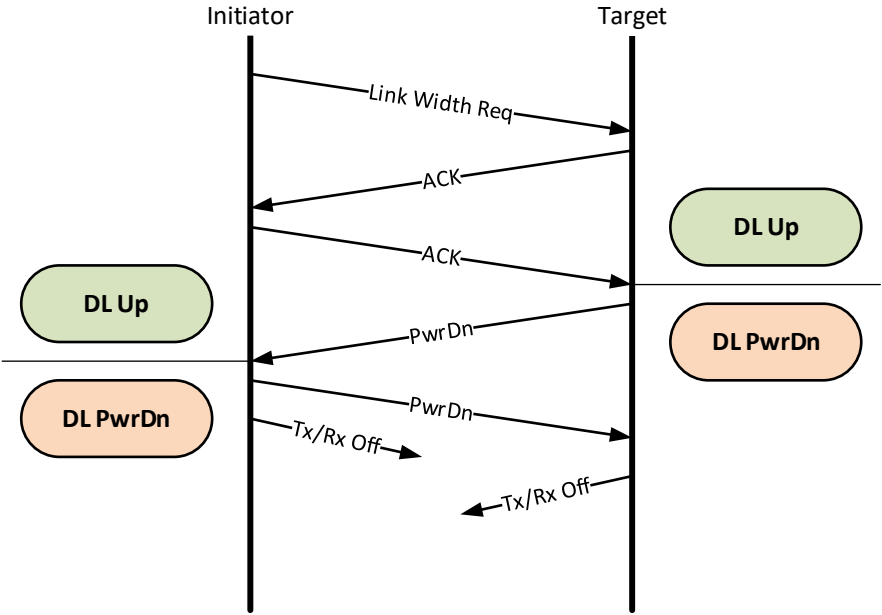


Figure 2-41 Link Folding Ladder Diagram

The determination of which physical layer to power down is with respect to the transmitted Accelerator PL ID. The PL ID to power down is specified in the Link Width Negotiation control message. Note system routing is permitted to swap physical layers, such that PL ID 1 may be received on PL ID 0.

When the request has completed at the Target to change the width (link fold), see section 2.4.3.3, the Link State Machine transitions to the state DL PwrDn, see section 2.7.1.

The following paragraphs describe the power down sequence. This shows the Flits on the wire, DL Flits (blue) are generated by the DL, and Control Flits (yellow) by the RS. AMs are shown for reference, indicating that PL[A] is sent ½ Flit before PL[B].

2.9.2.1 Link Folding Flit Sequence

In the figure below PL[B] transitions from sending DL Flits to sending PwrDn control Flits. PL[A] continues to send DL Flits.

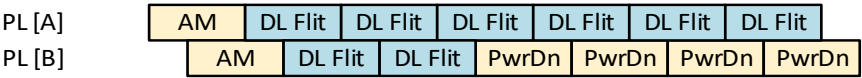


Figure 2-42 Link Folding Sequence to PwrDn

In the figure below PL[B] transitions from sending PwrDn control Flits to turning off the PMD Tx and Rx.

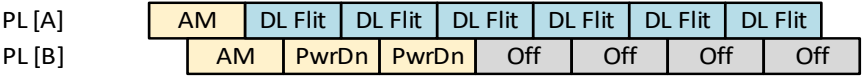


Figure 2-43 Link Folding Sequence to Off

2.9.3 Link Unfolding

The following is a high-level sequence:

1. Implementation specific mechanism in the Accelerator decides to restore the link to full width.

2. TL signals DL to restore link to full width.
3. Each Device completes DL request to change width, see 2.4.3.3.
4. Each DL signals the PL that was powered down to power up.
5. PL powers up, restores Tx and Rx settings.
6. DL state Machine progression for DL Fault to DL Up.

When the request has completed to change the width (link unfold), see section 2.4.3.3, the Link State Machine transitions to the state DL Fault, see section 2.7.1.

### 2.9.3.1.1 Link Unfolding Flit Sequence

Shown below is the transition of one physical layer powering up. Rapid Alignment Markers (RAM) are sent on the physical layers that is powering up. PL[B] is transmitting a remote fault; its receiver has just powered on and has not achieved alignment.

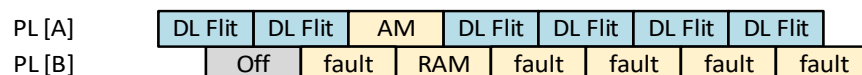


Figure 2-44 Link Unfolding in DL Fault

Shown below is the transition of one physical layer transitioning from DL Idle to DL NOP, and then the transition to DL Up.

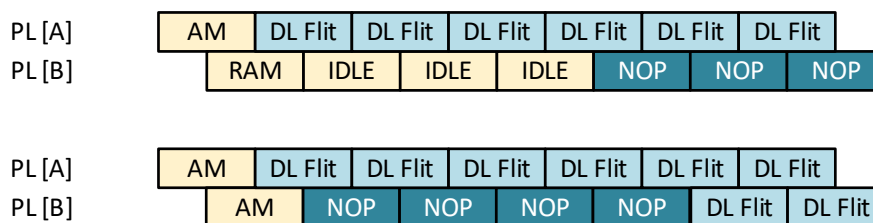


Figure 2-45 Link Unfolding to DL Up

Link Unfolding scenarios shall determine the reordering prior to receiving DL Flits, and thus not incur out of order and replays during the transition.

## 2.9.4 Link Folding Recovery

When the DL is operating with not all physical layers and a physical layer indicates a fault, all powered down physical layers are powered up, via a transition from DL PwrDn to DL Fault. The following paragraphs describe the sequence.

### 2.9.4.1 Link Folding Recovery Flit Sequence

In figure below PL[A] stops transmitting DL Flits, it received a local fault indication and thus the RS starts transmitting remote fault. This triggers PL[B] to transition from DL PwrDn to DL Fault, and power up the Tx and Rx. The Tx RS starts sending remote fault as its receiver has not achieved alignment lock. A RAM is sent with the same  $\frac{1}{2}$  Flit offset with respect to PL[A]’s AM. The AM and RAM placement is shown for reference, in general the RAMs will start without any reference to the AM on the active PL.

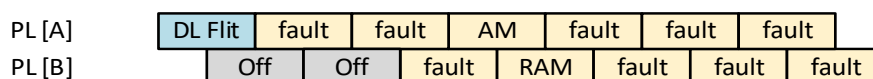


Figure 2-46 Link Folding Error

In the figure below PL[A] continues to send remote fault. PL[B] is in DL Idle and is sending Idle and is receiving Idle, the last RAM is sent and the DL transitions to DL NOP. In DL NOP the DL sends NOP DL Flits.

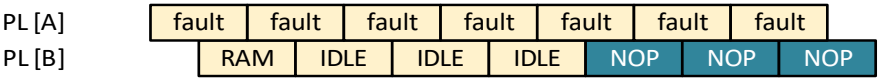


Figure 2-47 Link Folding Error Recovery to DL NOP

In the figure below PL[A] continues to send remote fault. PL[B] is in DL NOP, and transitions to DL Up, and the DL now sends DL Flits.

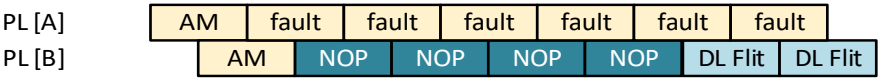


Figure 2-48 Link Folding Error Recovery to DL Up

If PL[A] is able to exit the faulted state prior to its DL exiting DL Up, then the Link shall operate with all physical layers active.

## 3 Physical Layer

### 3.1 Introduction

UALink Phy is based on 802.3 Ethernet Phy. UALink is defined for 1, 2, or 4 serial lanes running at a serial rate of 212.5Gbps (200GBASE-KR1/CR1, 400GBASE-KR2/CR2, 800GBASE-KR4/CR4) , as well as a lower speed serial rate option of 106.25Gbps (100GBASE-KR1/CR1, 200GBASE-KR2/CR2, 400GBASE-KR4/CR4).

The block diagram is shown below, where \* indicates modifications from IEEE 802.3. The PCS/PMA operates in additional codeword interleave modes, with reduced interleave, to achieve better FEC latency at the cost of decreased burst error correction. The PMD is unmodified from 802.3. Auto Negotiation and Link Training AN/LT is unmodified from 802.3. The 64B/66B encoding is a subset of what 802.3 supports.

The PCS and RS require additional behavior to synchronize DL Flits to codewords, so that 640-byte Flits from the DL fit exactly into one RS(544, 514) codeword. This will optimize latency and minimize replay Flits. The DL generates a CRC as part of each 640-byte DL Flit.

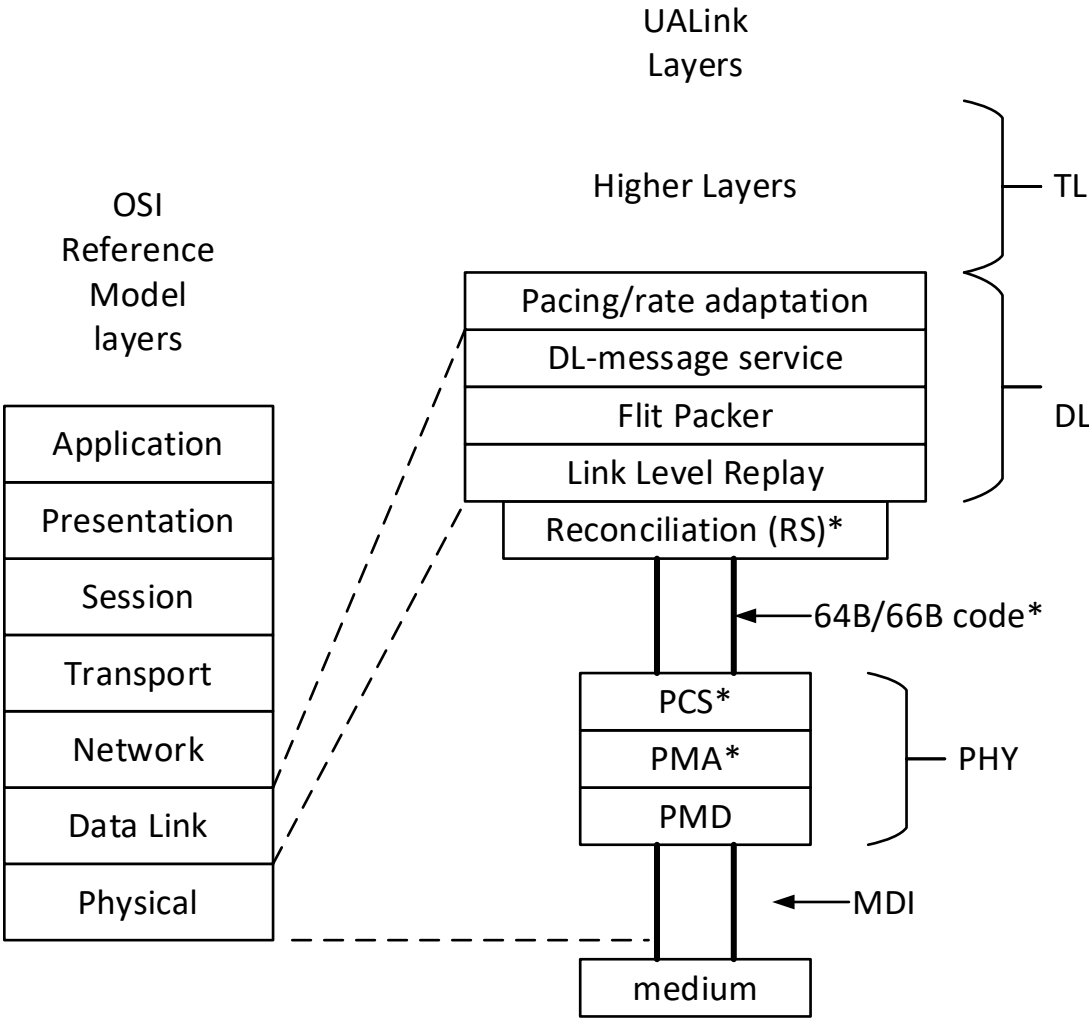


Figure 3-1 Physical Layer Block Diagram

Each port of a station can operate at a different data rate under certain error conditions and shall be capable of operating at a different data rates. The expected normal operation is that all Links of a Pod operate at the same data rate. All ports of a station shall be configured to operate at the same width. Port configuration, i.e. 1x4, 2x2, or 4x1 is determined prior to link training via front side network. All bifurcation modes should be supported. At least one bifurcation mode shall be supported.

The RS/PCS/PMA clauses from the 802.3 spec for 100Gbps serial that are defined for UALink are shown below. 100Gbps operation shall be supported.

| Rate             | RS  | FEC, PCS | PMA | interleave |
|------------------|-----|----------|-----|------------|
| 100GBASE-KR1/CR1 | 81  | 82, 91   | 135 | 1-way      |
| 200GBASE-KR2/CR2 | 117 | 119      | 120 | 2-way      |
| 400GBASE-KR4/CR4 | 117 | 119      | 120 | 2-way      |

**Table 3-1 100Gbps Serial Clauses**

The PCS/PMA clauses from the 802.3 spec for 200Gbps serial are shown below. 200Gbps shall be supported. These include the standard interleaves for 802.3 as well as latency reduced modes. The additional UALink latency reduced modes are strongly encouraged but not required.

| Rate             | RS  | PCS | PMA | Codeword interleave |                        |       |
|------------------|-----|-----|-----|---------------------|------------------------|-------|
|                  |     |     |     | 802.3               | UALink latency reduced |       |
| 200GBASE-KR1/CR1 | 117 | 119 | 176 | 4-way               | 2-way                  | 1-way |
| 400GBASE-KR2/CR2 | 117 | 119 | 176 | 4-way               | 2-way                  |       |
| 800GBASE-KR4/CR4 | 170 | 172 | 176 | 4-way               |                        |       |

**Table 3-2 200Gbps Serial Clauses**

## 3.2 Reconciliation Sublayer (RS)

### 3.2.1 Introduction

The 64B/66B block encoding is used between the RS and PCS to describe the optional logical interface between the DL sublayer and the physical layer device.

UALink RS supports a subset of functions based on 802.3 RS Clauses listed below for reference.

- Based on clause 81 for CGMII
- Based on clause 117 for 200GMII and 400GMII
- Based on clause 170 for 800GMII

UALink RS supports the following functionality:

- The RS adapts the Flit format of the DL into a stream of 64B/66B blocks.
- Each data direction is independent.
- The RS generates continuous data or control 64B/66B blocks on the transmit path and expects continuous data or control 64B/66B blocks on the receive path.
- The RS participates in link fault detection and reporting by monitoring the receive path for status reports that indicate an unreliable link and generating status reports on the transmit path to report detected link faults to the peer on the remote end of the connecting link.
- Indicate local/remote fault to the DL, to aid in DL link up, link down determination.
- Support only a subset of 802.3 64B/66B encodings.
- Provides DL Flit to codeword alignment. Each 640-byte Flit that the DL generates shall fit exactly into one RS(544,514) codeword.
- Indication to PCS when alignment markers shall be overwritten in the Tx direction, and indication of when alignment markers are received in the Rx direction.
- Indication to DL for start of receive Flit.
- Indication to DL if the received Flit is a control Flit or a data Flit. Only data Flits are passed to the DL. Receiving control Flits Vs Data Flits is used in the DL for link up, link down determination in the DL Link State Machine.
- Indication to DL for the type of control Flit received.
- Indication to the DL for PL ID, when operating with Link Resiliency
- The RS participates in the powering up and down of the PMD, supporting Link folding at the DL.
- Determination of when to send alignment markers (AM) or rapid alignment markers (RAM).

### 3.2.2 Data Flow

The UALink RS transmits a continuous stream of data or control 64B/66B blocks in the transmit direction and expects a continuous flow of data or control 64B/66B blocks on the receive path. When transmitting Data 64B/66B blocks they are back-to-back, there is no start or terminates blocks demarking the beginning and end of the DL Flit. A transmitted “Flit Code Sequence” consists of 80 consecutive 64B/66B code blocks. A transmitted Flit Code Sequence will have all data code blocks or all control code blocks. Data Flit Code Sequences originate and terminate at the DL. Control Flit Code Sequences originate and terminate at the RS.

From the DL perspective a Data Flit consists of 20, 256 blocks (640 bytes). These are logically serialized/deserialized by the RS into 80, 64B/66B, a Flit Code Sequence.

### 3.2.3 DL Flits

By default, DL Flit Code Sequences are transmitted by the RS, the DL shall always provide DL Flits (either NOP or payload) to the RS. Only under fault conditions, DL link down (Idle state) or for alignment marker insertion or Idle Codes for rate adaptation, will the RS transmit a Control Flit Code Sequence and block the DL from sending a DL Flit Code Sequence.

### 3.2.4 Control Flits

Control Flits Code Sequence are used for the following purposes:

1. Link fault signaling
  - a. Local fault
  - b. Remote fault
2. When to overwrite alignment markers in Tx Direction.
3. When a Flit contains alignment markers in Rx Direction.
4. Idle transmission or reception
  - a. for rate matching
  - b. in response to receiving remote fault.
5. Powering down the physical Layer.

#### 3.2.4.1 Idle Flit Code Sequence

An Idle Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with a block type of 0x1E, and every control code is /I/ in the 64B/66B block.

Idle Flit Code Sequences are transmitted as follows:

- When receiving remote fault indication.
- When the DL is in DL idle link state
- When the DL is in DL up or DL NOP states, see section 3.3.5 for rate matching

#### 3.2.4.2 Idle Start Flit Code Sequence

An Idle Start Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with the same content as an Idle Flit Code Sequence except the first eight transmitted 64B/66B control block has a block type of 0x78, and the data bytes are set as described below. In addition, when operating with Link Resiliency, the last 64B/66B control block has a block type of 0x78, and all data bytes are set to 0x00, with the exception of the D7 that is set to the PL ID.

- When an AM is scheduled:
  - All bytes in the first eight 64B/66B blocks set to 0x00
- When a RAM is scheduled and the transition to AM is not scheduled:
  - The bytes in the first 64B/66B block are set to 0xFF
  - The bytes in the second to eighth 64B/66B blocks, are set to 0x00.
- When a RAM is scheduled and the transition to AM is scheduled:
  - The bytes in the first 64B/66B block are set to the number of RAMs that remain until the scheduled AM
  - The bytes in the second to eighth 64B/66B blocks, are set to 0x00

Idle Start Flit Code Sequences are transmitted as follows:

- When the RS determines that it is time to send an alignment marker or rapid alignment marker, and the RS determines that it should not be sending Fault Flits, i.e. it is sending data Flits.

#### 3.2.4.3 Fault Flit Code Sequence

A fault Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with a block type of 0x4B, indicating a sequence ordered set. There are two types of fault Flit Code Sequences, local and remote.

Fault Flit Code Sequences are transmitted as follows:

- During certain fault states.

#### 3.2.4.4 Fault Start Flit Code Sequence

A Fault Start Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with the same content as a Fault Flit Code Sequence except the first eight transmitted 64B/66B control block has a block type of 0x78, and the data bytes are set as described below. In addition, when operating with Link Resiliency, the last 64B/66B control block has a block type of 0x78, and all data bytes are set to 0x00, with the exception of the D7 that is set to the PL ID.

- When an AM is scheduled:
  - All bytes in the first eight 64B/66B blocks set to 0x00
- When a RAM is scheduled and the transition to AM is not scheduled:
  - The bytes in the first 64B/66B block are set to 0xFF
  - The bytes in the second to eighth 64B/66B blocks, are set to 0x00.
- When a RAM is scheduled and the transition to AM is scheduled:
  - The bytes in the first 64B/66B block are set to the number of RAMs that remain until the scheduled AM
  - The bytes in the second to eighth 64B/66B blocks, are set to 0x00

Fault Start Flit Code Sequences are transmitted as follows:

- When the RS determines that it is time to send an alignment marker or rapid alignment marker, and the RS also determines that it should be sending Fault Flits.

#### 3.2.4.5 Start Flit Code Sequences

There are 3 types of start Flit Code Sequences, Idle Start Flit Code Sequences, Fault Start Flit Code Sequences, and PwrDn Start Flit Code Sequences. These Start Flit Code Sequences are transmitted RS to PCS to indicate the start of a Flit Code Sequence, and to indicate to the PCS when to overwrite an alignment marker. The Flit that is transmitted by the PCS will have the alignment markers in the first four to sixteen 256B/256B block codes, as a function of PCS clause.

#### 3.2.4.6 Alignment Marker (AM) Flit Code Sequence

Alignment marker Flit Code Sequences are only used in the Rx direction PCS to RS and are used by the RS to determine receive Flit alignment. An AM Flit Code Sequence consists of 80 consecutive 64B/66B control blocks. The first n transmitted 64B/66B control blocks have a block type of 0x78, and all data bytes are set to 0xFF. The remaining 64B/66B blocks contain the received 64B/66B control blocks. The first n 64B/66B control blocks contain the alignment markers and bypass the descrambler. The AM Flit code sequence is overwritten in the Rx direction before the transcoder. The transcoded version of the code sequence is used.

The value of  $n$  is a function of the length of the alignment markers, which is Clause and data rate dependent:

- Clause 91, 100G:  $5 \times 4 = 20$  64B/66B blocks
- Clause 119, 200G:  $4 \times 4 = 16$  64B/66B blocks
- Clause 119, 400G:  $8 \times 4 = 32$  64B/66B blocks
- Clause 172, 800G:  $16 \times 4 = 64$  64B/66B blocks

AM Flits are transmitted as follows:

- AM flit Code Sequences are sent only when the PCS has achieved alignment lock. They are sent when the PCS is expected or does receive a codeword containing the alignment marker. Bit errors may impact alignment marker detection, this occurs prior to FEC decode. The AM Flit replaces the received Flit containing the alignment marker.

#### 3.2.4.7 Power Down Flit Code Sequence

A PwrDn Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with a block type of 0xFF, and every data byte is 0x00.

PwrDn Flit Code Sequences are transmitted as follows:

- When powering down the physical layer

#### 3.2.4.8 Power Down Start Flit Code Sequence

A PwrDn Start Flit Code Sequence consists of 80 consecutive 64B/66B control blocks with the same content as an PwrDn Flit Code Sequence except the first eight transmitted 64B/66B control block has a block type of 0x78, and all data bytes are set to 0x00. In addition, the last 64B/66B control block has a block type of 0x78, and all data bytes are set to 0x00, with the exception of the D7 that is set to the PL ID.

PwrDn Start Flit Code Sequences are transmitted as follows:

- When the RS determines that it is time to send an alignment marker, and the RS determines that it should be sending a PwrDn Flit Code Sequence.

### 3.2.5 Data and Control Blocks and Codes

A subset of control blocks and codes are supported from 802.3 Clause 82. The supported block codes are shown below.

| Input Data                                                                                                              | Sync | Block Payload  |                |                |                |                |                |                |                |                |
|-------------------------------------------------------------------------------------------------------------------------|------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
|                                                                                                                         | 065  |                |                |                |                |                |                |                |                |                |
| Data Block Format                                                                                                       |      |                |                |                |                |                |                |                |                |                |
| D <sub>0</sub> D <sub>1</sub> D <sub>2</sub> D <sub>3</sub> D <sub>4</sub> D <sub>5</sub> D <sub>6</sub> D <sub>7</sub> | 01   | D <sub>0</sub> | D <sub>1</sub> | D <sub>2</sub> | D <sub>3</sub> | D <sub>4</sub> | D <sub>5</sub> | D <sub>6</sub> | D <sub>7</sub> |                |
| Control Block Format                                                                                                    |      | Block Type     |                |                |                |                |                |                |                |                |
| C <sub>0</sub> C <sub>1</sub> C <sub>2</sub> C <sub>3</sub> C <sub>4</sub> C <sub>5</sub> C <sub>6</sub> C <sub>7</sub> | 10   | 0x1E           | C <sub>0</sub> | C <sub>1</sub> | C <sub>2</sub> | C <sub>3</sub> | C <sub>4</sub> | C <sub>5</sub> | C <sub>6</sub> | C <sub>7</sub> |
| S <sub>0</sub> D <sub>1</sub> D <sub>2</sub> D <sub>3</sub> D <sub>4</sub> D <sub>5</sub> D <sub>6</sub> D <sub>7</sub> | 10   | 0x78           | D <sub>1</sub> | D <sub>2</sub> | D <sub>3</sub> | D <sub>4</sub> | D <sub>5</sub> | D <sub>6</sub> | D <sub>7</sub> |                |
| O <sub>0</sub> D <sub>1</sub> D <sub>2</sub> D <sub>3</sub> Z <sub>4</sub> Z <sub>5</sub> Z <sub>6</sub> Z <sub>7</sub> | 10   | 0x4B           | D <sub>1</sub> | D <sub>2</sub> | D <sub>3</sub> | O <sub>0</sub> | 0x000_0000     |                |                |                |
| D <sub>0</sub> D <sub>1</sub> D <sub>2</sub> D <sub>3</sub> D <sub>4</sub> D <sub>5</sub> D <sub>6</sub> T <sub>7</sub> | 10   | 0xFF           | D <sub>0</sub> | D <sub>1</sub> | D <sub>2</sub> | D <sub>3</sub> | D <sub>4</sub> | D <sub>5</sub> | D <sub>6</sub> |                |

Figure 3-2 64B/66B Block Codes

Data blocks use Sync header 01. Control blocks use sync header 10.

- Block type 0x1E is used for:
  - Idle, C<sub>0</sub> to C<sub>7</sub> contain control codes for /I/.
  - Error, C<sub>0</sub> to C<sub>7</sub> contain control codes for /E/.
- Block type 0x78 is used for:
  - start of Flit indication for Flits related to alignment markers, i.e., Idle Start Flit, Fault Start Flit, and AM Flit.
- Block type 0x4B is used for:
  - sequential ordered sets to indicate local or remote fault.
- Block type 0xFF is used for:
  - PwrDn Flit Code Sequence, with D<sub>0</sub>-D<sub>6</sub> = 0x0

The sequential ordered set definition is shown below.

| Lane0    | Lane1 | Lane2 | Lane3 | Lane4 | Lane5 | Lane6 | Lane7 | Description       |
|----------|-------|-------|-------|-------|-------|-------|-------|-------------------|
| Sequence | 0x00  | 0x00  | 0x00  | 0x00  | 0x00  | 0x00  | 0x00  | Reserved          |
| Sequence | 0x00  | 0x00  | 0x01  | 0x00  | 0x00  | 0x00  | 0x00  | Local fault       |
| Sequence | 0x00  | 0x00  | 0x02  | 0x00  | 0x00  | 0x00  | 0x00  | Remote fault      |
| Sequence | 0x00  | 0x00  | 0x03  | 0x00  | 0x00  | 0x00  | 0x00  | Link Interruption |

Table 3-3 Sequential Ordered Sets

The following control codes are supported.

| Control Character | Notation | 64B/66B code |
|-------------------|----------|--------------|
| Idle              | /I/      | 0x00         |
| Error             | /E/      | 0x1E         |

### 3.2.6 Link fault signaling

Link fault signaling follows the 802.3 clause 81.3.4 with the exceptions described below.

Sublayers within the PHY are capable of detecting faults that render a link unreliable for communication. Upon recognition of a fault condition a PHY sublayer indicates Local Fault status on the data path. When this Local Fault status reaches an RS, the RS stops sending DL data Flits, and continuously generates a Remote Fault Flits on the transmit data path. When a Remote Fault Flit is received by an RS, the RS stops sending DL data Flits, and continuously generates Idle Flits. When the RS no longer receives fault status messages, it returns to normal operation, sending DL data Flits.

In the transmit direction all Link Fault indication shall transition on a Flit boundary. In the receive direction Link Fault indication is not required to transition on a Flit boundary, Retimers may not transition on a Flit boundary.

- Note: Standard Ethernet Retimers are not Flit aware.

The Link Fault state diagram in 81.3.4 is modified as follows:

- When an AM Flit is received col\_cnt is not updated.
- 64B/66B encoding is used.
- When operating with Link Resiliency and a local fault is indicated, the FAULT state to INIT state transition occurs only if the RS has indicated the PL ID to the DL, based on the recent AM Flits. In other words, the RS stays in the FAULT state until the RS indicates the PL ID to the DL.

### 3.2.7 Flit and Lane Alignment

In the transmit direction Start Flit Code Sequences are send at the interval required for the PCS to transmit alignment markers as a function of data rate:

- 100G: every 4,096 Flits
- 200G: every 4,096 Flits
- 400G: every 8,192 Flits
- 800G: every 16,384 Flits

The PCS shall overwrite the first n 257-bit blocks with alignment markers as a function of data rate:

- 100G: the first 5, 257-bit blocks
- 200G: the first 4, 257-bit blocks
- 400G: the first 8, 257-bit blocks
- 800G: the first 16, 257-bit blocks

The Start Flit Code Sequence in the transmit direction provides Flit synchronization to the PCS, so that each Flit is packed into a RS(5440, 5140) codeword.

In the Receive direction the alignment markers provide lane deskew and reorder to the PCS. In addition, the Start Flit Code Sequence is converted to an AM Flit Code Sequence by the PCS, and this provides Flit synchronization for the RS.

### 3.2.8 Receive State Machine

The RS in the Rx direction shall determine the alignment of each group of 80 \* 64B/66B blocks that are destined for the DL. The receipt of the first 64B/66B block from the AM Flit indicates the nominal timing for each group of 80 \* 64B/66B blocks. Due to rate matching and Idle insertion or deletion this timing may need to be adjusted.

If Idles are inserted between DL Flits in the PCS for rate matching, then these Idles are consumed by the RS, the sync header indicates 0b10 for control codes. In addition, Idles are also found in the groups of the 80 \* 64B/66B blocks corresponding to Idle Flit Code Sequence, Idle Start Flit Code Sequence, or AM Flit Code Sequence. These are also consumed by the RS.

If Idles are removed from Idle Flit Code Sequence, Idle Start Flit Code Sequence, or AM Flit Code Sequence, the DL Flit will occur earlier than the nominal 80 \* 64B/66B block.

When the RS receives a sync header of 0b01 indicating data and the previous sync header was 0b10 indicating control, this always indicates the start of a Data Flit and all 80 \* 64B/66B blocks are sent to the DL. When the RS receives a sync header of 0b01 indicating data and a Data Flit was just received and sent to the DL, then the next 80 \* 64B/66B blocks are sent to the DL. This indicates the typical case of back-to-back DL Flits.

### 3.2.9 Link Power down

This state is entered when the DL signals the RS to send PwrDn control Flits. When the DL signals the RS to transmit PwrDn Control Flits, the RS will do so and block DL Flits.

While the DL is signaling the RS to transmit PwrDn Control Flits the Link fault signaling is suspended, see section 3.2.6 Link fault signaling. The RS will transmit PwrDn Control Flits, independent of receiving local fault or remote fault. The RS will continue to report the reception of local and remote faults to the DL.

The DL signals the PL to power down when DL enters DL PwrDn. See section 2.9.2.

### 3.2.10 Link Power up

This state is entered when the DL stops signaling the RS to send PwrDn control Flits. When the DL signals the RS to stop transmitting PwrDn Control Flits, the RS will do so and resume normal operation.

The DL signals the PL to power up from a power down state, the Tx and Rx shall be powered up and the saved Tx and Rx settings are applied. Link training and auto negotiation shall not occur. Target time for powering up the Tx and Rx and the physical layer achieving alignment lock is < 250us.

The RS sends RAM to speed up alignment at the link partner's receiver. When an Idle Flit is received this indicates that both the link partner and the local device have achieved alignment lock. At this point the RS schedules the next AM and transmits the last 7 RAMs with the appropriate am\_next\_count value, see Section 3.3.7, and then transitions to sending AMs. The maximum time to transition to AMs is 8 RAM periods. The RS shall support a debug register to determine if RAM or normal AM are sent during power up, the default setting is RAMs.

## 3.3 PCS/PMA modifications

### 3.3.1 Introduction

The various PCS/PMA clauses of 802.3 are modified in several ways:

*Physical Layer*

1. Alignment markers frequency is determined by the RS, and they are used in the normal purpose of lane deskew and lane reorder, and in addition to align Flits to RS(544, 514) codewords.
2. Provide reduced codeword interleave ways to reduce latency.
3. No decode encode, the interface is specified at 64B/66B encoding.

### 3.3.2 DL Flit to PCS codeword alignment

In Order to maintain DL Flit to PCS codeword alignment The RS designates an entire Flit for the purpose of transmitting the alignment markers. The PCS operation is slightly different than 802.3. Alignment markers are not inserted which would impact Flit to codeword alignment. Alignment markers are overwritten into the designated Start Flit Code Sequences.

In the transmit direction the PCS overwrites the first several 257-bit blocks (same step as alignment insertion) of the Start Flit Code Sequence with the alignment markers specified in the 802.3 clauses (91, 119, 172). The PCS uses the Start Flit Code Sequence to align groups of 80, 64B/66B blocks into one RS(544, 514) code word.

In the receive direction the Flit Code Sequences containing (or should contain) the alignment markers are converted to AM Flit Code Sequences in the PCS and used for Flit alignment by the RS. Flit Code Sequences are sent only when the PCS has achieved alignment. The AM Flit Code Sequence replaces the received Flit Code Sequence according to the alignment lock determined in the PCS.

The alignment markers on the wire are identical to 802.3, and serve the same purpose, lane deskew, and an additionally provide DL Flit alignment on Receive.

When developing the IP that exposes the 64B/66B interface between RS and PCS the DL Flit to PCS codeword alignment shall implement as specified in this chapter.

When developing IP that does not expose the 64B/66B interface, the DL Flit to codeword alignment may be implemented with a different mechanism, providing the behavior is identical on the wire, e.g. one DL Flit is contained in one codeword, and the codewords containing the alignment markers are as specified. For example, an implementation could have the PCS alignment insertion logic send a pulse to the RS indicating when it is inserting the alignment marker, and the RS would send the appropriate 256B/257B blocks, such that the insertion occurs just before the appropriate 256B/257B blocks. On the receive side the alignment markers are removed, and a side band signal indicates this, such that the RS can determine DL Flit alignment.

### 3.3.3 Reduced FEC interleave

Additional PCS/PMA behavior at 200G and 400G that is different than 802.3 is defined as shown below. These options have less codeword interleave, and thus faster FEC decode time, with higher post FEC BER.

| Rate             | RS  | PCS  | PMA  | interleave |
|------------------|-----|------|------|------------|
| 200GBASE-KR1/CR1 | 117 | 119* | 176* | 1-way      |
| 200GBASE-KR1/CR1 | 117 | 119  | 176* | 2-way      |
| 400GBASE-KR2/CR2 | 117 | 172  | 176* | 2-way      |

Table 3-4 Reduced Interleave FEC

\* Modified clauses for interleave.

### 3.3.4 Decode Encode

UALink is specified at the 64B/66B encoding level, and such encode/decode from xGMI is not defined.

### 3.3.5 Rate Matching

The Tx RS injects Idle Flit code sequences at a rate of every 1024 codewords when the DL is in the DL up or DL NOP state, except when a codeword containing the alignment markers or rapid alignment markers are sent. The codeword containing the alignment markers or rapid alignment markers contain Idle Codes, which can be used for rate matching. The rate of rapid alignment markers is such that no Idle Flit code sequences are injected while rapid alignment markers are sent, see section 3.3.7 for transmit rate of rapid alignment markers.

- Note: there are no natural idle code blocks with UALink as the DL Flits are packed exactly into a code word. With Ethernet there is IPG which results in sufficient idle code blocks for rate matching or clock compensation on the Rx.

When the DL is in the DL Idle or DL Fault state, there are continuous Idle codes, or sequence ordered sets to remove/add if needed for rate adaption.

See IEEE 802.3 clauses, 82.2.3.6 Idle (/I/) and 82.2.3.9 ordered set (/O/), for rules on rate matching.

Addition or removal of Idle codes or removal of sequence ordered sets, in the Rx direction, shall occur on a 64B/66B block basis. This rule is more restrictive than clauses 82.2.3.6.

- Note: implementations that do not expose IP and the 64B/66B interface may choose to use a 256B/257B interface to skip the transcoding step. In this case the addition or removal of Idle codes or removal of sequence ordered sets shall occur on a four naturally aligned 64B/66B block or a single 256B/257B block.

The insertion of Idle Codes, in the Rx direction, may occur between Data Flits and shall not occur in the middle of a Data Flit. Idle codes may be deleted from codewords containing alignment markers, or from an Idle code sequence, as needed.

The following subsections describe the transmit sequence when the DL is in the DL up or DL NOP state.

#### 3.3.5.1 Tx Sequence 200GBASE-KR1/CR1 and 200GBASE-KR2/CR2 and 100GBASE-KR1/CR1

The Tx sequence is as follows; alignment markers sent every 4096 codewords.

1. Codeword 0: Idle Start Flit Code Sequence, i.e. alignment marker indication
2. Codeword 1-1023: NOP or payload Flits
3. Codeword 1024: Idle Flit Code Sequence
4. Codeword 1025- 2047: NOP or payload Flits
5. ...
6. Codeword 4096: Idle Start Flit Code Sequence, i.e. alignment marker indication
7. ...

#### 3.3.5.2 Tx Sequence 400GBASE-KR2/CR2 and 400GBASE-KR4/CR4

The Tx sequence is as follows; alignment markers sent every 8192 codewords.

1. Codeword 0: Idle Start Flit Code Sequence, i.e. alignment marker indication
2. Codeword 1-1023: NOP or payload Flits
3. Codeword 1024: Idle Flit Code Sequence

4. Codeword 1025- 2047: NOP or payload Flits
5. ...
6. Codeword 8192: Idle Start Flit Code Sequence, i.e. alignment marker indication
7. ...

### 3.3.5.3 Tx Sequence 800GBASE-KR4/CR4

The Tx sequence is as follows; alignment markers sent every 16384 codewords.

1. Codeword 0: Idle Start Flit Code Sequence, i.e. alignment marker indication
2. Codeword 1-1023: NOP or payload Flits
3. Codeword 1024: Idle Flit Code Sequence
4. Codeword 1025- 2047: NOP or payload Flits
5. ...
6. Codeword 16384: Idle Start Flit Code Sequence, i.e. alignment marker indication
7. ...

### 3.3.6 Back-to-Back DL Flits

640-byte DL Flits are sent continuously, there is no IPG, start or terminate between DL Flits. With the exceptions noted above for control Flits, i.e. alignment markers and rate matching.

### 3.3.7 Rapid Alignment Markers

IEEE 802.3 Clause 119 does not define rapid alignment markers (RAMs). Rapid alignment markers are required to support Link Folding.

RAMs use the same format as the alignment markers described in IEEE 802.3 Clause 119.2.4.4. UP0 for Lane 0 is XOR'd with `am_next_count`. The `am_next_count` variable indicates how many RAM intervals until the next scheduled AM. If an AM is not scheduled, then `am_next_count` is 0xFF.

Rapid alignment markers are sent 128 times more frequently than AMs; this speeds up alignment lock in the Rx PCS. The AM and RAM period is shown below in code words (CW) and ns. This yields 819 ns between RAMs at 200G serial, compared to 100us for AMs.

|                 | 100G Serial |     |     | 200G serial |     |     |
|-----------------|-------------|-----|-----|-------------|-----|-----|
| lanes           | 1           | 2   | 4   | 1           | 2   | 4   |
| AM period (CW)  | 4K          | 4K  | 8K  | 4K          | 8K  | 16K |
| RAM period (CW) | 32          | 32  | 64  | 32          | 64  | 128 |
| RAM Period (ns) | 1638        | 819 | 819 | 819         | 819 | 819 |

Table 3-5 RAM period

The transmit AM alignment between PL[A] and PL[B] before and after the power down shall be the same, and the RAMs that are initially sent during power up must be aligned to the ultimate AM placement.

The rules for transmitting RAMs/AMs are described below:

1. When the link is powered up after the power down sequence RAMs are sent with `am_next_count` = 0xFF.
2. After an Idle Flit is received 7 RAMs are sent, with required interval, with `am_next_count` pointing to when the AM is scheduled to start.
3. The first RAM sent shall have an `am_next_count` such that the 3 lsbs are equal to 6. There are 7 RAMs sent and thus the last RAM sent will have an `am_next_count` with the 3 lsbs set to 0.

4. The first RAM sent shall have an `am_next_count` > 7, if not 128 is added. For example, if the first AM is 3 RAM periods away, then the first `am_next_count` is equal to  $128 + 3 = 131$ . This allows 7 RAMs to be sent that point to the second AM.
5. After the last RAM is sent with `am_next_count` pointing to when the AM is scheduled to start, AMs are sent.

AM lock follows IEEE 802.3 alignment marker lock state diagram, figure 119-12, and RAM lock follows IEEE 802.3 alignment marker lock state diagram, figure 119-12 with `amp_counter` divided by 128.

The rules for the alignment lock when Link Folding is enabled are as follows:

1. If not all the PCS lanes are locked (!`all_locked`), for both all AMs and all RAMs, then:
  - a. `all_locked_rams` = false
  - b. `all_locked_ams` = false
  - c. All PCS lanes attempt to lock to both RAM and AM
    - i. If all PCS lanes lock to RAMs, then `all_locked_rams` = true
    - ii. If all PCS lanes lock to AMs, and no PCS lanes lock to RAMs then `all_locked_ams` = true
2. Else If `all_locked_rams`, then:
  - a. Only consider RAMs for alignment lock
  - b. If two RAMs are received on PCS lane 0 with `am_next_count` pointing to the same location for when the AM should start, then:
    - i. AMs shall be used for alignment lock
    - ii. `all_locked_rams` = false
    - iii. `all_locked_ams` = true
3. Else If `all_locked_ams`, then:
  - a. Only consider AMs for alignment lock

The above rules are applicable to both PCS alignment lock, and PMA symbol pair alignment.

The diagram below describes the transition from transmitted RAMs to AM on PL[A] for 200G-KR1/CR1, i.e. RAMs spaced at 32 Flit times. `RAM[i]` indicates a RAM with `am_next_count` XOR'd into UP0. `RAM[9]` means 9 more RAM periods until AM. Seven RAMs are sent with `am_next_count` = [14, 13, 12, 11, 10, 9, 8], only [9, 8] are shown. The 3 lsb's of the `am_next_count` indicate when the last RAM is sent, i.e. when they are 0. This is used by the receiver to stop processing RAMs and is robust in the event of RAM code word errors.

The RS shall not request DL Flits for transmission until the last RAM is sent, otherwise the link partners DL reorder will be miss ordered, due to Control Flits injected in PL[A] but not in PL[B].

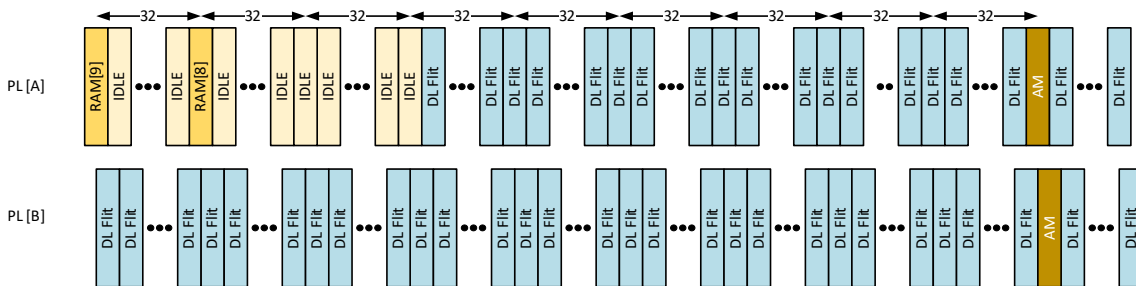


Figure 3-3 RAM to AM transition

### 3.3.8 PMA Symbol Pair Demultiplexing

IEEE 802.3dj clause 176 introduces symbol multiplexing between PCS lanes. Symbol pair multiplexing requires symbol pair alignment on the receiver; this is accomplished using PCS lane alignment markers. The 20-bit demultiplexer has 20 possible alignments, and each alignment is tested to see if the PCS alignment markers can be locked. A serial search for symbol pair alignment could take 20 times 3 alignment marker intervals: up to 60 alignment marker intervals, or ~6ms. Each possible symbol pair alignment is checked for 3 alignment marker intervals.

The PMA shall consider all 20 possible symbol pair alignments in parallel. Thus, symbol alignment should occur within 3 alignment marker intervals, or ~300us.

### 3.4 PCS and FEC for 100GBASE-R

802.3 PCS Clause 82 and FEC Clause 91 are used for 100GBASE-KR1/CR1. Shown below is a simplified block diagram. Simplified in that UALink does not include the conversion of 20 PCS lanes into 4 PCS lanes, and the mapping of alignment markers. Colored blocks indicate changes from 802.3.

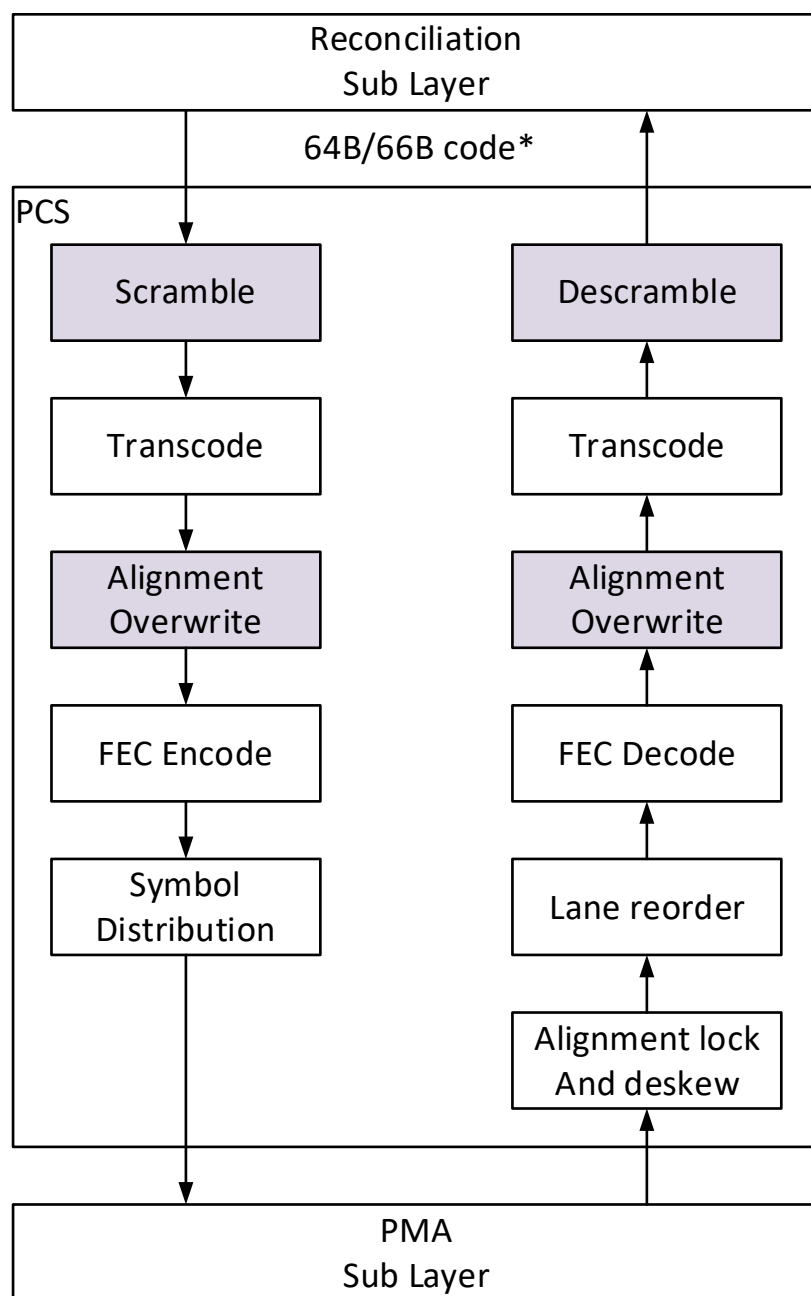


Figure 3-4 100GBASE-R

### 3.4.1 Removed functional Blocks

Several functional blocks are made redundant when Clause 91 is added to clause 82. These are not present with UALink.

#### 3.4.1.1 Transmit direction

- Clause 82.2.6 Block distribution
- Clause 82.2.7 Alignment marker Insertion
- Clause 91.5.2.1 Lane block synchronization
- Clause 91.5.2.2 Alignment lock and deskew
- Clause 91.5.2.3 Lane reorder

*Physical Layer*

- Clause 91.5.2.4 Alignment marker removal

#### **3.4.1.2 Receive Direction**

- Clause 91.5.3.6 Block distribution
- Clause 91.5.3.7 Alignment marker mapping and insertion
- Clause 82.2.12 block synchronization
- Clause 82.2.13 PCS Lane deskew
- Clause 82.2.14 PCS Lane reorder
- Clause 82.2.15 alignment marker removal

### **3.4.2 Transmit Function**

#### **3.4.2.1 Scrambler**

See 802.3 clause 82.2.5, with the following additional requirements. The Start Flit Code Sequence's first twenty 64B/66B blocks bypass the scrambler, and the scrambler is not advanced. This allows the Alignment Overwrite block to decode the Start Flit Code Sequence and overwrite the first five 256B/257B blocks with the required alignment marker.

#### **3.4.2.2 Transcode**

See 802.3 clause 91.5.2.5.

#### **3.4.2.3 Alignment Marker Overwrite**

When a Start Flit Code Sequence is detected the alignment marker overwrite replaces the first five 256B/257B blocks with the alignment marker specified in 91.5.2.6.

#### **3.4.2.4 FEC Encode**

See 802.3 clause 91.5.2.7. Only RS(544, 514) is supported.

#### **3.4.2.5 Symbol distribution**

See 802.3 clause 91.5.2.8.

#### **3.4.2.6 Transmit bit ordering**

See 802.3 clause 91.5.2.9.

### **3.4.3 Receive Function**

#### **3.4.3.1 Alignment Lock and Deskew**

See 802.3 91.5.3.1. Static and dynamic deskew not required, this is a single lane.

#### **3.4.3.2 Lane Reorder**

See 802.3 clause 91.5.3.2.

#### **3.4.3.3 FEC decode**

See 802.3 clause 91.5.3.3. Only RS(544, 514) is supported.

#### **3.4.3.4 Alignment Overwrite**

The Flit Code Sequence that is expected to contain the alignment markers (there could be bit errors) based on the alignment lock, is converted to an AM Flit Code Sequence.

#### **3.4.3.5 Transcoder**

See 802.3 clause 91.5.3.5.

#### 3.4.3.6 Descrambler

See 802.3 clause 82.2.16, with the following additional requirements. The first twenty 64B/66B blocks of an AM Flit Code Sequence bypass the descrambler, and the descrambler is not advanced. The remaining sixty 64B/66B blocks of an AM Flit Code Sequence are descrambled, and the descrambler is advanced. This enables the RS to decode the AM Flit, and the Rx descrambler to stay synchronized with the Tx scrambler.

### 3.5 PCS for 200GBASE-R and 400GBASE-R

802.3 Clause 119 PCS is used for 200GBASE-KR1/CR1, 200GBASE-KR2/CR2, 400GBASE-KR2/CR2 and 400GBASE-KR4/CR4. Shown below is the modified block diagram to support UALink. Colored blocks indicate changes from 802.3.

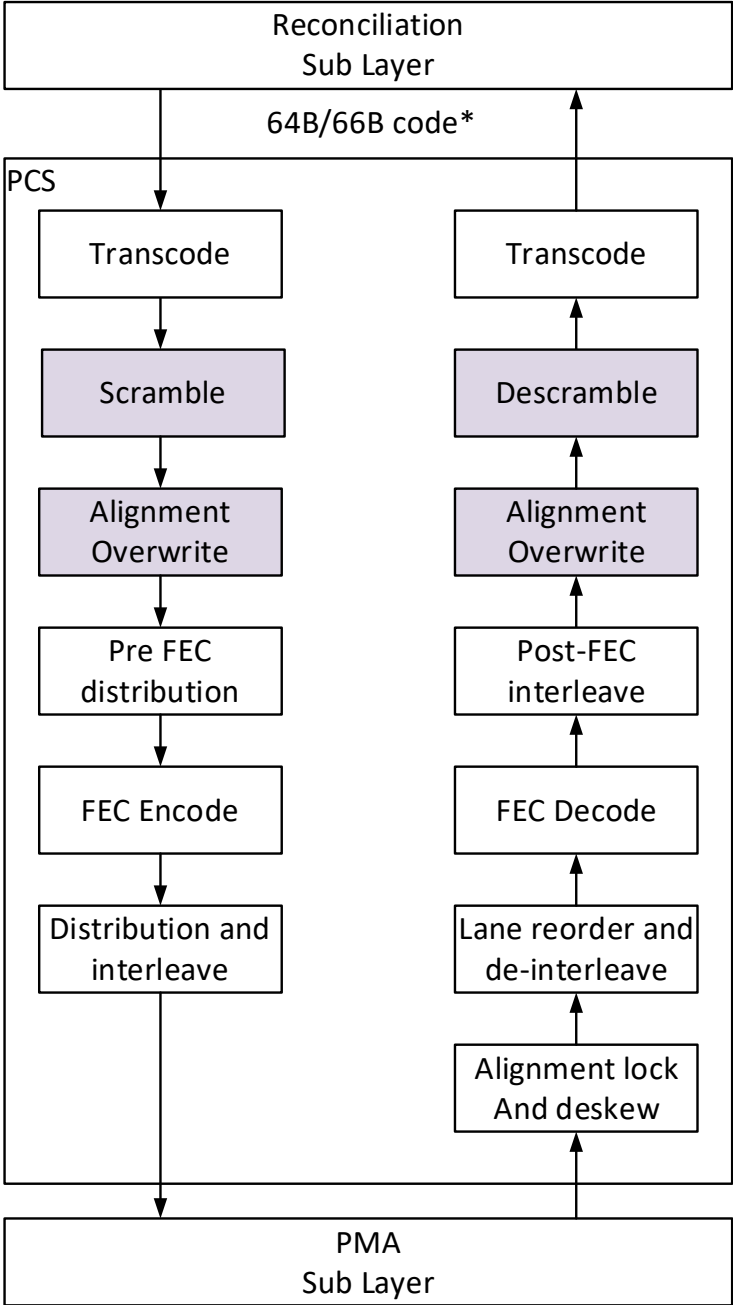


Figure 3-5 200GBASE-R & 400GBASE-R

#### 3.5.1 Transmit Function

##### 3.5.1.1 Transcode

See 802.3 clause 119.2.4.2.

### 3.5.1.2 Scrambler

See 802.3 clause 119.2.4.3, with the following additional requirements. The Start Flit Code Sequence's first four/eight 256B/257B blocks bypass the scrambler, for 200G/400G operation. This allows the Alignment Overwrite block to decode the Start Flit Code Sequence and overwrite the first four or eight 256B/257B blocks with the required alignment marker.

### 3.5.1.3 Alignment Marker Overwrite

When a Start Flit Code Sequence is detected the alignment marker overwrite replaces the first four 256B/257B blocks with the alignment marker specified in 119.2.4.4.1 for 200GBASE-R, or the first eight 256B/257B blocks with the alignment marker specified in 119.2.4.4.2 for 400GBASE-R.

### 3.5.1.4 Pre-FEC distribution

See 802.3 clause 119.2.4.5.

### 3.5.1.5 FEC Encoder

See 802.3 clause 119.2.4.6.

### 3.5.1.6 Distribution and interleave

See 802.3 clause 119.2.4.7.

### 3.5.1.7 Transmit bit ordering

See 802.3 clause 119.2.4.8.

### 3.5.1.8 Test pattern generator

See 802.3 clause 119.2.4.9.

## 3.5.2 Receive Function

### 3.5.2.1 Alignment Lock and deskew

See 802.3 clause 119.2.5.1.

### 3.5.2.2 Lane reorder and de-interleave

See 802.3 clause 119.2.5.2.

### 3.5.2.3 FEC decoder

See 802.3 clause 119.2.5.3.

### 3.5.2.4 Post FEC interleave

See 802.3 clause 119.2.5.4.

### 3.5.2.5 Alignment Marker Overwrite

The Flit Code Sequence that is expected to contain the alignment markers (there could be bit errors) based on the alignment lock, is converted to an AM Flit Code Sequence.

### 3.5.2.6 Descrambler

See 802.3 clause 119.2.5.6, with the following additional requirements. The first four/eight 256B/257B blocks of an AM Flit Code Sequence bypass the descrambler, and the descrambler is not advanced, for 200G/400G operation. The remaining sixteen/twelve 256B/257B blocks of an AM Flit Code Sequence are descrambled, and the descrambler is advanced. This enables the RS to decode the AM Flit Code Sequence, and the Rx descrambler to stay synchronized with the Tx scrambler.

#### 3.5.2.7 Transcoder

See 802.3 clause 119.2.5.7.

### 3.6 PCS for 800GBASE-R

802.3 Clause 172 PCS is used for 800GBASE-KR4/CR4. Shown below is the modified block diagram to support UALink. Colored blocks indicate changes from 802.3.

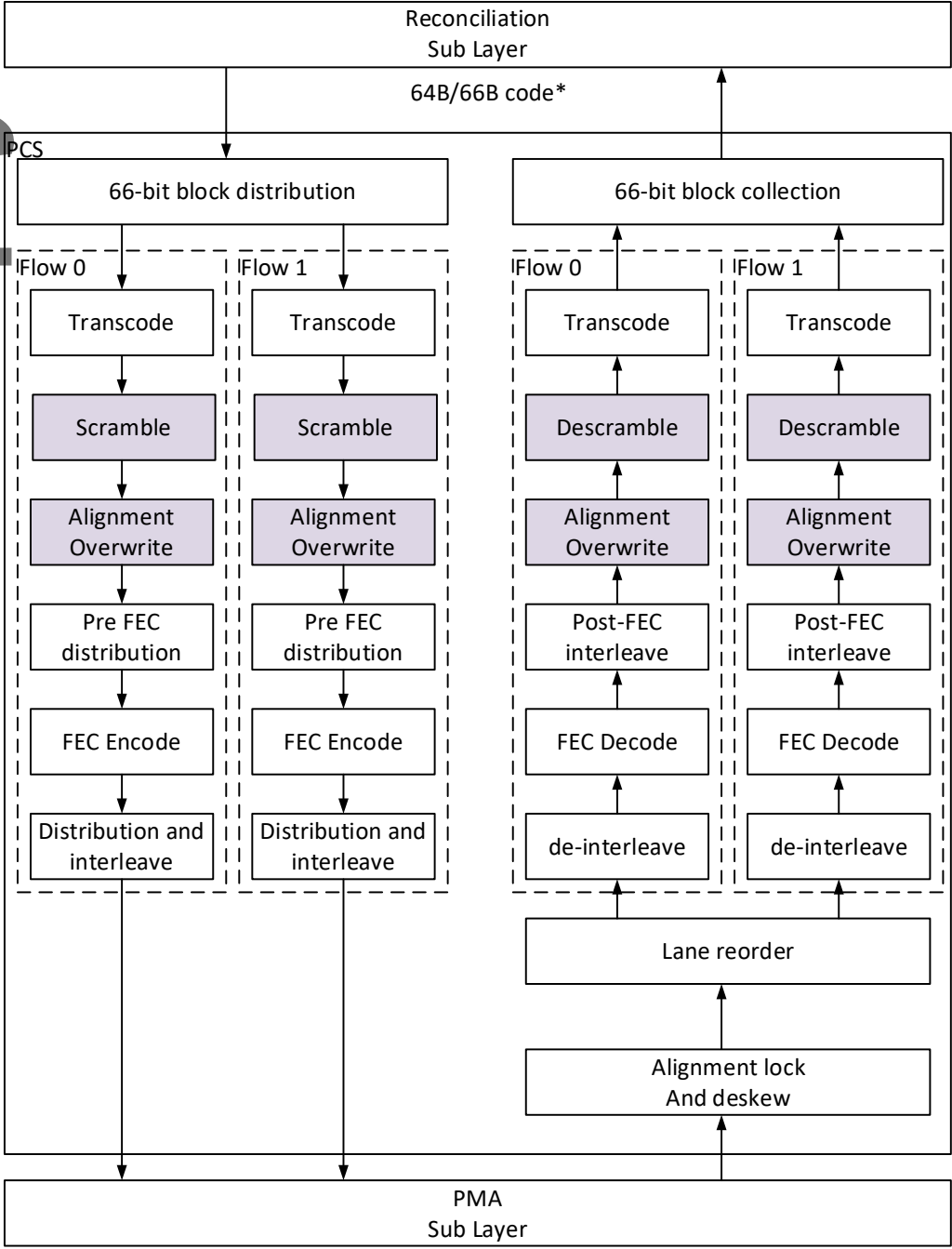


Figure 3-6 800GBASE-R

### 3.6.1 Transmit Function

#### 3.6.1.1 Block Distribution

See 802.3 clause 172.2.4.3.

#### 3.6.1.2 Transcoder

See 802.3 clause 172.2.4.4.

### 3.6.1.3 Scrambler

See 802.3 clause 172.2.4.5, with the following additional requirements. The Start Flit Code Sequence's first sixteen 256B/257B blocks bypass the scramblers. There are two flows, each flow bypasses the eight 256B/257B blocks. This allows the Alignment Overwrite block to decode the Start Flit Code Sequence and overwrite the first eight 256B/257B blocks (per flow) with the required alignment markers.

### 3.6.1.4 Alignment Marker Overwrite

When a Start Flit Code Sequence is detected the alignment marker overwrites the first eight 256B/257B blocks with the alignment marker specified in 172.2.4.6. There are two flows, each flow overwrites the first eight 256B/257B blocks, see figure 172-3 in 802.3 spec.

### 3.6.1.5 Pre-FEC distribution

See 802.3 clause 172.2.4.7.

### 3.6.1.6 FEC Encoder

See 802.3 clause 172.2.4.8.

### 3.6.1.7 Symbol distribution

See 802.3 clause 172.2.4.9.

### 3.6.1.8 Transmit bit ordering

See 802.3 clause 172.2.4.10.

### 3.6.1.9 Test Pattern Generation

See 802.3 clause 172.2.4.11.

## 3.6.2 Receive Function

### 3.6.2.1 Alignment lock and deskew

See 802.3 clause 172.2.4.1.

### 3.6.2.2 Lane reorder and de-interleave

See 802.3 clause 172.2.4.2.

### 3.6.2.3 FEC decode

See 802.3 clause 172.2.4.3.

### 3.6.2.4 Post FEC interleave

See 802.3 clause 172.2.4.4.

### 3.6.2.5 Alignment Marker Overwrite

The Flit Code Sequence that is expected to contain the alignment markers (there could be bit errors) based on the alignment lock, is converted to an AM Flit Code Sequence. There are two flows, each flow creates  $\frac{1}{2}$  an Interleaved AM Flit Code Sequence. Even 64B/66B blocks on flow 0, odd 64B/66B blocks on flow 1. Each  $\frac{1}{2}$  AM Flit Code Sequence is identical and contains the following:

- Thirty-two 64B/66B block type of 0x78, and all data bytes are set to 0xF
  - Transcoded to Eight 256B/257B blocks
- Eight 64B/66B block type of 0x78, and all data bytes are set to 0xFF
  - Transcoded to Two 256B/257B blocks

### 3.6.2.6 Descrambler

See 802.3 clause 172.2.4.6, with the following additional requirements. The first eight 256B/257B blocks (per flow) of an AM Flit Code Sequence bypass the descrambler, and the descrambler is not advanced. The remaining twelve 256B/257B blocks of an AM Flit Code Sequence are descrambled, and the descrambler is advanced. This enables the RS to decode the AM Flit, and the per flow Rx descrambler to stay synchronized with the per flow Tx scrambler.

### 3.6.2.7 Transcoder

See 802.3 clause 172.2.4.7.

### 3.6.2.8 Block collection

See 802.3 clause 172.2.4.8.

## 3.7 Low Latency FEC Interleave

### 3.7.1 400GBASE-KR2/CR2 (2-way interleave)

802.3 specification defines 4-way interleave of 400GBASE-KR2/CR2. 2-way interleave occurs in the PCS via 10-bit distribution to two FEC encoders, see Clause 119.2.4.8. An additional 2-way interleave occurs in the PMA via delaying odd symbols 2 codewords, see Clause 176.4.2.4.2, for a total of 4-way interleaving.

UALink defines a mode of operation for 2-way interleave.

- Clause 176.4.2.4.2 in the transmit direction is modified as follows: The PMA does not perform the additional two codeword delay.
- Clause 176.4.3.3 in the receive direction is modified as follows: The PMA adds a delay of only one FEC symbol to each even lane (not 69 FEC symbols).

### 3.7.2 200GBASE-KR1/CR1 (2-way interleave)

802.3 specification defines 4-way interleave of 200GBASE-KR1/CR1. 2-way interleave occurs in the PCS via 10-bit distribution to two FEC encoders, see Clause 119.2.4.8. An additional 2-way interleave occurs in the PMA via delaying odd symbols 2 codewords, see Clause 176.4.2.4.2, for a total of 4-way interleaving.

UALink defines a mode of operation for 2-way interleave.

- Clause 176.4.2.4.2 in the transmit direction is modified as follows: The PMA does not perform the additional two codeword delay.
- Clause 176.4.3.3 in the receive direction is modified as follows: The PMA adds a delay of only one FEC symbol to each even lane (not 137 FEC symbols).

### 3.7.3 200GBASE-KR1/CR1 (1-way interleave)

802.3 specification defines 4-way interleave of 200GBASE-KR1/CR1. 2-way interleave occurs in the PCS via 10-bit distribution to two FEC encoders, see Clause 119.2.4.8. An additional 2-way interleave occurs in the PMA via delaying odd symbols 2 codewords, see Clause 176.4.2.4.2, for a total of 4-way interleaving.

UALink defines a mode of operation for 1-way interleave.

- Clause 176.4.2.4.2 in the transmit direction is modified as follows: The PMA does not perform the additional two codeword delay.

- Clause 176.4.3.3 in the receive direction is modified as follows: The PMA adds a delay of one FEC symbol to each even lane (not 137 FEC symbols).

Note: RS symbols are interleaved in symbol-pairs (consistent with CL176).

In addition, the PCS does not perform 10-bit symbol distribution to two FEC encoders, to reduce to 1-way interleave.

### 3.7.3.1 PCS Transmit changes

Clause 119.2.4.4.1 AM creation for the 200GBASE-R PCS is modified. The variable `am_mapped` is derived from 10-bit interleaving the group of 8 alignment markers `am_x`

per the following procedure:

For all  $k=0$  to 11

For all  $j=0$  to 7

$am\_mapped\langle 80k+10j+9:80k+10j \rangle = am\_j\langle 10k+9:10k \rangle$

Clause 119.2.4.5 Pre-FEC distribution is not performed and a single FEC encoder is used. 802.3 figure 119-10 is updated as follows.

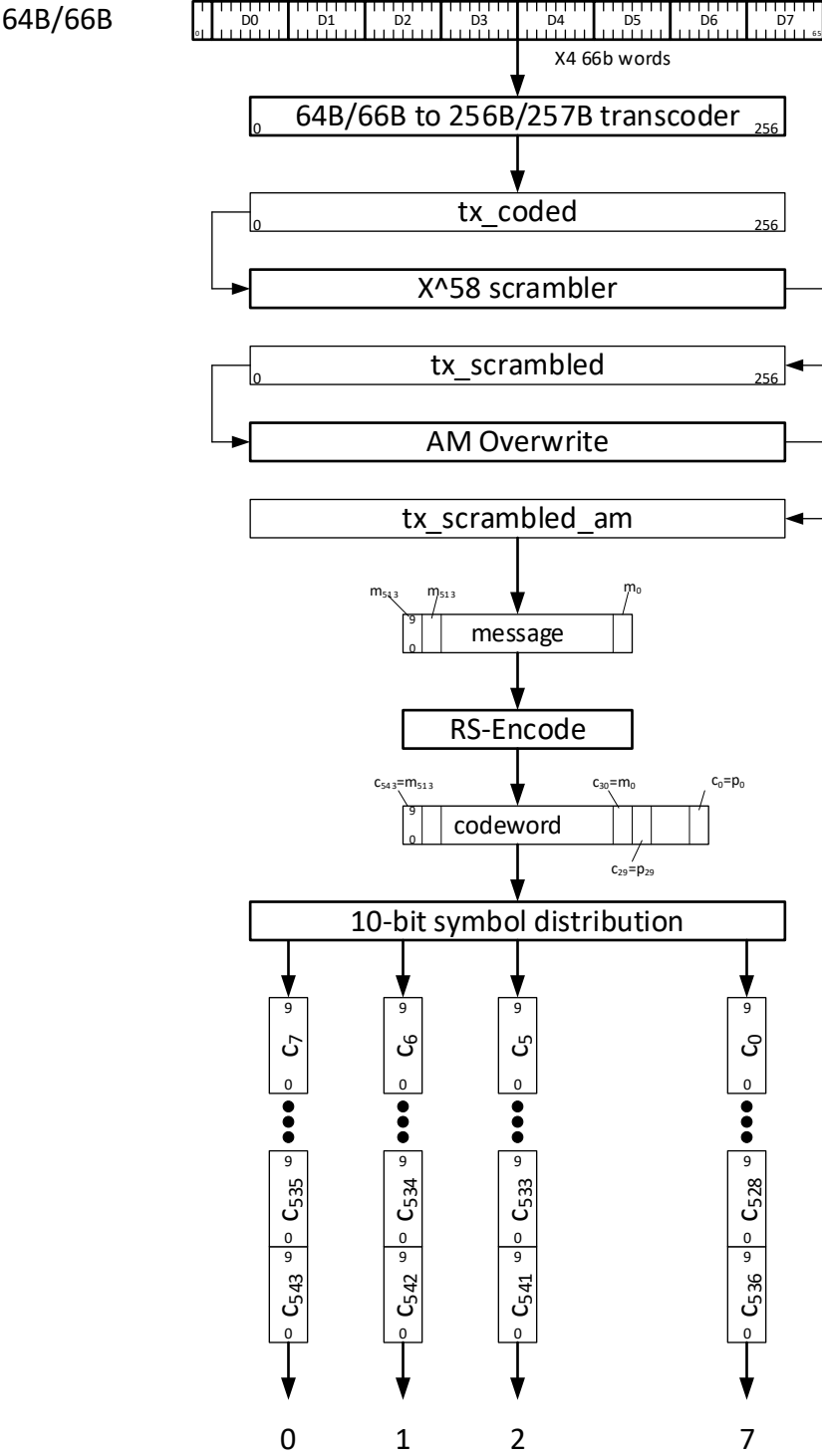


Figure 3-7 200GBASE-KR1/CR1, 1-Way Interleave

3.7.3.2 PCS Receive changes

The following Clauses of 802.3 are modified:

- Clause 119.2.5.2 Lane reorder and de-interleave, de-interleave is not performed, and the single stream is sent to a single FEC decode.

- Clause 119.2.5.4 Post FEC interleave is not performed, there is a single stream from the FEC decode.

### 3.8 Link Resiliency

#### 3.8.1 Transmit Alignment

The Link Resiliency capability is primarily a Data Link feature and is described in section 2.8. In order to optimize the transmit mux buffer latency in the DL, the attached physical layers shall support the following:

- The code word alignment (and thus DL Flit alignment) between physical layers is staggered by 50%. Physical layer A is first.
- The insertion of alignment markers occurs at the same time, plus the 50% stagger offset above.

The alignment between the two physical layers on transmitter is shown below. The AM represent alignment marker code word; control indicates Idle control blocks for rate matching. DL Flit indicates data from the DL, either NOP Flits or payload Flits.

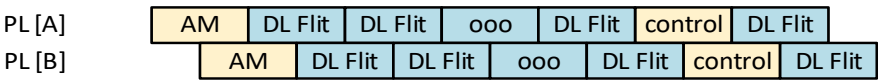


Figure 3-8 Code Word Alignment

#### 3.8.2 Physical Layer ID

To facilitate reorder at the DL, see section 2.8.4, the PL shall transmit the PL ID in the Control Flits that carry the Alignment Markers. This is described in sections 3.2.4.2 and 3.2.4.4. PL[A] shall have a PL ID 0, and PL[B] shall have a PL ID of 1. When the RS receives the AM Flit it signals to the DL what the PL ID is. The rules are as follows:

- When the PL receives an FEC corrected codeword that contains the alignment marker the RS indicates the PL ID to the DL.

## 4 Appendices

### Appendix A DL CRC Example (informative)

A cyclic redundancy check (CRC) is used by the transmit and receive logic to generate a 32-bit CRC value. This value is computed as a function of the entire contents of the 640-byte DL Flit, with the CRC field zero'd out. Zeroing out the CRC field for the calculation simplifies the implementation for wide data paths, typically 64-bytes wide at 800G. The encoding is defined by the following generating polynomial.

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

Mathematically, the CRC value corresponding to a given DL Flit is defined by the following procedure:

The first 32 bits of the DL Flit are complemented.

The  $n$  bits of the DL Flit, including zero'd CRC field, are then considered to be the coefficients of a polynomial  $M(x)$  of degree  $n - 1$ . The first bit of the S0.B0 corresponds to the  $x(n-1)$  term and the last bit of the zero'd CRC field corresponds to the  $x_0$  term.

$M(x)$  is multiplied by  $x^{32}$  and divided by  $G(x)$ , producing a remainder  $R(x)$  of degree  $\leq 31$ .

The coefficients of  $R(x)$  are considered to be a 32-bit sequence.

The bit sequence is complemented, and the result is the CRC.

The 32 bits of the CRC value are placed in the CRC field so that the CRC is transmitted in the order  $x^0, x^1, \dots, x^{30}, x^{31}$ . Note that this is an exact bit reversal of the CRC as it would be transmitted for IEEE 802.3 FCS.

For example, for a Flit with a Flit header of 0h4FFF00 and all-zero data payload, i.e., op = 0b010 (Ack), payload=0 (NOP), and ackReqSeq=0h1FF, the CRC calculated following IEEE 802.3 FCS would be 0h4009BC57. After reversing the bit order, 0hEA3D9002 is transmitted in the following order, 1110 1010 0011 1101 1001 0000 0000 0010.

## Appendix B DL to PL Interface (informative)

This section is informative. The purpose is to provide readers with guidance of the signaling interface between the DL-PL layers. The interface is described in a TDM fashion, to be consistent with the UPLI interface.

The interface is 64-Bytes wide. The interface typically runs at ~1.66 GHz, derived from the Ethernet 156.25 MHz local reference clock.

### B.1 Tx Signals

The interface is TDM. If the interface is configured for 4 ports the \*\_port\_id[1:0] counts from 0 to 3, and repeats. Similarly, if the interface is configured for 2 ports the \*\_port\_id[1:0] counts from 0 to 1, and repeats. For a single port the \*\_port\_id[1:0] is static at 0.

When operating without Link Resiliency, i.e., each DL is mapped to a single PL. The portID TDM calendar is:

- 0, 1, 2, 3, 0, 1... with 4 ports
- 0, 1, 0, 1, 0, 1... with two ports
- 0, 0, 0, 0, 0, 0... for a single port

When operating with Link Resiliency, i.e., each DL is mapped to two PLs. The portID TDM calendar is:

- 0, 2, 1, 3, 0, 2... with two DLs
- 0, 1, 0, 1, 0, 1... for a single DL

The pattern 0, 2, 1, 3 repeating for two DLs improves latency in the DL mux/demux between each DL and two PLs.

All signals sourced from the DL are referenced according to tx\_dp\_port\_id[1:0]. Port 0 signals are sent when tx\_dp\_port\_id[1:0] = 0, etc. All signals sourced from the PL are referenced according to tx\_pd\_port\_id[1:0]. Port 0 signals are sent when tx\_pd\_port\_id[1:0] = 0, etc.

| Name               | Source | Description                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tx_dp_active       | DL     | Active high. Indicating that the DL layer is in an active state able to send DL Flits, i.e., the DL layer is "DL Up" or "DL NOP" state, and the PL does not indicate a fault condition.                                                                                                                                                                                                                                                                   |
| tx_dp_command[2:0] | DL     | Indicates a command to the PL. Encoded as follows: <ul style="list-style-type: none"> <li>• 000b: NOP, i.e. no command and normal operation</li> <li>• 001b: the DL instructs the PL to send PwrDn control Flits, regardless of RS fault status. Must be asserted on the same cycle as tx_dp_active is deasserted. May be deasserted at any time</li> <li>• 010b-111b: Reserved</li> </ul>                                                                |
| tx_dp_uerr         | DL     | Active high. Indicating that the DL layer has encountered an unrecoverable error condition and is no longer able to send reliable data. This signal is asserted in the same cycle that tx_dp_active is deasserted. The DL transitions to DL Idle. The PL transmits Idle code blocks. The transition to transmitting Idle control blocks occurs on the assertion of this signal and may not be on a Flit boundary. When asserted this must remain asserted |

| Name               | Source | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |        | for at least 100ms, to ensure both link partners exit DL Up. Management S/W may then clear this bit and re-enable the DL Idle to DL NOP on both link partners. When this bit is deasserted the RS returns to normal operation on the next convenient Flit boundary.                                                                                                                                                                                                                                                                                                                                                                        |
| tx_dp_valid        | DL     | Active high. Indicating that the data on tx_dp_data[511:0] and tx_dp_flit_start is valid. When tx_pd_ready is asserted, the DL layer must assert tx_dp_valid after a fixed number of clock cycles following the assertion on tx_pd_ready, in accordance with a predefined implementation specific timing relationship. If tx_pd_ready is not asserted, then tx_dp_valid must not be asserted in accordance with the same timing relationship.                                                                                                                                                                                              |
| tx_dp_flit_start   | DL     | Active high. When tx_dp_valid is asserted indicates the start of a DL Flit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| tx_dp_port_id[1:0] | DL     | TDM signal indicates the port number for transmit signals sourced from the DL.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| tx_dp_data[511:0]  | DL     | When tx_dp_valid is asserted, this contains valid DL flit data. [511:504] is the first byte to the wire and [7:0] is the last byte to the wire. Bit [504] is the first bit to the wire and bit [7] is last bit to the wire.                                                                                                                                                                                                                                                                                                                                                                                                                |
| tx_pd_ready        | PL     | Active high signal. Indicating that the PL layer is ready to accept data from the DL layer, and the DL layer must supply it, see tx_dp_valid.<br><br>The PL shall not assert this signal unless tx_dp_active is asserted. If tx_dp_active is deasserted this signal shall be deasserted such that it occurs on a Flit boundary, see Tx Waveforms.                                                                                                                                                                                                                                                                                          |
| tx_pd_port_id[1:0] | PL     | TDM signal indicating the port number for transmit signals sourced from the PL.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| tx_pd_group_start  | PL     | Active high signal. Indicating the start of a group of 4 or 2 or 1 Flits. The group of Flits shall be packed into a code word group by the physical layer. Group size is a function of FEC code word interleave. Qualified by tx_pd_ready. This signal shall only be asserted when tx_pd_flit_start is asserted. The link layer replay sends three identical replay requests for the same event; these replay requests shall not be placed in the same code word interleave group. Burst errors that cannot be corrected by FEC in one code word, are highly correlated such that all code words in that group cannot be corrected by FEC. |
| tx_pd_flit_start   | PL     | Active high signal. Indicating the start of Flit is requested. Qualified by tx_pd_ready.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

Table Appendix B-1 Transmit DL to PL Data Path Signals

## B.2 Tx Waveforms

The timing diagram below shows the basic timing with TDM. This example assumes 4 ports. Port 1 and Port 3 assert `tx_pd_ready` as shown. `Tx_pd_group_start` and `tx_pd_flit_start` are asserted as applicable and latched by the DL.

Five cycles later, from `tx_pd_ready`, Port 1 must assert `tx_dp_valid`, on that clock tick and `tx_dp_data`, `tx_dp_flit_start` are qualified and latched by the PL. Every TDM cycle Port 1 latches `tx_dp_uerr`, and `tx_dp_active`, they are not qualified by `tx_dp_valid`.

Five cycles later Port 3 must also assert `tx_dp_valid`, on that clock tick and `tx_dp_data`, `tx_dp_flit_start` are qualified and latched by the PL. Every TDM cycle Port 3 latches `tx_dp_uerr`, and `tx_dp_active`, they are not qualified by `tx_dp_valid`.

The offset between `tx_pd_port_id` and `tx_dp_port_id` is implementation specific and is set to five in this example.

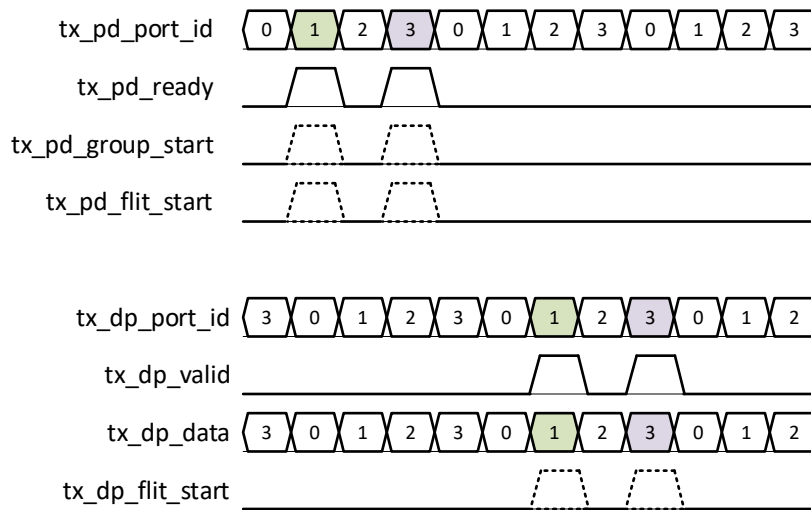


Figure Appendix B-1 Tx TDM timing qualified by valid

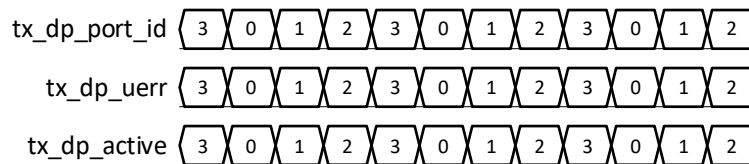


Figure Appendix B-2 Tx TDM timing (not qualified by valid)

The diagram below is shown as a single port for simplicity. This illustrates the DL asserting `tx_dp_active`, i.e. the DL Link state machine has reached DL NOP and is able to send DL Flits. There will be several ticks of delay before the PL asserts `tx_pd_ready`, it will need to align this such that the `tx_dp_valid` asserts on a Flit boundary, i.e. at the end of a control Flit that the RS is currently sending.

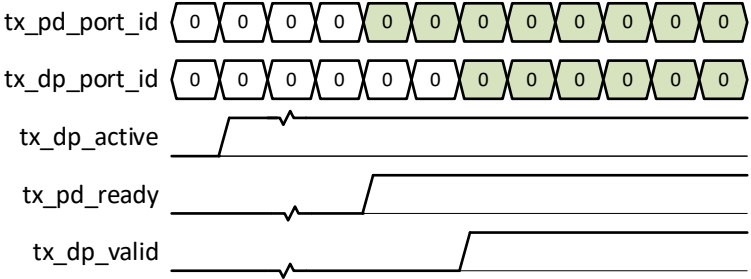


Figure Appendix B-3 Active Assertion Timing

The diagram below is shown as a single port for simplicity. In this example there are three ticks between ready and valid. This illustrates the DL de-asserting tx\_dp\_active, i.e. the DL Link state machine has exited DL Up and is not able to send DL Flits. There may be several ticks of delay before the PL de-asserts tx\_pd\_ready. The signal tx\_pd\_ready shall de-asserts on a Flit boundary, i.e. when tx\_pd\_flit\_start would assert. PL/DL beat count is an internal counter in the PL/DL that tracks how many 64-byte beats of data have been received/transmitted.

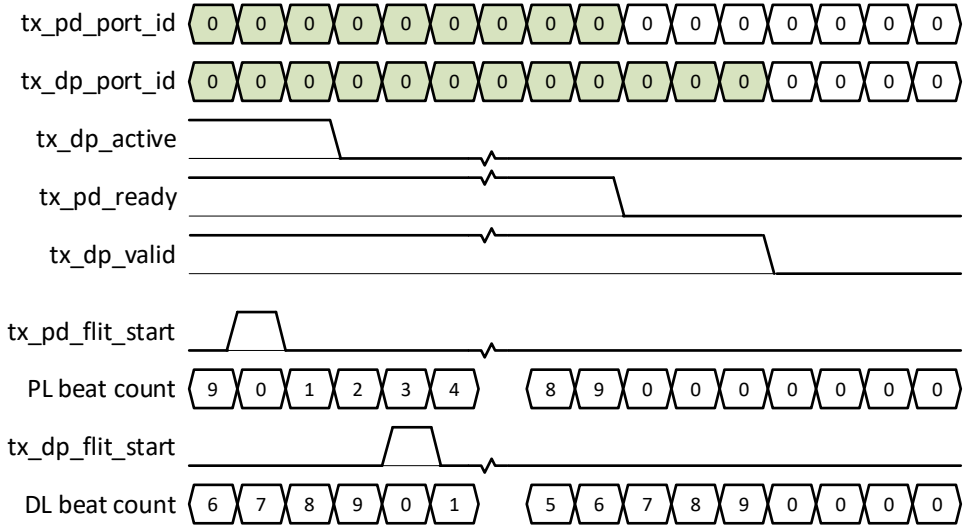


Figure Appendix B-4 Active de-assertion

The diagram below is shown as a single port for simplicity. In this example there are three ticks between ready and valid. This illustrates the DL entering the DL PwrDn state. The DL de-asserts tx\_dp\_active on the same tick as tx\_dp\_command[2:0] = 1, instructs the PL to send PwrDn control Flits. There may be several ticks of delay before the PL de-asserts tx\_pd\_ready. The signal tx\_pd\_ready shall de-asserts on a Flit boundary, i.e. when tx\_pd\_flit\_start would assert. PL/DL beat count is an internal counter in the PL/DL that tracks how many 64-byte beats of data have been received/transmitted.

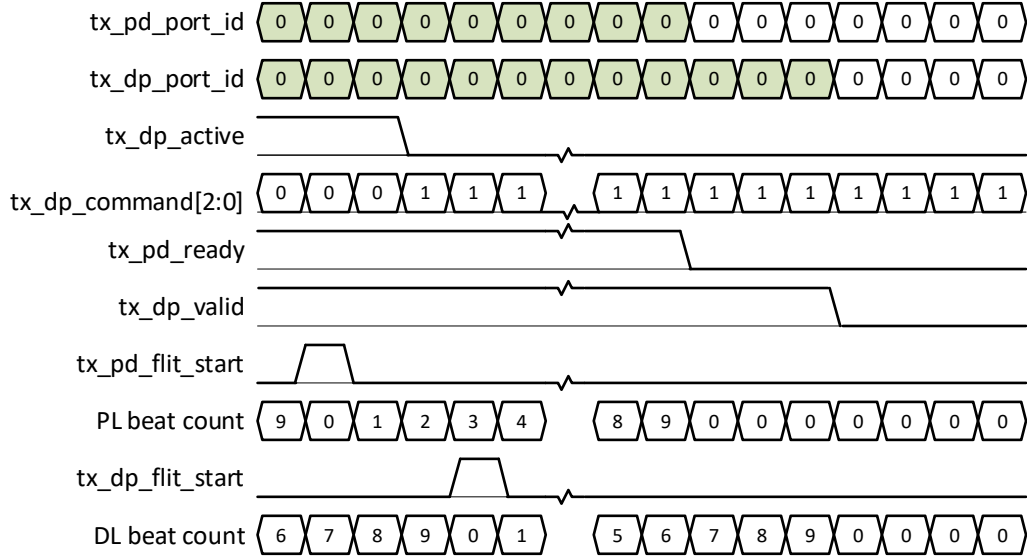


Figure Appendix B-5 DL Enters PwrDn State

The diagram below is shown as a single port for simplicity. In this example there are three ticks between ready and valid. This illustrates the DL in the DL Up state, and the Rx detects a local fault,  $rx\_pd\_link\_event[3:0] = 0xF$ , and the DL remain is DL Up. The DL de-asserts  $tx\_dp\_active$  on the next tick of the local fault indication. There may be several ticks of delay before the PL de-asserts  $tx\_pd\_ready$ . The signal  $tx\_pd\_ready$  shall de-asserts on a Flit boundary, i.e. when  $tx\_pd\_flit\_start$  would assert. PL/DL beat count is an internal counter in the PL/DL that tracks how many 64-byte beats of data have been received/transmitted.

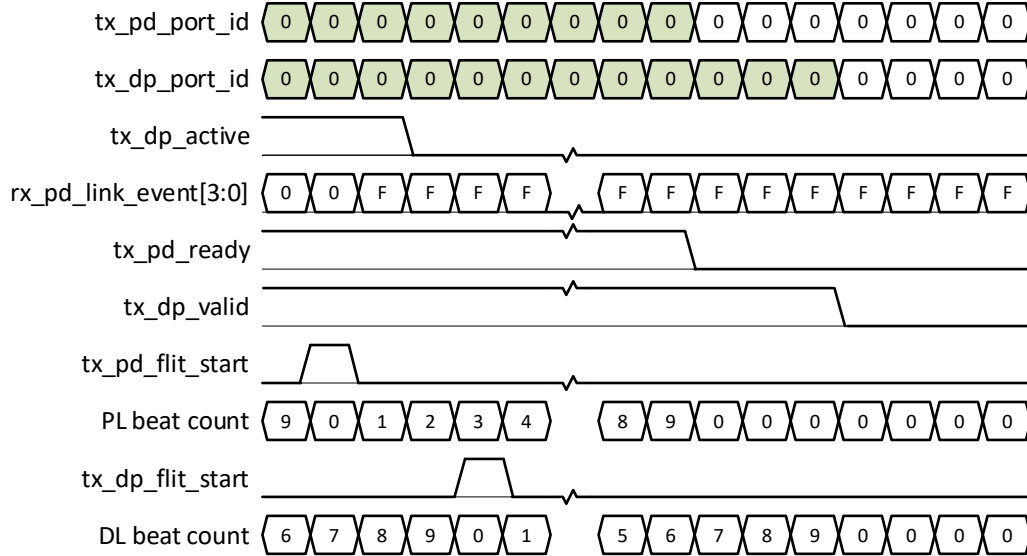


Figure Appendix B-6 PL Local Fault Detected

### B.3 Tx TDM Initialization

There is no requirement that the PL and DL exit reset at the same time. The PL dictates the TDM cycle via  $tx\_pd\_port\_id$ . The DL synchronizes its TDM cycle to this and adjusts  $tx\_dp\_port\_id$  as needed. The DL will have an implementation specific delay from receiving  $tx\_pd\_ready$  to

generating tx\_dp\_valid, ready2valid. For example, with four ports and a ready2valid delay of 9,  $\text{mod}(9,4) = 1$ , therefore the tx\_dp\_port\_id shall be 1 cycle delayed from tx\_pd\_port\_id. The PL shall support a register setting that configures the expected ready2valid delay. The PL shall support a max delay of 128 clock cycles.

## B.4 Rx Signals

The interface is TDM. If the interface is configured for 4 ports the \*\_port\_id[1:0] counts from 0 to 3, and repeats. Similarly, if the interface is configured for 2 ports the \*\_port\_id[1:0] counts from 0 to 1, and repeats. For a single port the \*\_port\_id[1:0] is static at 0.

Refer to section 1.1.1 for TDM mapping with and without Link Resiliency.

All signals sourced from the PL are referenced according to rx\_pd\_port\_id[1:0]. Port 0 signals are sent when rx\_pd\_port\_id[1:0] = 0, etc.

Ready is not used with this interface.

| Name               | Source | Description                                                                                                                                                                                                                         |
|--------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rx_pd_valid        | PL     | Active high. Indicating that rx_pd_flit_start and rx_pd_data is valid.                                                                                                                                                              |
| rx_pd_flit_start   | PL     | Active high. When rx_pd_valid is asserted indicates the start of a DL Flit. Determined when the first eight 64B/66B codes are data blocks.                                                                                          |
| rx_pd_port_id[1:0] | PL     | TDM signal indicating the port number for receive signals sourced from the PL.                                                                                                                                                      |
| rx_pd_data[511:0]  | PL     | When rx_pd_valid is asserted, this contains valid DL flit data. [511:504] is the first byte from the wire and [7:0] is the last byte from the wire. Bit [504] is the first bit from the wire and bit [7] is last bit from the wire. |

| Name                  | Source | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rx_pd_link_event[3:0] | PL     | <p>This signal indicates that the RS layer has detected a link Event.</p> <p>The encoded values are defined as:</p> <ul style="list-style-type: none"> <li>• 0000b: NOP</li> <li>• 0001b: Idle Control Flit Received</li> <li>• 0010b: Remote Fault Control Flit Received</li> <li>• 0011b: PwrDn Control Flit Received</li> <li>• 0100b: AM/RAM Control Flit Received with PL ID 0</li> <li>• 0101b: AM/RAM Control Flit Received with PL ID 1</li> <li>• 0110b: AM/RAM Control Flit Received with no PL ID</li> <li>• 0111b-1101b reserved</li> <li>• 1110b: Bad Flit received</li> <li>• 1111b: Local Fault detected</li> </ul> <p>Local fault detected code is sent for the duration of the event, this may be asserted or deasserted at any time. Control Flits or Bad Flits received are pulsed for a single cycle after the Flit has been received by the RS, see B.7. Local Fault takes precedence when asserted no other events are reported.</p> <p>The encoding is determined as follows:</p> <ul style="list-style-type: none"> <li>• If the first eight 64B/66B codes are data blocks, then NOP code is indicated.</li> <li>• Else If all control blocks match one of defined Control Flits, and FEC correction is successful then pulse associated code.</li> <li>• Else the Bad Flit code is pulsed. The DL shall treat this as a DL Flit with a bad CRC i.e., update Rx_bad_crc_count.</li> </ul> <p>The NOP code is sent when nothing else is indicated.</p> <p>Only when operating link resiliency is AM Flit with PL ID indicated, else no PL ID is indicated.</p> <p>If a local fault detected code is received, then the DL layer shall discard any associated DL Flit in progress.</p> |
| rx_pd_data_err        | PL     | Active high. Indicates data error. Asserted on the following beat after the last tick of a DL Flit indicating that the Flit failed FEC correction, and thus the DL shall discard. Also asserted if not all 64B/66B blocks received were data blocks.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| rx_pd_uerr            | PL     | Active high, unrecoverable error.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

Table Appendix B-2 DL to PL Data Path Signals

## B.5 Rx Waveforms

The basic TDM timing diagram is shown below. This example assumes 4 ports.

Port 1 asserts rx\_pd\_valid in its time slot, and rx\_pd\_data, rx\_pd\_flit\_start, rx\_pd\_data\_err are qualified and latched by the DL. Every TDM Port-1 time slot, the DL latches rx\_pd\_uerr, and rx\_pd\_link\_fault, they are not qualified by rx\_pd\_valid.

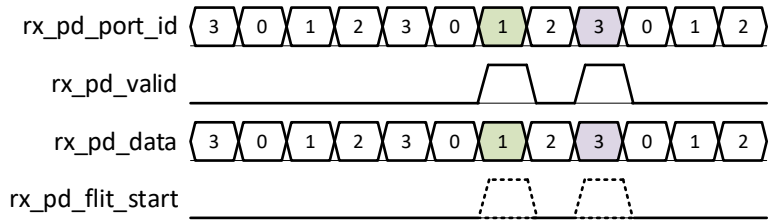


Figure Appendix B-7 Rx TDM timing qualified by valid

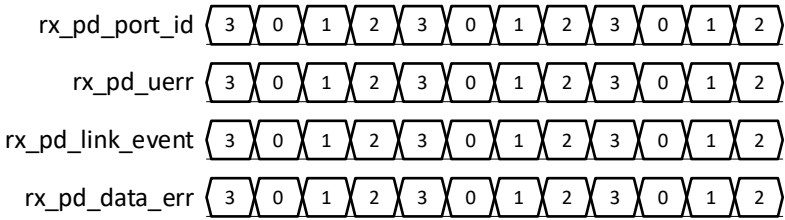


Figure Appendix B-8 Rx TDM timing (not qualified by valid)

B.6 Start Up Waveform

A typical start up scenario is shown below, as a single port for simplicity. In this example there are three ticks between ready and valid. The receiver starts receiving Idle control Flits, as indicated by the 001 rx\_pd\_link\_event code on the next beat after the code word was received. Then a DL Flit is received, indicated by rx\_pd\_valid, and rx\_pd\_flit\_start. The DL state machine transitions to DL NOP, its receiving Ilde, and F/W enables the transition. The DL asserts tx\_dp\_active indicating that it can source DL Flits. The PL then asserts tx\_pd\_ready and tx\_pd\_flit\_start considering the ready2valid delay so that the DL asserts tx\_dp\_valid and tx\_dp\_flit\_start to align with the PL’s Flit transmit timing.

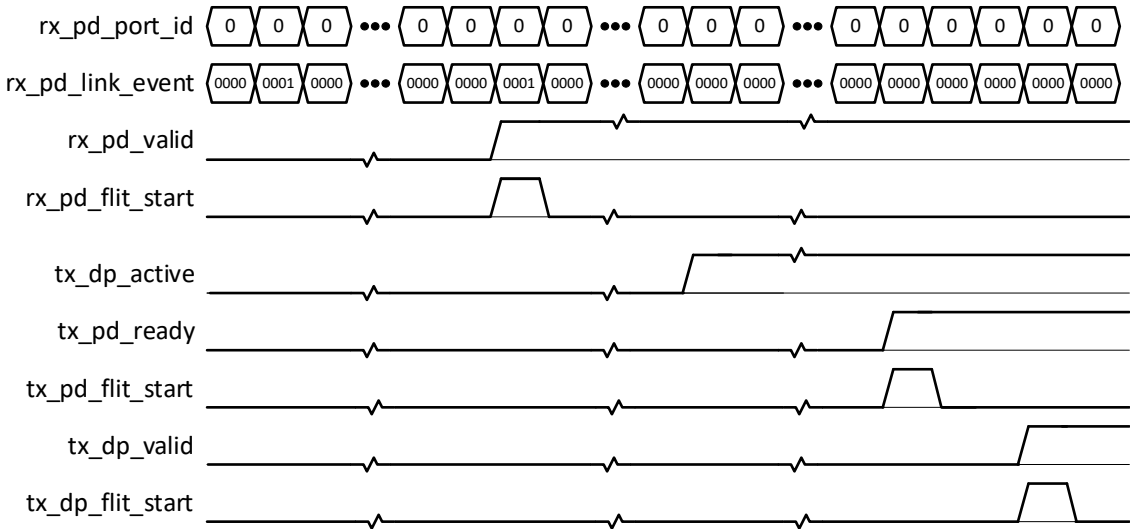


Figure Appendix B-9 Start Up Timing

B.7 Link Event and Error Timing

The last data beats of a DL Flit are shown below, beats 6, 7, 8, 9 for port\_id 0. The error signal rx\_pd\_data\_err asserts on the next rx\_pd\_port\_id cycle. Rx\_pd\_valid does not assert in the case of the next cycle assertion, as rx\_pd\_valid indicates if rx\_pd\_data is valid.

Rx\_pd\_link\_event asserts in the same relative location as the rx\_pd\_data\_error, i.e. on cycle after the RS would have sent the last beat of a DL Flit, had the received codeword decoded to a DL Flit.

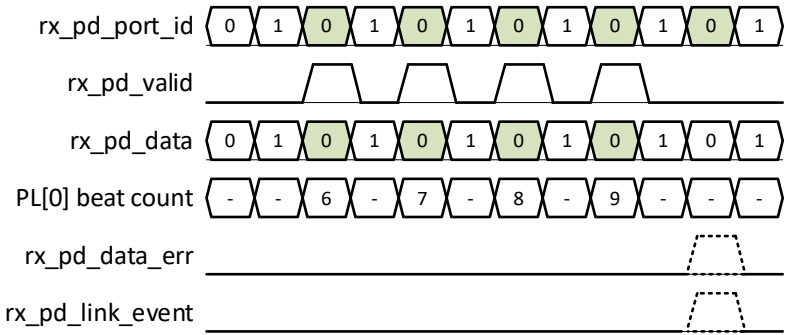


Figure Appendix B-10 Link Event and Error